SISTEMAS OPERATIVOS

UNIDAD II

Contenidos

Ejecución de instrucciones e interrupciones. 4
Búsqueda y ejecución de una instrucción. 4
Interrupciones. 6
Referencias y bibliografía de la sección. 11
Definición de proceso y su representación. 11
Más sobre el modelo del proceso. 13
Referencias y bibliografía de la sección. 17
Estados del proceso. 17
Creación y terminación de un proceso. 23
Referencias y bibliografía de la sección. 25
Bloque de control de proceso. 25
Referencias y bibliografía de la sección. 30
Estructuras de control del sistema operativo. 30
Referencias y bibliografía de la sección. 33
Definición de hilo. 33
Estados de los hilos. 36
Uso de hilos. 37
El modelo clásico de hilo. 41
Hilos en POSIX. 46
Referencias y bibliografía de la sección. 48
Implementación de hilos a nivel núcleo y a nivel usuario. 48
Implementación de hilos en el espacio de usuario. 49
Implementación de hilos en el kernel. 53
Implementaciones híbridas/combinadas. 56
Relación entre ULT y KLT. 57
Modelo Mx1 (Many to one). 58
Modelo 1x1 (One to one). 58
Modelo MxM (Many to many). 59
Referencias y bibliografía de la sección. 59
Comunicación entre procesos. 59

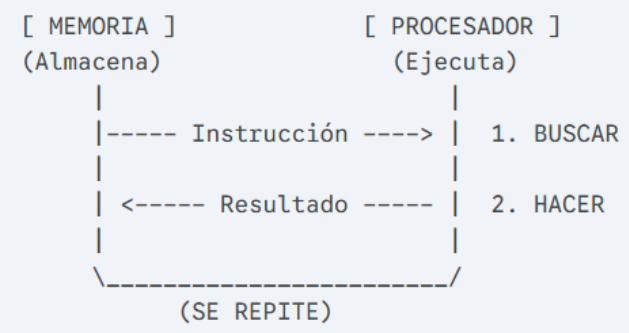
Sistemas de memoria compartida. 62
Sistemas de paso de mensajes. 63
Nombrado. 65
Sincronización. 67
Almacenamiento en búfer. 67
Referencias y bibliografía de la sección. 68
Más herramientas de sincronización y comunicación en UNIX. 68
Tuberías. 68
Mensajes. 69
Memoria compartida. 70
Señales. 70
Referencias y bibliografía de la sección. 72
Competencia entre procesos. 72
Mas sobre exclusión mutua. 76
Condición de carrera. 76
Referencias y bibliografía de la sección. 77
Sección crítica. 77
Referencias y bibliografía de la sección. 81
Intentos de solución. 81
Deshabilitar interrupciones. 81
Cerrojos y hardware de sincronización. 82
Solución de Peterson. 86
Semáforos. 88
Utilización. 89
Implementación 91
Monitores. 92
Desventajas del modelo de monitor. 97
Espera activa e inactiva. 98
Referencias y bibliografía de la sección. 99
Otros problemas de comunicación y sincronización entre procesos. 100
El problema de los filósofos comensales. 100
El problema de los lectores y escritores. 104
El problema productor/consumidor. 106
Problema del búfer limitado. 110
Interbloqueos e inanición. 111
Referencias y bibliografía de la sección. 112

Planificación del uso de la CPU. 112
Conceptos básicos. 112
Ciclo de ráfagas de CPU y de E/S. 113
Planificador de la CPU. 114
Planificación apropiativa y cooperativa. 115
El despachador. 117
Criterios de planificación. 117
Mas sobre planificación. 118
Cuando planificar procesos. 119
Categorías de los algoritmos de planificación. 120
Planificación de hilos. 121
Niveles de planificación. 123
Referencias y bibliografía de la sección. 124
Algoritmos de planificación. 124
Planificación FCFS. 124
Planificación SJF. 126
Planificación por prioridades. 128
Planificación por turnos. 131
Planificación mediante colas multinivel. 133
Referencias y bibliografía de la sección. 134
Planificación de sistemas multiprocesador. 135
Métodos de planificación en los sistemas multiprocesador. 135
Referencias y bibliografía de la sección. 135
Hardware de multiprocesador. 135
Multiprocesadores UMA con arquitecturas basadas en bus. 136
Multiprocesadores NUMA. 137
Chips multinúcleo. 137
Referencias y bibliografía de la sección. 138

Ejecución de instrucciones e interrupciones.

Búsqueda y ejecución de una instrucción.

Un programa que va a ejecutarse en un procesador consta de un conjunto de ***instrucciones*** almacenado en memoria. En su forma más simple, el procesamiento de una instrucción consta de dos pasos: el procesador lee (busca) instrucciones de la memoria, una cada vez, y ejecuta cada una de ellas. La ejecución del programa consiste en repetir el mecanismo de búsqueda y ejecución de instrucciones.



La ejecución de la instrucción puede involucrar varias operaciones dependiendo de la naturaleza de la misma.

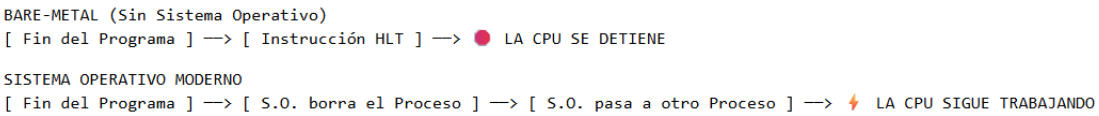
Se denomina ***ciclo de instrucción*** al procesamiento requerido por una única instrucción. Un ciclo de instrucción utilizando la descripción simplificada de dos pasos consta de la ***fase de búsqueda*** y ***fase de ejecución***.

La ejecución del programa se detiene sólo si se apaga la máquina, se produce algún tipo de error irrecuperable o se ejecuta una instrucción del programa que detiene/para al procesador.

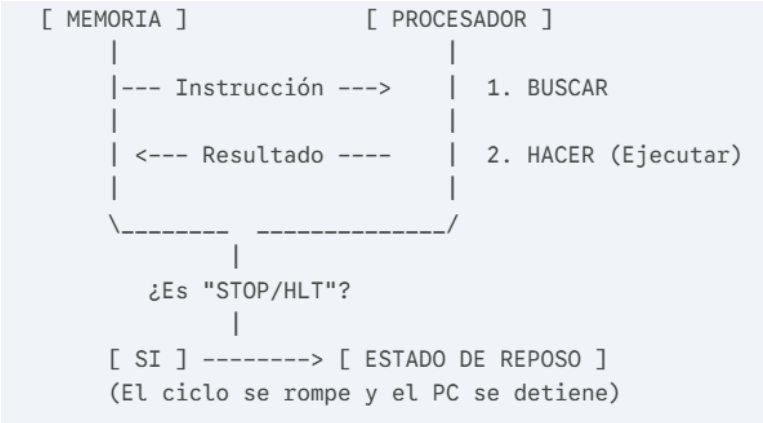
Esto de “detener” al procesador en realidad significa que se deja de ejecutar el ciclo infinito de buscar y procesar nuevas instrucciones de la memoria.

A nivel de hardware puro (bare-metal), esto se logra mediante instrucciones de máquina específicas como HLT (halt) o STOP, las cuales suspenden el avance del contador de programa y dejan a la CPU en estado de reposo.

Vale aclarar que esta última idea se enfoca exclusivamente en la perspectiva del ciclo de instrucción a nivel de hardware puro (bare-metal). Para un programa de usuario en un sistema operativo moderno, la finalización del programa detiene la ejecución de ese proceso, pero no detiene a la CPU. En otras palabras, la afirmación anterior aplica al ciclo de instrucción básico de la CPU y no a la finalización de un proceso de usuario.



En lenguajes de alto nivel como C, la finalización del main() no genera una instrucción HLT, ya que es una instrucción privilegiada, sino que invoca una función del entorno de ejecución que realiza una llamada al sistema (exit) para devolver el control al sistema operativo.



Al principio de cada ciclo de instrucción, el procesador lee una instrucción de la memoria. En un procesador típico, el ***contador del programa*** (PC) almacena la dirección de la siguiente instrucción que se va a leer. A menos que se le indique otra cosa, el procesador siempre incrementa el PC después de cada instrucción ejecutada, de manera que se leerá la siguiente instrucción en orden secuencial (es decir, la instrucción situada en la siguiente dirección de memoria más alta).

La instrucción leída se carga dentro de un registro del procesador conocido como ***registro de instrucción*** (IR). La instrucción contiene bits que especifican la acción que debe realizar el procesador.

El procesador interpreta la instrucción y lleva a cabo la acción requerida. En general, estas acciones se dividen en cuatro categorías:

- **Procesador-memoria.** Se pueden transferir datos desde el procesador a la memoria o viceversa.
- **Procesador-E/S.** Se pueden enviar datos a un dispositivo periférico o recibirlos desde el mismo, transfiriéndolos entre el procesador y un módulo de E/S.
- **Procesamiento de datos.** El procesador puede realizar algunas operaciones aritméticas o lógicas sobre los datos.
- **Control.** Una instrucción puede especificar que se va a alterar la secuencia de ejecución. Por ejemplo, el procesador puede leer una instrucción de la posición 149, que especifica que la siguiente instrucción estará en la posición 182. El procesador almacenará en el contador del programa un valor de 182. Como consecuencia, en la siguiente fase de búsqueda, se leerá la instrucción de la posición 182 en vez de la 150. Dicho de otra forma, cuando aparece una instrucción de Control, el procesador recibe una orden tipo: "Ignorar la que sigue, saltar a la 182". Permite que el software tenga lógica, bucles y decisiones.

1. Procesador-Memoria	Mueve datos (Carga/Guarda).	`CPU` ↔ `RAM`
2. Procesador-E/S	Habla con el exterior.	`CPU` ↔ `Teclado/Pantalla`
3. Procesamiento	Cálculos en la **ALU**.	`Dato A` + `Dato B` = `Resultado`
4. Control	Cambia el **PC** (Saltos).	`PC: 149` → `PC: 182`

Interrupciones.

Anteriormente se dijo que el procesador solo se detiene (muere o entra en reposo) con un ***HLT***, un error fatal o al apagarse. Las ***interrupciones***, en cambio, no detienen el procesador, más bien lo desvían o pausan.

Prácticamente todos los computadores proporcionan un mecanismo por el cual otros módulos (memoria y E/S) pueden interrumpir la secuenciación normal del procesador. La siguiente tabla detalla los tipos más comunes de interrupciones.

De programa	Generada por alguna condición que se produce como resultado de la ejecución de una instrucción, tales como un desbordamiento aritmético, una división por cero, un intento de ejecutar una instrucción de máquina ilegal, y las referencias fuera del espacio de la memoria permitido para un usuario.
Por temporizador	Generada por un temporizador del procesador. Permite al sistema operativo realizar ciertas funciones de forma regular.
De E/S	Generada por un controlador de E/S para señalar la conclusión normal de una operación o para indicar diversas condiciones de error.
Por fallo del hardware	Generada por un fallo, como un fallo en el suministro de energía o un error de paridad en la memoria.

Básicamente, las interrupciones constituyen una manera de mejorar la utilización del procesador. Por ejemplo, la mayoría de los dispositivos de E/S son mucho más lentos que el procesador. Supongamos que el procesador está transfiriendo datos a una impresora utilizando el esquema de ciclo de instrucción de la figura siguiente.

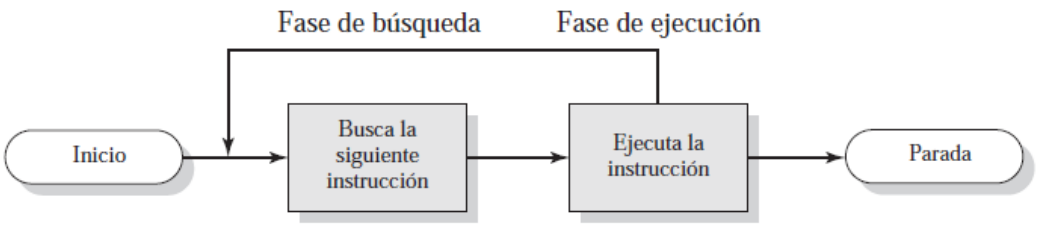


Ilustración 1 - Ciclo de instrucción básico.

Después de cada instrucción de escritura, el procesador debe parar y permanecer inactivo hasta que la impresora la lleve a cabo. La longitud de esta pausa puede ser del orden de muchos miles o incluso millones de ciclos de instrucción. Claramente, es un enorme desperdicio de la capacidad del procesador.

```
[ MEMORIA ]            [ PROCESADOR ]          [ DISPOSITIVO E/S ]
     |                        |                         |
     |-- Orden de Escritura ->|------------------------>| (Inicia)
     |                        |                         |
     |-- ¿Terminaste? <-------|--- Consulta estado ---->| "No"
     |-- ¿Terminaste? <-------|--- Consulta estado ---->| "No"
     |-- ¿Terminaste? <-------|--- Consulta estado ---->| "No"
     |                        |                         |
     |  ( Millones de ciclos perdidos en este bucle )   |
```

Gracias a las interrupciones, el procesador puede dedicarse a ejecutar otras instrucciones mientras la operación de E/S se está llevando a cabo.

De cara al programa de usuario, una interrupción suspende la secuencia normal de ejecución. Cuando se completa el procesamiento de la interrupción, se reanuda la ejecución.

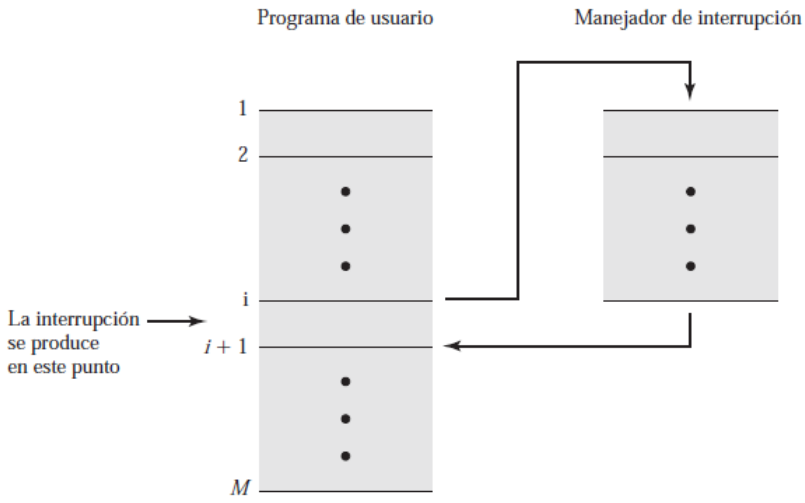


Ilustración 2 - Transferencia de control mediante interrupciones.

Por lo tanto, el programa de usuario no tiene que contener ningún código especial para tratar las interrupciones; el procesador y el sistema operativo son responsables de suspender el programa de usuario y, posteriormente, reanudarlo en el mismo punto.

Para tratar las *interrupciones*, se añade una ***fase de interrupción*** al ciclo de instrucción, como se muestra en la figura siguiente.

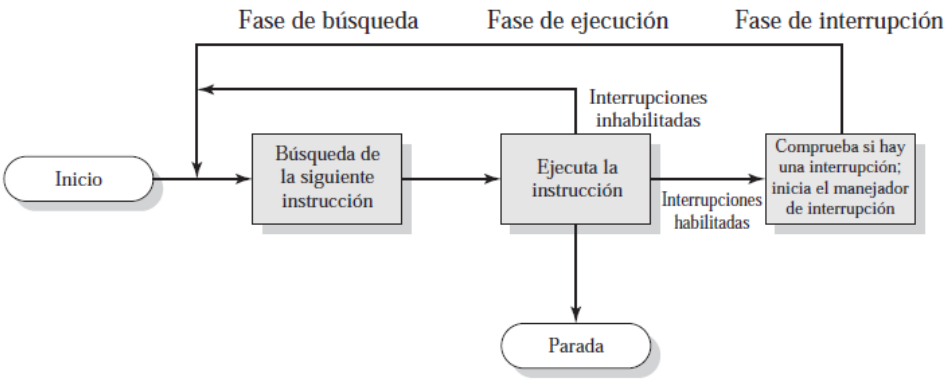


Ilustración 3 - Ciclo de instrucción con interrupciones.

En la fase de interrupción, el procesador comprueba si se ha producido cualquier interrupción, hecho indicado por la presencia de una señal de interrupción. Si no hay interrupciones pendientes, el procesador continúa con la fase de búsqueda y lee la siguiente instrucción del programa actual. Si está pendiente una interrupción, el procesador suspende la ejecución del programa actual y ejecuta la rutina del ***manejador de interrupción***.

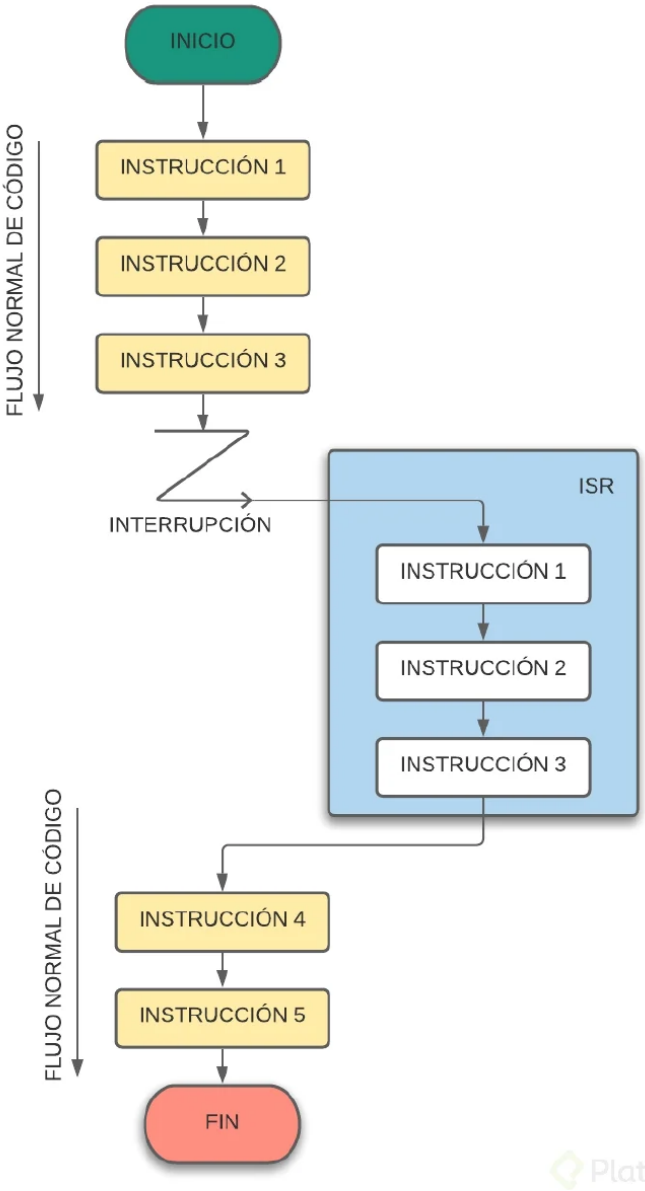
La rutina del manejador de interrupción es generalmente parte del sistema operativo. Normalmente, esta rutina determina la naturaleza de la interrupción y realiza las acciones que se requieran.

Es evidente que este proceso implica cierta sobrecarga. Deben ejecutarse instrucciones adicionales (en el manejador de interrupción) para determinar la naturaleza de la interrupción y decidir sobre la acción apropiada. Sin embargo, debido a la cantidad relativamente elevada de tiempo que se gastaría simplemente a la espera de una operación de E/S, el procesador se puede emplear mucho más eficientemente con el uso de interrupciones.

Queda claro que interrupciones son una de las formas más eficientes para manejar las solicitudes de servicios de dispositivos periféricos o eventos del sistema.

Las ***IRQ*** o Interrupt Request, en español Solicitud de Interrupción, son, en breves palabras, una señal de hardware enviada a la CPU para disparar una interrupción.

Una ***ISR*** o Interrupt Service Routine, en español Rutina de Servicio de Interrupciones es un bloque de código específicamente diseñado para trabajar con una condición de interrupción.



Tomando como base la imagen anterior, podemos resumir que una interrupción ocurre cuando la CPU se encuentra ejecutando una secuencia de instrucciones con el flujo normal de código, al recibir una IRQ se detendrá y guardará en memoria su contexto actual (incluyendo el contador de programa con la dirección de la siguiente instrucción), y a continuación, comenzará con la ejecución de una ISR específica para la interrupción disparada.

Término	Naturaleza	Función
IRQ	**Hardware** (Señal física).	Es la **señal** o aviso de que un componente externo requiere atención.
ISR	**Software** (Rutina de servicio).	Es la respuesta o el conjunto de instrucciones para procesar esa condición.

Si quisiéramos clasificar las interrupciones de acuerdo a su prioridad, podríamos diferenciar las interrupciones ***enmascarables*** y las interrupciones ***no enmascarables***.

- **Interrupciones enmascarables:** Son interrupciones que la CPU puede ignorar si se encuentra en un estado crítico en el que no podrá atender la interrupción, o simplemente ya se encuentra ejecutando una interrupción anterior.
- **Interrupciones no enmascarables:** Son interrupciones que no puede ignorar la CPU, siempre tienen que ejecutarse. Se reserva únicamente para eventos críticos y normalmente son reprogramadas por los fabricantes del chipset.

Las interrupciones de hardware son disparadas por eventos de hardware. Estos eventos pueden ser el presionar un botón o la expiración de un timer.

Se debe mencionar que las interrupciones también pueden llamarse por software. A diferencia de las interrupciones llamadas por hardware estas son completamente predecibles y forman parte de la secuencia del programa.

Característica	Interrupción de Hardware	Interrupción de Software (**TRAP**)
Origen	**Externo** (Periférico o Timer).	**Interno** (Instrucción del propio programa).
Propósito	Avisar al procesador de un evento del mundo físico.	Solicitar servicios al Kernel o manejar errores.

El TRAP es el mecanismo que el procesador usa para manejar eventos que ocurren "dentro" del flujo de ejecución.

Por otro lado, en la gran mayoría de los sistemas operativos (como Red Hat o Windows), a una interrupción por software también se la conoce como trap.

A menudo, se usa la mayúscula sostenida para referirse a la ***instrucción de máquina*** específica y la minúscula para el ***evento o mecanismo general***, lo cual puede generar confusión. Sin embargo, en la enorme mayoría de los libros de texto modernos TRAP y trap son sinónimos.

Entonces, un TRAP es una interrupción sincrónica. Esto significa que ocurre siempre en el mismo punto de tu programa cada vez que se ejecuta (es predecible).

El uso más común se da en el ámbito de las llamadas al sistema, cuando un programa de usuario no tiene permisos para acceder al hardware. El procesador genera un TRAP, salta al Kernel, este realiza la tarea y devuelve el control al programa. Es decir, TRAP es el "mecanismo de entrada" que permite que el sistema operativo tome el control para realizar el cambio de contexto.

Referencias y bibliografía de la sección.

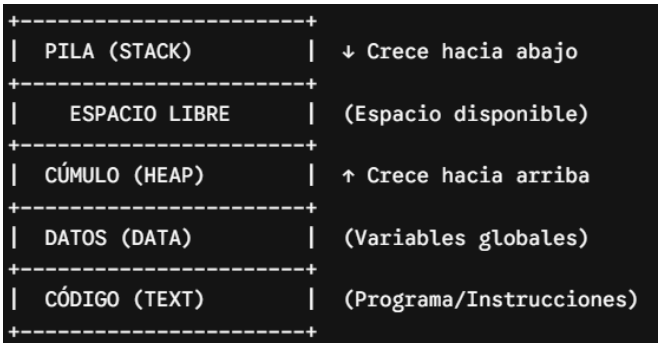
- Platzi - Conozcamos un poco más sobre Interrupciones.
- Stallings 1-3 / 1-4 pág. 14.

Definición de proceso y su representación.

Uno de los conceptos más importantes en cualquier sistema operativo es el de ***proceso***, una abstracción de un programa en ejecución; todo lo demás depende de este concepto. Por este motivo, es importante que se tenga una comprensión profunda acerca de lo que es un proceso. Entonces, un programa o comando que se ejecuta realmente en el sistema recibe la denominación de proceso.

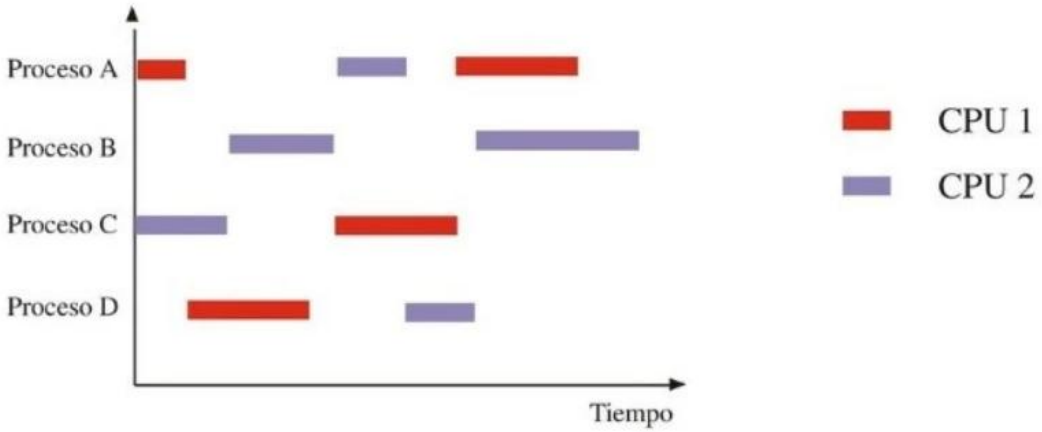
Los procesos son una de las abstracciones más antiguas y fundamentales que nos brindan los sistemas operativos: proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible. Convierten una CPU en varias CPU virtuales. Sin la abstracción de los procesos, la computación moderna no podría existir.

Generalmente, un proceso incluye la ***sección de código*** o texto (que contiene el código ejecutable), la ***pila*** del proceso (que contiene datos temporales como parámetros de funciones, direcciones de retorno y variables locales) y la ***sección de datos*** (que contiene las variables globales). El proceso puede incluir, asimismo, un cúmulo o ***sección de memoria*** (heap), que es la memoria asignada dinámicamente al proceso en tiempo de ejecución.



Todas las computadoras modernas ofrecen varias cosas al mismo tiempo; quienes están acostumbrados a trabajar con ellas tal vez no estén completamente conscientes de este hecho.

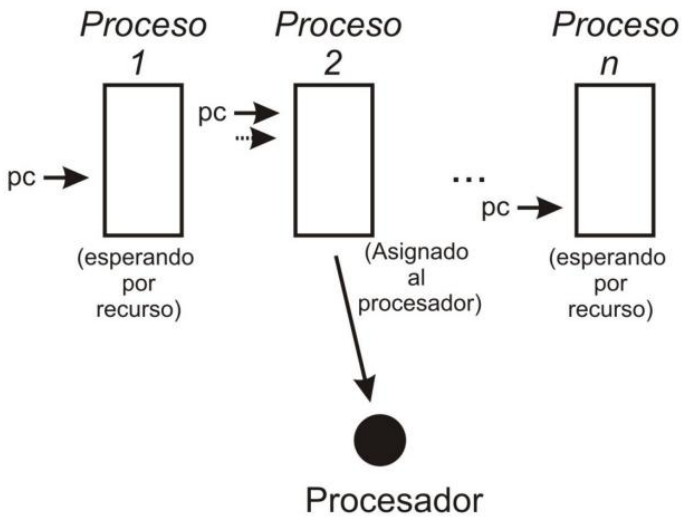
Consideremos primero un servidor Web, a donde convergen las peticiones de páginas Web provenientes de todos lados. Cuando llega una petición, el servidor verifica si la página que se necesita está en la caché. De ser así, devuelve la página; en caso contrario, inicia una petición al disco para obtenerla y, desde la perspectiva de la CPU, estas peticiones tardan eternidades. Mientras se espera el cumplimiento de una petición, muchas más pueden llegar. Si hay varios discos presentes, algunas o todas las demás peticiones podrían dirigirse a otros discos mucho antes de que se cumpla la primera petición.



Es evidente que se necesita cierta forma de modelar y controlar esta concurrencia. Los procesos (y en especial los ***hilos***) pueden ayudar en este caso.

Ahora consideremos una PC de usuario. Cuando arranca el sistema, se inician muchos procesos en segundo plano (de manera transparente al usuario). Por ejemplo, se podría iniciar un proceso para esperar el correo electrónico entrante; otro que permite al antivirus comprobar periódicamente la disponibilidad de nuevas definiciones de virus; algunos procesos de usuario explícitos como un proceso que imprime archivos, y todo esto mientras el usuario navega por la Web.

Toda esta actividad se tiene que administrar, y en este caso un sistema de multiprogramación con soporte para múltiples procesos es muy útil.



En cualquier sistema de multiprogramación, la CPU conmuta de un proceso a otro con rapidez, ejecutando cada uno durante décimas o centésimas de milisegundos: Hablando en sentido estricto, en cualquier instante una CPU está ejecutando solo un proceso, y en el transcurso de 1 segundo podría trabajar en varios de ellos, dando la apariencia de un paralelismo (o pseudoparalelismo, para distinguirlo del verdadero paralelismo de hardware de los sistemas multiprocesadores con dos o más CPUs que comparten la misma memoria física).

Es difícil para las personas llevar la cuenta de varias actividades en paralelo; por lo tanto, los diseñadores de sistemas operativos han evolucionado con el paso de los años a un modelo conceptual (procesos secuenciales) que facilita el trabajo con el paralelismo.

Más sobre el modelo del proceso.

En este modelo, todo el software ejecutable en la computadora, que algunas veces incluye al sistema operativo, se organiza en varios procesos secuenciales (procesos, para abreviar).

Un proceso no es más que una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables. En concepto, cada proceso tiene su propia CPU virtual; en la realidad, la CPU real conmuta de un proceso a otro, pero para entender el sistema es mucho más fácil pensar en una colección de procesos que se ejecutan en (pseudo) paralelo, en vez de tratar de llevar la cuenta de cómo la CPU conmuta de programa en programa. Esta conmutación rápida de un proceso a otro se conoce como multiprogramación.

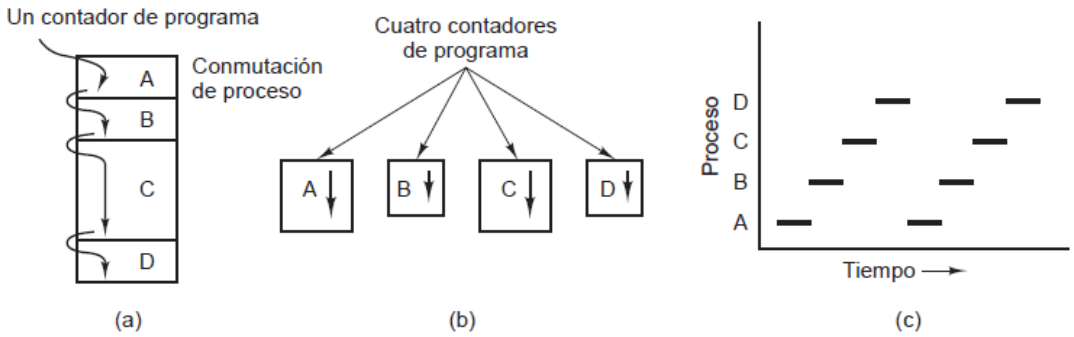


Ilustración 4 - (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

La figura (a) muestra una computadora administrando cuatro programas en memoria; la figura (b) lista cuatro procesos, cada uno con su propio flujo de control (es decir, su propio contador de programa lógico) y cada uno ejecutándose en forma independiente. Desde luego que sólo hay un contador de programa físico, por lo que cuando se ejecuta cada proceso, se carga su contador de programa lógico en el contador de programa real. Cuando termina (por el tiempo que tenga asignado), el contador de programa físico se guarda en el contador de programa lógico almacenado, del proceso en memoria. En la figura (c) podemos ver que durante un intervalo suficientemente largo todos los procesos han progresado, pero en cualquier momento dado sólo hay un proceso en ejecución.

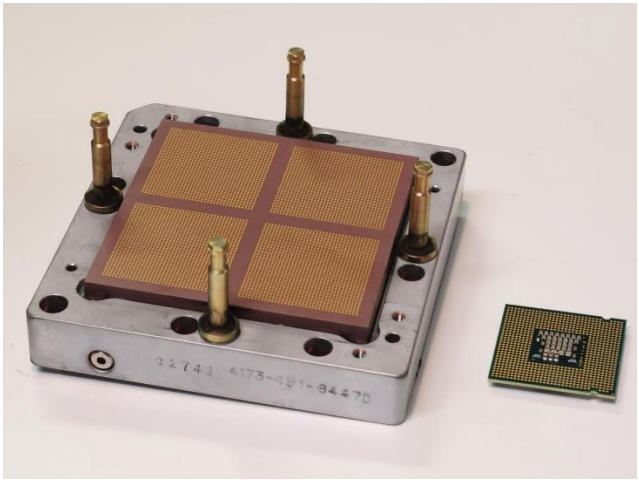
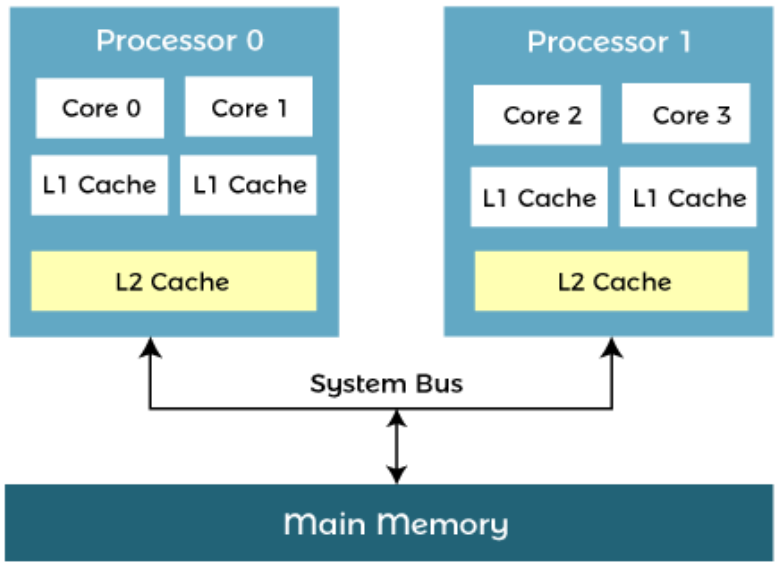


Ilustración 5 – Uno de los primeros procesadores multinúcleo comerciales fue el IBM POWER4 (izquierda), lanzado en el año 2001.

Si bien en más sencillo suponer que sólo hay una CPU, es importante tener en cuenta que en la actualidad esa condición cada vez es más rara debido a que los nuevos chips son ***multinúcleos***, con dos, cuatro o más CPUs. Entonces, cuando decimos que una CPU puede ejecutar sólo un proceso a la vez, si hay dos núcleos (o CPUs) cada uno de ellos puede ejecutar sólo un proceso a la vez.

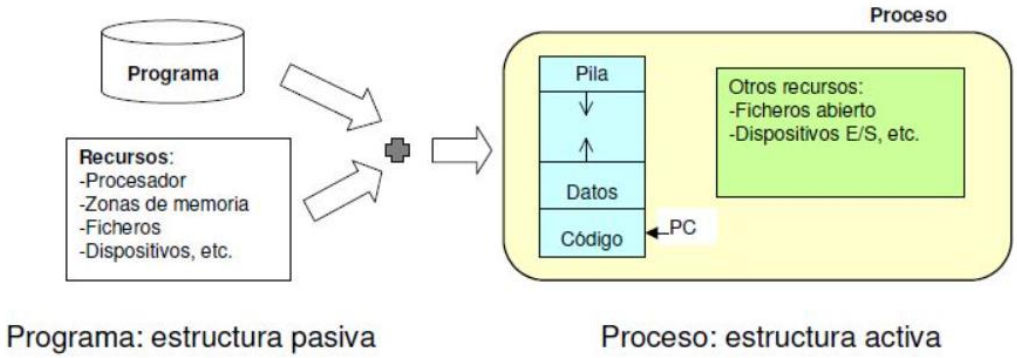


Dado que la CPU conmuta rápidamente entre un proceso y otro, la velocidad a la que un proceso ejecuta sus cálculos no es uniforme y tal vez ni siquiera sea reproducible si se ejecutan los mismos procesos de nuevo. Por ende, al programar los procesos debe asumirse esta variación de velocidad.

Por ejemplo, un proceso de E/S que inicia una unidad de cinta magnética para restaurar los archivos respaldados, ejecuta un ciclo de inactividad 10,000 veces para permitir que obtenga la velocidad adecuada y después emite un comando para leer el primer registro. Si la CPU decide conmutar a otro proceso durante el ciclo de inactividad, el proceso de la cinta tal vez no se ejecute de nuevo sino hasta que el primer registro se encuentre más allá de la cabeza de lectura.

Cuando un proceso tiene requerimientos críticos de tiempo real como éste, es decir, cuando deben ocurrir eventos específicos dentro de un número especificado de milisegundos, es necesario tomar medidas especiales para asegurar que ocurran. Sin embargo, por lo general la mayoría de los procesos no se ven afectados por la multiprogramación subyacente de la CPU o las velocidades relativas de distintos procesos.

La diferencia entre un proceso y un ***programa*** es sutil pero crucial. Un programa es un conjunto de instrucciones, mientras que un proceso es la ejecución de esas instrucciones.



Programa: estructura pasiva

Proceso: estructura activa

Para comprender mejor esta diferencia, supongamos que una persona hornea una torta de cumpleaños para su hija; tiene la receta para la torta y una cocina bien equipada con todos los ingredientes: harina, huevos, azúcar, etc. En esta analogía, la receta es el programa (es decir, un algoritmo expresado en cierta notación adecuada), el pastelero es el procesador (CPU) y los ingredientes de la torta son los datos de entrada. El proceso es la actividad que consiste en que nuestro cocinero vaya leyendo la receta, obteniendo los ingredientes y horneando la torta.

Ahora imaginemos que el hijo del cocinero entra corriendo y gritando que una abeja acaba de picarlo. El cocinero registra el punto de la receta en el que estaba (el estado del proceso en curso se guarda), saca un libro de primeros auxilios y empieza a seguir las instrucciones que contiene. Aquí el procesador conmuta de un proceso (hornear torta) a uno de mayor prioridad (administrar cuidados médicos), cada uno de los cuales tiene un programa distinto (la receta y el libro de primeros auxilios). Cuando ya se ha ocupado de la picadura de abeja, el cocinero regresa a su torta y continúa en el punto en el que se había quedado.

La idea clave es que un proceso es una actividad de cierto tipo: tiene un programa, una entrada, una salida y un estado. Varios procesos pueden compartir un solo procesador mediante el uso de un ***algoritmo de planificación*** para determinar cuándo se debe detener el trabajo en un proceso para dar servicio a otro.

Insistamos en que un programa, por sí mismo, no es un proceso; un programa es una entidad pasiva, un archivo que contiene una lista de instrucciones almacenadas en disco (a menudo denominado archivo ejecutable), mientras que un proceso es una entidad activa, con un contador de programa que especifica la siguiente instrucción que hay que ejecutar y un conjunto de recursos asociados.

Vale la pena recalcar, también, que si un programa se está ejecutando por duplicado cuenta como dos procesos.

Por ejemplo, a menudo es posible iniciar un procesador de palabras dos veces o imprimir dos archivos al mismo tiempo si hay dos impresoras disponibles. El hecho de que dos procesos en ejecución tengan el mismo programa no importa; son procesos distintos. El sistema operativo puede compartir el código entre ellos de manera que sólo haya una copia en la memoria, pero ése es un detalle técnico que no cambia la situación conceptual de dos procesos en ejecución.

Referencias y bibliografía de la sección.

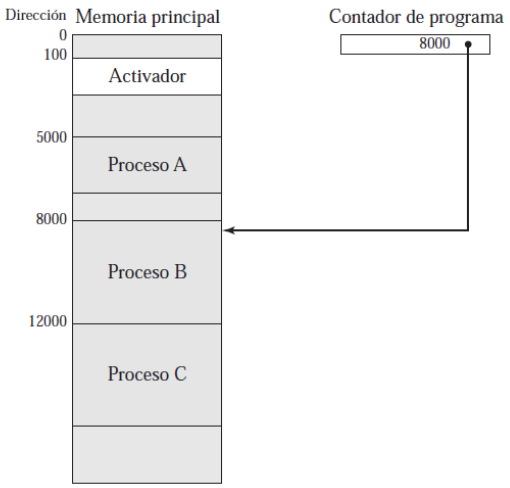
- Silberschatz 3-1.
- Tanenbaum 2-1.

Estados del proceso.

Para que un programa se ejecute, se debe crear un proceso o tarea para dicho programa. Desde el punto de vista del procesador, él ejecuta instrucciones de su repertorio de instrucciones en una secuencia dictada por el cambio de los valores del registro contador de programa (PC).

A lo largo del tiempo, el contador de programa (PC) puede apuntar al código de diferentes programas (en memoria) que son parte de diferentes procesos. Desde el punto de vista de un programa individual, su ejecución implica una secuencia de instrucciones dentro de dicho programa.

Se puede caracterizar el comportamiento de un determinado proceso, listando la secuencia de instrucciones que se ejecutan para dicho proceso. A esta lista se la denomina ***traza del proceso***. Se puede caracterizar el comportamiento de un procesador mostrando cómo las trazas de varios procesos se entrelazan.



La figura anterior muestra el despliegue en memoria de tres procesos. Para simplificar, se asume que dichos procesos no usan memoria virtual; por tanto, los tres procesos están representados por programas que residen en memoria principal.

De manera adicional, existe un pequeño programa ***activador/despachador*** (*dispatcher*), que es parte del OS, el cual intercambia el procesador de un proceso a otro.

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011
(a) Traza del Proceso A	**(b) Traza del Proceso B**	**(c) Traza del Proceso C**

5000 = Dirección de comienzo del programa del Proceso A.
8000 = Dirección de comienzo del programa del Proceso B.
12000 = Dirección de comienzo del programa del Proceso C.

Esta figura muestra las trazas de cada uno de los procesos en los primeros instantes de ejecución. Se muestran las 12 primeras instrucciones ejecutadas por los procesos A y C. El proceso B ejecuta 4 instrucciones y se asume que la cuarta instrucción invoca una operación de E/S, a la cual el proceso debe esperar.

1	5000	
2	5001	
3	5002	
4	5003	
5	5004	
6	5005	
		Temporización
7	100	
8	101	
9	102	
10	103	
11	104	
12	105	
13	8000	
14	8001	
15	8002	
16	8003	
		Petición de E/S
17	100	
18	101	
19	102	
20	103	
21	104	
22	105	
23	12000	
24	12001	
25	12002	
26	12003	
27	12004	
28	12005	
		Temporización
29	100	
30	101	
31	102	
32	103	
33	104	
34	105	
35	5006	
36	5007	
37	5008	
38	5009	
39	5010	
40	5011	
		Temporización
41	100	
42	101	
43	102	
44	103	
45	104	
46	105	
47	12006	
48	12007	
49	12008	
50	12009	
51	12010	
52	12011	
		Temporización

100 = Dirección de comienzo del programa activador.
Las zonas sombreadas indican la ejecución del proceso de activación;
la primera y la tercera columna cuentan ciclos de instrucciones;
la segunda y la cuarta columna las direcciones de las instrucciones que se ejecutan

Ahora veamos estas trazas desde el punto de vista del procesador. La figura anterior muestra las trazas entrelazadas resultante de los 52 primeros ciclos de ejecución (por conveniencia los ciclos de instrucciones han sido numerados). En este ejemplo, se asume que el sistema operativo sólo deja que un proceso continúe durante seis ciclos de instrucción, después de los cuales se interrumpe; lo cual previene que un solo proceso monopolice el uso del tiempo del procesador.

Las primeras seis instrucciones del proceso A se ejecutan seguidas de una alarma de temporización (*timeout*) y de la ejecución de cierto código del activador, que ejecuta seis instrucciones antes de devolver el control al proceso B. Después de que se ejecuten cuatro instrucciones, el proceso B solicita una acción de E/S, para la cual debe esperar. Por tanto, el procesador deja de ejecutar el proceso B y pasa a ejecutar el proceso C, por medio del activador. Después de otra alarma de temporización, el procesador vuelve al proceso A. Cuando este proceso llega a su temporización, el proceso B aún estará esperando que se complete su operación de E/S, por lo que el activador pasa de nuevo al proceso C.

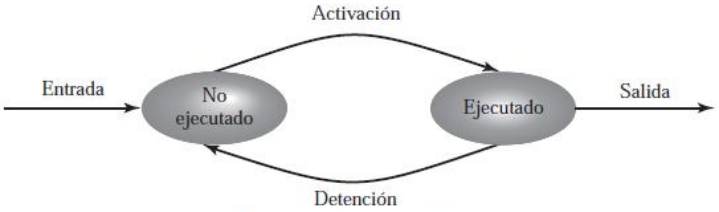
Una de las responsabilidades principales del sistema operativo es controlar la ejecución de los procesos; esto incluye determinar el patrón de entrelazado para la ejecución y asignar recursos a los procesos. El primer paso en el diseño de un sistema operativo para el control de procesos es describir el comportamiento que se desea que tengan los procesos.

El modelo más simple posible determina que, en un instante dado, un proceso está siendo ejecutando por el procesador o no. En este modelo, un proceso puede estar en ***dos estados***: ejecutando o no ejecutando.

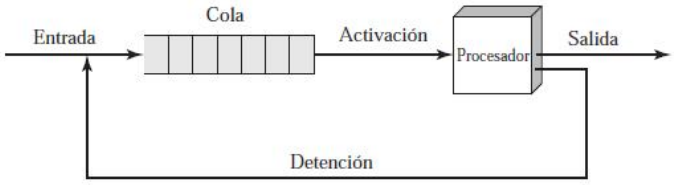
Cuando el sistema operativo crea un nuevo proceso, crea el bloque de control de proceso (***BCP***) para el nuevo proceso e inserta dicho proceso en el sistema en estado no ejecutando. El proceso existe, es conocido por el sistema operativo, y está esperando su oportunidad de ejecutar.

De cuando en cuando, el proceso actualmente en ejecución se interrumpirá y una parte del sistema operativo, el activador, seleccionará otro proceso a ejecutar. El proceso saliente pasará del estado ejecutando a no ejecutando y pasará a ejecutando un nuevo proceso.

Cada proceso debe representarse de tal manera que el sistema operativo pueda seguirle la pista. Es decir, debe haber información correspondiente a cada proceso, incluyendo el estado actual y su localización en memoria; esto es el ***bloque de control de proceso***. Los procesos que no están ejecutando deben estar en una especie de cola, esperando su turno de ejecución.



(a) Diagrama de transiciones de estados



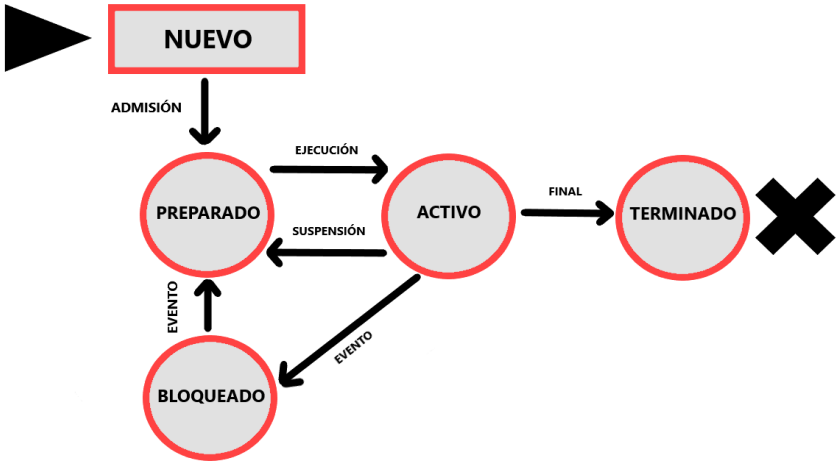
(b) Modelos de colas

Existe una cola cuyas entradas son punteros al BCP de un proceso en particular. Alternativamente, la cola debe consistir en una lista enlazada de bloques de datos, en la cual cada bloque representa un proceso.

Podemos describir el comportamiento del activador en términos de este diagrama de colas. Un proceso que se interrumpe se transfiere a la cola de procesos en espera. Alternativamente, si el proceso ha finalizado o ha sido abortado, se descarta (sale del sistema). En cualquier caso, el activador selecciona un proceso de la cola para ejecutar.

Podríamos pensar en un modelo de procesos un poco más complejo que el anterior. En un modelo de cinco estados cada proceso puede estar en uno de los estados siguientes:

- **Nuevo.** El proceso está siendo creado.
- **En ejecución.** Se están ejecutando las instrucciones.
- **En espera/bloqueado.** El proceso está esperando a que se produzca un suceso (como la terminación de una operación de E/S o la recepción de una señal).
- **Preparado/listo.** El proceso está a la espera de que le asignen a un procesador.
- **Terminado.** Ha terminado la ejecución del proceso.



Estos nombres son arbitrarios y varían de un sistema operativo a otro. Otra interpretación más de los estados de los procesos podría ser:

- **En ejecución** (el proceso está usando la CPU en ese instante).
- **Listo** (ejecutable; se detuvo temporalmente para dejar que se ejecute otro proceso).
- **Bloqueado** (no puede ejecutarse sino hasta que ocurra cierto evento externo).

Es importante darse cuenta de que sólo puede haber un proceso ejecutándose en cualquier procesador en cada instante concreto. Sin embargo, puede haber muchos procesos preparados y en espera.

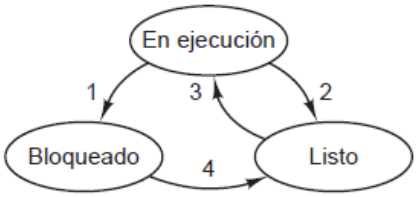
Aunque cada proceso es una entidad independiente, con su propio contador de programa y estado interno, a menudo los procesos necesitan interactuar con otros. Un proceso puede generar cierta salida que otro proceso utiliza como entrada.

```
cat capitulo1 capitulo2 capitulo3 | grep arbol
```

En este comando de shell, el primer proceso que ejecuta *cat* concatena tres archivos y los envía como salida. El segundo proceso, que ejecuta *grep*, selecciona todas las líneas que contengan la palabra "*arbol*". Dependiendo de la velocidad relativa de los dos procesos (que dependen tanto de la complejidad relativa de los programas, como de cuánto tiempo ha tenido cada uno la CPU), puede ocurrir que *grep* esté listo para ejecutarse, pero que no haya una entrada esperándolo. Entonces se debe bloquear hasta que haya una entrada disponible.

Cuando un proceso se bloquea, lo hace debido a que, por lógica, no puede continuar, comúnmente porque está esperando una entrada que todavía no está disponible. También es posible que un proceso, que esté listo en concepto y pueda ejecutarse, se detenga debido a que el sistema operativo ha decidido asignar la CPU a otro proceso por cierto tiempo. Estas dos condiciones son completamente distintas.

En el primer caso, la suspensión está inherente en el problema (no se puede procesar la línea de comandos del usuario sino hasta que éste la haya escrito mediante el teclado). En el segundo caso, es un tecnicismo del sistema (no hay suficientes CPUs como para otorgar a cada proceso su propio procesador privado).



1. El proceso se bloquea para recibir entrada
2. El planificador selecciona otro proceso
3. El planificador selecciona este proceso
4. La entrada ya está disponible

Siguiendo la lógica del grafico anterior, hay cuatro transiciones posibles entre los tres estados. La transición 1 ocurre cuando el sistema operativo descubre que un proceso no puede continuar justo en ese momento. En algunos sistemas el proceso puede ejecutar una llamada al sistema, como ***pause***, para entrar al estado bloqueado. En otros sistemas, incluyendo a UNIX, cuando un proceso lee datos de una canalización[1] o de un archivo especial (como una terminal) y no hay entrada disponible, el proceso se bloquea en forma automática.

Las transiciones 2 y 3 son producidas por el planificador de procesos[2], una parte del sistema operativo, sin que el proceso sepa siquiera acerca de ellas. La transición 2 ocurre cuando el planificador decide que el proceso en ejecución se ha ejecutado el tiempo suficiente y es momento de dejar que otro proceso tenga una parte del tiempo de la CPU. La transición 3 ocurre cuando todos los demás procesos han tenido su parte del tiempo de la CPU y es momento de que el primer proceso obtenga la CPU para ejecutarse de nuevo.

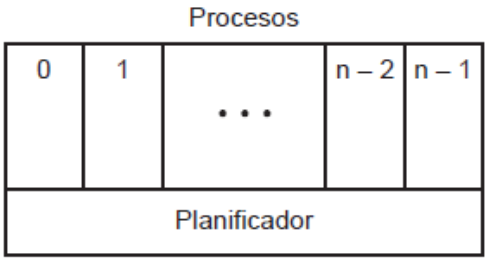
El tema de la planificación de procesos (decidir qué proceso debe ejecutarse en qué momento y por cuánto tiempo) es de suma importancia. Se han ideado muchos algoritmos para tratar de balancear las contrastantes demandas de eficiencia para el sistema como un todo y de equidad para los procesos individuales.

La transición 4 ocurre cuando se produce el evento externo por el que un proceso estaba esperando (como la llegada de ciertos datos de entrada). Si no hay otro proceso en ejecución en ese instante, se activa la transición 3 y el proceso empieza a ejecutarse. En caso contrario, tal vez tenga que esperar en el estado ***listo*** por unos instantes, hasta que la CPU esté disponible y sea su turno de utilizarla.

Algunos de los procesos ejecutan programas que llevan a cabo los comandos que escribe un usuario; otros son parte del sistema y se encargan de tareas como cumplir con las peticiones de los servicios de archivos o administrar los detalles de ejecutar una unidad de disco. Cuando ocurre una interrupción de disco, el sistema toma una decisión para dejar de ejecutar el proceso actual y ejecutar el proceso de disco que está bloqueado esperando esta interrupción.

[1] La canalización es un mecanismo de comunicación entre procesos que permite que intercambien datos, señales, solicitudes y respuestas.
[2] El planificador de procesos es un componente del sistema operativo que decide qué proceso usará el procesador y por cuánto tiempo.

Así, en vez de pensar en las interrupciones, podemos pensar en los procesos de usuario, procesos de disco, procesos de terminal, etc., que se bloquean cuando están esperando a que algo ocurra. Cuando se ha leído el disco o se ha escrito el carácter, el proceso que espera se desbloquea y es elegible para continuar ejecutándose.



La imagen anterior ilustra la lógica de un sistema operativo estructurado por procesos. En el nivel más bajo del sistema está el ***planificador***, con una variedad de procesos encima de él. Todo el manejo de las interrupciones y los detalles relacionados con iniciar y detener los procesos se ocultan en lo que aquí se denomina planificador. El resto del sistema operativo está muy bien estructurado en forma de procesos. Sin embargo, pocos sistemas reales están tan bien organizados como éste.

Es importante señalar que el planificador y el activador/despachador trabajan juntos, pero cumplen funciones distintas. El planificador decide qué proceso se ejecuta, y el despachador hace que eso suceda en el CPU.

Creación y terminación de un proceso.

De forma independiente del modelo de estados de procesos que se use, la vida de un proceso está acotada entre su creación y su terminación.

Cuando se va a añadir un nuevo proceso a aquellos que se están gestionando en un determinado momento, el sistema operativo construye las estructuras de datos que se usan para manejar el proceso y reserva el espacio de direcciones en memoria principal para el proceso. Estas acciones constituyen la ***creación*** de un nuevo proceso.

Nuevo proceso de lotes	El sistema operativo dispone de un flujo de control de lotes de trabajos, habitualmente una cinta un disco. Cuando el sistema operativo está listo para procesar un nuevo trabajo, leerá la siguiente secuencia de mandatos de control de trabajos.
Sesión interactiva	Un usuario desde un terminal entra en el sistema.
Creado por el sistema operativo para proporcionar un servicio	El sistema operativo puede crear un proceso para realizar una función en representación de un programa de usuario, sin que el usuario tenga que esperar (por ejemplo, un proceso para controlar la impresión).
Creado por un proceso existente	Por motivos de modularidad o para explotar el paralelismo, un programa de usuario puede ordenar la creación de un número de procesos.

Ilustración 6 - Razones para la creación de un proceso.

Existen diferentes eventos comunes que llevan a la creación de un proceso. En un entorno por lotes, un proceso se crea como respuesta a una solicitud de trabajo. En un entorno interactivo, un proceso se crea cuando un nuevo usuario entra en el sistema. En ambos casos el sistema operativo es responsable de la creación de nuevos procesos.

Un sistema operativo puede, a petición de una aplicación, crear procesos. Por ejemplo, si un usuario solicita que se imprima un fichero, el sistema operativo puede crear un proceso que gestione la impresión. El proceso solicitado puede, de esta manera, operar independientemente del tiempo requerido para completar la tarea de impresión.

También, sistema operativo puede crear un proceso a petición explícita de otro proceso. Cuando un proceso lanza otro, al primero se lo denomina ***proceso padre***, y al proceso creado se lo denomina ***proceso hijo***.

Habitualmente, la relación entre procesos necesita comunicación y cooperación entre ellos. Alcanzar esta cooperación es una tarea complicada para un programador.



Todo sistema debe proporcionar los mecanismos mediante los cuales un proceso indica su finalización, o que ha completado su tarea.

Finalización normal	El proceso ejecuta una llamada al sistema operativo para indicar que ha completado su ejecución
Límite de tiempo excedido	El proceso ha ejecutado más tiempo del especificado en un límite máximo. Existen varias posibilidades para medir dicho tiempo. Estas incluyen el tiempo total utilizado, el tiempo utilizado únicamente en ejecución, y, en el caso de procesos interactivos, la cantidad de tiempo desde que el usuario realizó la última entrada.
Memoria no disponible	El proceso requiere más memoria de la que el sistema puede proporcionar.
Violaciones de frontera	El proceso trata de acceder a una posición de memoria a la cual no tiene acceso permitido.
Error de protección	El proceso trata de usar un recurso, por ejemplo un fichero, al que no tiene permitido acceder, o trata de utilizarlo de una forma no apropia, por ejemplo, escribiendo en un fichero de sólo lectura.
Error aritmético	El proceso trata de realizar una operación de cálculo no permitida, tal como una división por 0, o trata de almacenar números mayores de los que la representación hardware puede codificar.
Límite de tiempo	El proceso ha esperado más tiempo que el especificado en un valor máximo para que se cumpla un determinado evento.
Fallo de E/S	Se ha producido un error durante una operación de entrada o salida, por ejemplo la imposibilidad de encontrar un fichero, fallo en la lectura o escritura después de un límite máximo de intentos (cuando, por ejemplo, se encuentra un área defectuosa en una cinta), o una operación inválida (la lectura de una impresora en línea).
Instrucción no válida	El proceso intenta ejecutar una instrucción inexistente (habitualmente el resultado de un salto a un área de datos y el intento de ejecutar dichos datos).
Instrucción privilegiada	El proceso intenta utilizar una instrucción reservada al sistema operativo.
Uso inapropiado de datos	Una porción de datos es de tipo erróneo o no se encuentra inicializada.
Intervención del operador por el sistema operativo	Por alguna razón, el operador o el sistema operativo ha finalizado el proceso (por ejemplo, se ha dado una condición de interbloqueo).
Terminación del proceso padre	Cuando un proceso padre termina, el sistema operativo puede automáticamente finalizar todos los procesos hijos descendientes de dicho padre.
Solicitud del proceso padre	Un proceso padre habitualmente tiene autoridad para finalizar sus propios procesos descendientes.

Ilustración 7 - Razones para la terminación de un proceso.

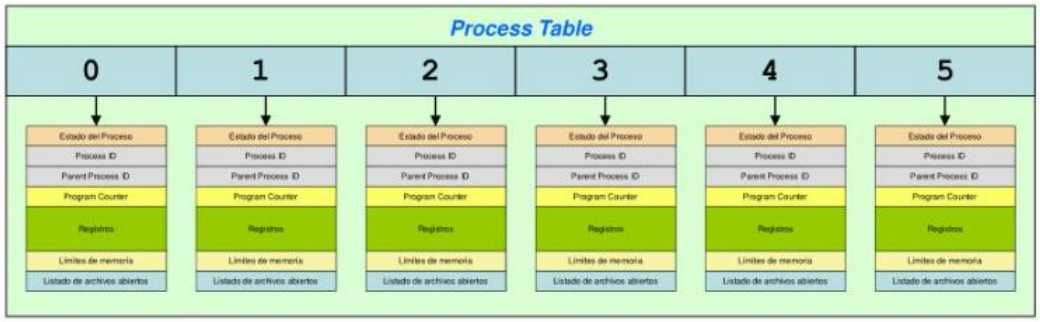
Referencias y bibliografía de la sección.

- Tanenbaum 2-1-5.
- Silberschatz 3-1-2.
- Stallings 3-2 pág. 110.

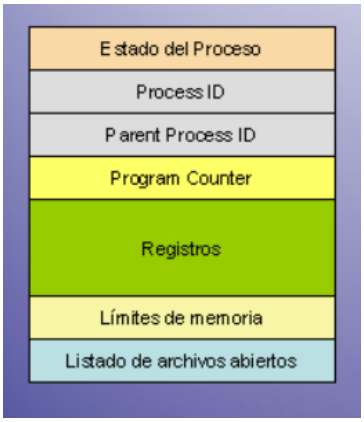
Bloque de control de proceso.

Cada proceso está asociado a un conjunto de ***atributos*** que son utilizados por el sistema operativo para controlar el proceso (como el identificador del proceso o ***PID***, y el identificador del usuario que lo lanzó o ***UID***), estructurados dentro del bloque de control de proceso (***BCP***).

Para implementar el modelo de procesos, el sistema operativo mantiene una tabla (un arreglo de estructuras) llamada ***tabla de procesos***, con sólo una entrada por cada proceso (muchos autores llaman a esta entrada ***bloque de control de proceso***). Esta entrada contiene información importante acerca del estado del proceso, incluyendo su contador de programa, apuntador de pila, asignación de memoria, estado de sus archivos abiertos, información de contabilidad y planificación, y todo lo demás que debe guardarse acerca del proceso cuando éste cambia del estado en ejecución a listo o bloqueado, de manera que se pueda reiniciar posteriormente como si nunca se hubiera detenido.



El ***BCP*** es una estructura de datos que el sistema operativo usa para guardar toda la información relevante sobre un proceso. Funciona como una especie de “legajo” de los procesos, que el sistema operativo necesita para administrarlos correctamente.

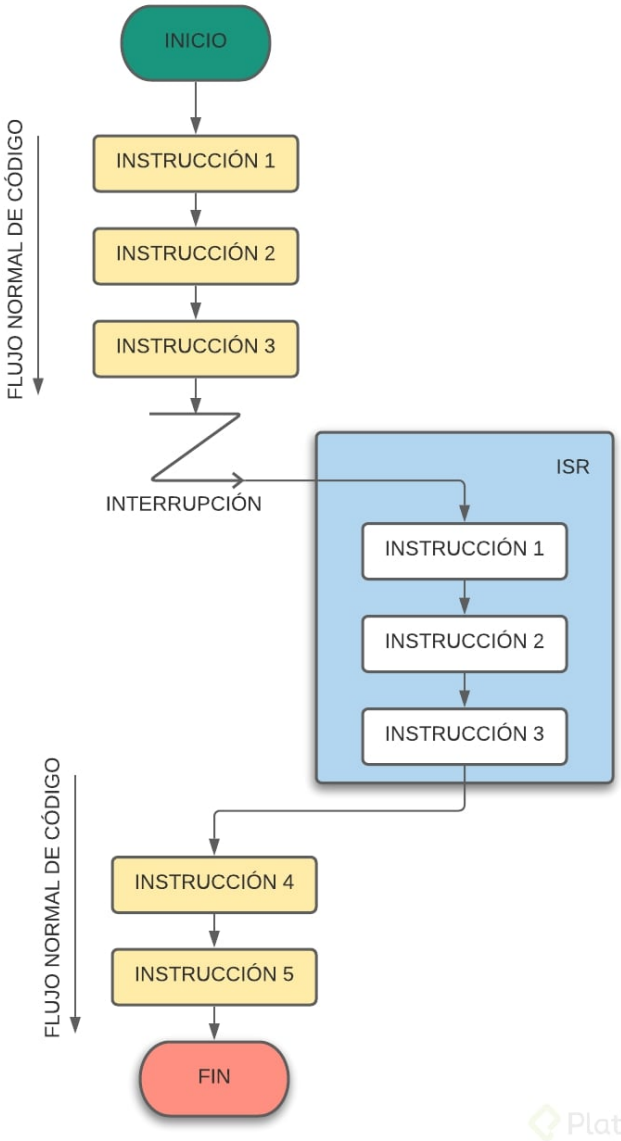


El BCP le permite al sistema operativo gestionar múltiples procesos al mismo tiempo de forma organizada y eficiente. Es útil especialmente a la hora de crear, suspender, reanudar o finalizar un proceso. También durante un cambio de contexto (cuando el sistema cambia de un proceso a otro).

Administración de procesos	Administración de memoria	Administración de archivos
Registros	Apuntador a la información del segmento de texto	Directorio raíz
Contador del programa	Apuntador a la información del segmento de datos	Directorio de trabajo
Palabra de estado del programa	Apuntador a la información del segmento de pila	Descripciones de archivos
Apuntador de la pila		ID de usuario
Estado del proceso		ID de grupo
Prioridad		
Parámetros de planificación		
ID del proceso		
Proceso padre		
Grupo de procesos		
Señales		
Tiempo de inicio del proceso		
Tiempo utilizado de la CPU		
Tiempo de la CPU utilizado por el hijo		
Hora de la siguiente alarma		

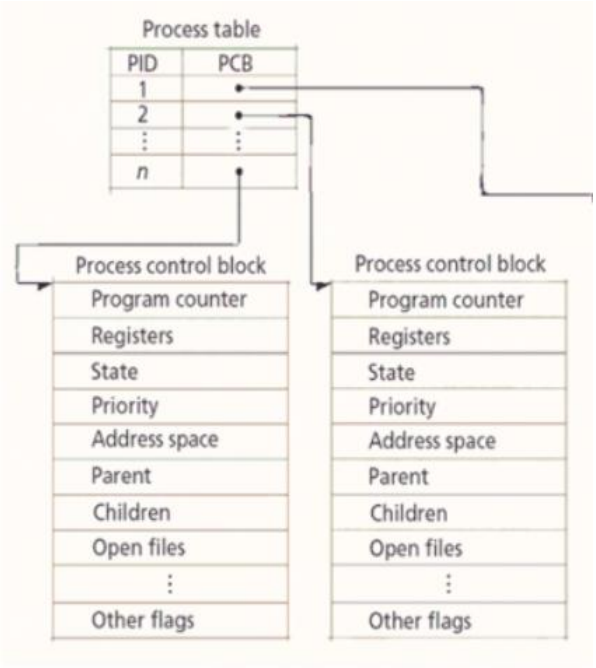
En la figura anterior, los campos en la primera columna se relacionan con la administración de procesos; los otros dos se relacionan con la administración de memoria y archivos, respectivamente. Hay que recalcar que los campos contenidos en la tabla de procesos varían de un sistema a otro.

Con cada clase de E/S hay una ubicación asociada (por lo general, en una ubicación cerca de la parte final de la memoria), a la cual se le llama vector de interrupción. Esta ubicación contiene la dirección del procedimiento del servicio de interrupciones (ISR o Interrupt Service Routine).



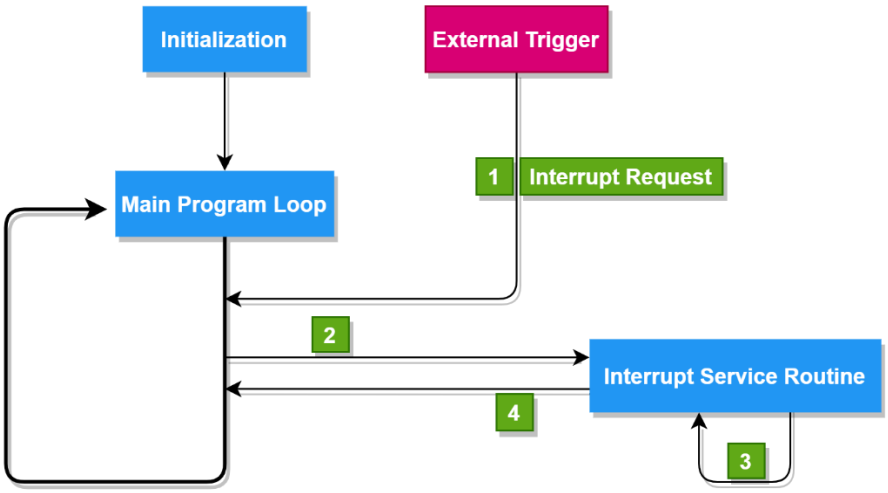
Supongamos que un proceso de usuario está en ejecución cuando ocurre una interrupción de disco. El contador de programa, la palabra de estado del programa y algunas veces uno o más registros del proceso del usuario se meten en la pila (actual) mediante el hardware de interrupción. Después, la computadora salta a la dirección especificada en el vector de interrupción. Esto es todo lo que hace el hardware. De aquí en adelante, depende del software y en especial del procedimiento del servicio de interrupciones.

Todas las interrupciones empiezan por guardar los registros, a menudo en la entrada de la ***tabla de procesos*** para el proceso actual. Después, se quita la información que la interrupción metió en la pila y el apuntador de pila se establece para que apunte a una pila temporal utilizada por el manejador de procesos.



Las acciones como guardar los registros y establecer el apuntador de pila no se pueden expresar ni siquiera en lenguajes como C, por lo que se realizan mediante una pequeña rutina en lenguaje ensamblador, que por lo general es la misma para todas las interrupciones, ya que el trabajo de guardar los registros es idéntico, sin importar cuál sea la causa de la interrupción.

Cuando se termina con esta rutina, se llama a un procedimiento escrito en un lenguaje de programas de un nivel más alto para realizar el resto del trabajo para este tipo de interrupción específico (suponemos que el sistema operativo está escrito en C, la elección usual para todos los sistemas operativos reales).



Cuando ha terminado su trabajo y tal vez ocasionando que algún otro proceso esté entonces listo, el planificador es llamado para ver qué proceso se debe ejecutar a continuación. Después

de eso, el control se pasa de vuelta al código en lenguaje ensamblador para cargar los registros y el mapa de memoria para el proceso que entonces es el actual y se empieza a ejecutar.

1. El hardware mete el contador del programa a la pila, etc.
2. El hardware carga el nuevo contador de programa del vector de interrupciones.
3. Procedimiento en lenguaje ensamblador guarda los registros.
4. Procedimiento en lenguaje ensamblador establece la nueva pila.
5. El servicio de interrupciones de C se ejecuta (por lo general lee y guarda la entrada en el búfer).
6. El planificador decide qué proceso se va a ejecutar a continuación.
7. Procedimiento en C regresa al código de ensamblador.
8. Procedimiento en lenguaje ensamblador inicia el nuevo proceso actual.

Ilustración 8 - Esqueleto de lo que hace el nivel más bajo del sistema operativo cuando ocurre una interrupción. Los detalles varían dependiendo del sistema.

Referencias y bibliografía de la sección.

- Tanenbaum 2-1-6.
- Silberschatz 3-1-3.
- Stallings 3-3, pág. 128.

Estructuras de control del sistema operativo.

Si el sistema operativo se encarga de la gestión de procesos y recursos, debe disponer de información sobre el estado actual de cada proceso y cada recurso. El mecanismo universal para proporcionar esta información es el siguiente: el sistema operativo construye y mantiene tablas de información sobre cada entidad que gestiona.

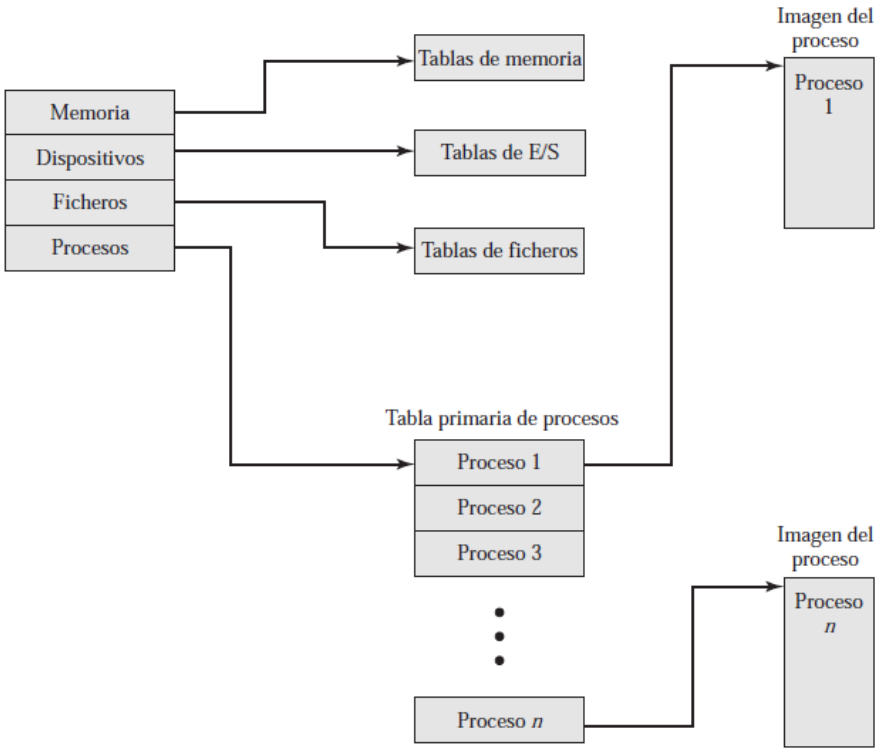


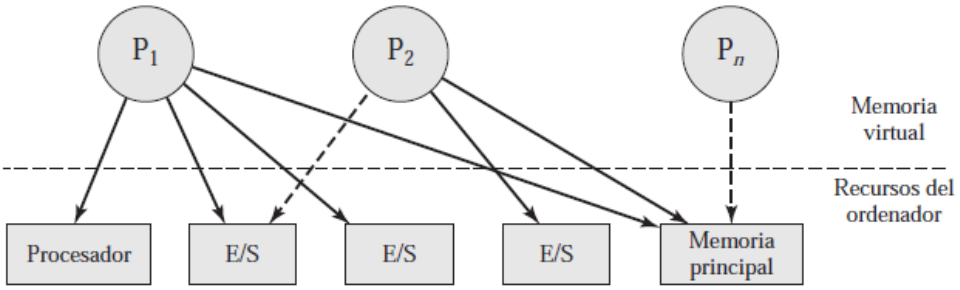
Ilustración 9 - Estructura general de las tablas de control del sistema operativo.

En la figura anterior se muestra cuatro diferentes tipos de tablas mantenidas por el sistema operativo: memoria, E/S, ficheros y procesos. A pesar de que los detalles difieren de un sistema operativo a otro, fundamentalmente, todos los sistemas operativos mantienen información de estas cuatro categorías.

Las ***tablas de memoria*** se usan para mantener un registro tanto de la memoria principal (real) como de la secundaria (virtual). Parte de la memoria principal está reservada para el uso del sistema operativo; el resto está disponible para el uso de los procesos. Los procesos se mantienen en memoria secundaria utilizando algún tipo de memoria virtual o técnicas de swapping[3]. Las tablas de memoria deben incluir la siguiente información:

- Las reservas de memoria principal por parte de los procesos.
- Las reservas de memoria secundaria por parte de los procesos.
- Todos los atributos de protección que restringe el uso de la memoria principal y virtual, de forma que los procesos puedan acceder a ciertas áreas de memoria compartida.
- La información necesaria para manejar la memoria virtual.

[3] El swapping o espacio de intercambio es un proceso que consiste en copiar páginas de memoria al disco duro para liberar memoria RAM.



El sistema operativo debe utilizar las ***tablas de E/S*** para gestionar los dispositivos de E/S y los canales de la computadora. En un instante determinado, un dispositivo E/S puede estar disponible o asignado a un proceso en particular. Si la operación de E/S se está realizando, el sistema operativo necesita conocer el estado de la operación y la dirección de memoria principal del área usada como fuente o destino de la transferencia de E/S.

- **IDLE:** El dispositivo no está haciendo nada.
- **BUSY:** El dispositivo está moviendo datos.
- **ERROR:** Algo falló en la comunicación (ej. un cable desconectado).

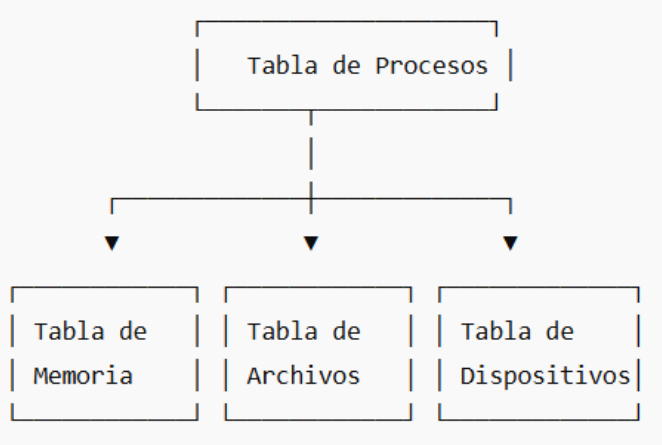
El sistema operativo también puede mantener las ***tablas de archivos***. Estas tablas proporcionan información sobre la existencia de archivos, su posición en almacenamiento secundario, su estado actual, y otros atributos. La mayoría de esta información se puede gestionar por el ***sistema de archivos***, en cuyo caso el sistema operativo tiene poco o ningún conocimiento sobre los archivos.

La tabla de archivos es como el "formulario estándar" que llena el sistema operativo para cada archivo abierto. Sin embargo, los datos que se ponen en ese formulario (especialmente la posición en el almacenamiento secundario y los atributos de seguridad) en muchos casos dependen de cómo el sistema de archivos haya organizado la información en el disco.

En otros sistemas operativos, la gran mayoría del detalle de la gestión de archivos sí que gestiona en el propio sistema operativo.

En último lugar, el sistema operativo debe mantener ***tablas de procesos*** para gestionar los procesos.

Los cuatro tipos diferentes de tablas se encuentran entrelazadas y referenciadas entre sí de alguna manera. Memoria, E/S, y ficheros se gestionan por parte de los procesos, de forma que debe haber algunas referencias en estos recursos, directa o indirectamente, desde las tablas de procesos. Los archivos indicados en las tablas de ficheros son accesibles mediante dispositivos E/S y estarán, en algún caso, residentes en memoria virtual o principal. Las tablas son también accesibles para el sistema operativo, y además están controladas por la gestión de memoria.



Para crear las tablas iniciales, el sistema operativo debe tener alguna información sobre el entorno básico. Por ejemplo, cuánta memoria física existe, cuáles son los dispositivos de E/S y cuáles son sus identificadores. Esto es una cuestión de configuración. Cuando el sistema operativo se inicializa, debe tener acceso a algunos datos de configuración que definen el entorno básico y estos datos se deben crear fuera del sistema operativo, con asistencia humana o por medio de algún software de autoconfiguración.

- **Procesos** → necesitan **memoria** para ejecutarse.
- **Procesos** → acceden a **archivos** (que pueden estar en disco o ser dispositivos).
- **Procesos** → interactúan con **dispositivos** (directamente o vía archivos especiales).

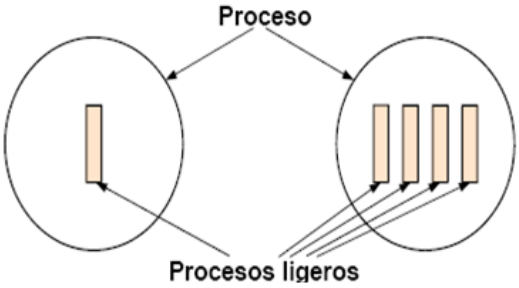
Referencias y bibliografía de la sección.

- Stallings 3-3, pág. 126.

Definición de hilo.

Un ***hilo*** de ejecución (o thread) es la unidad más pequeña de procesamiento que puede ser planificada por un sistema operativo.

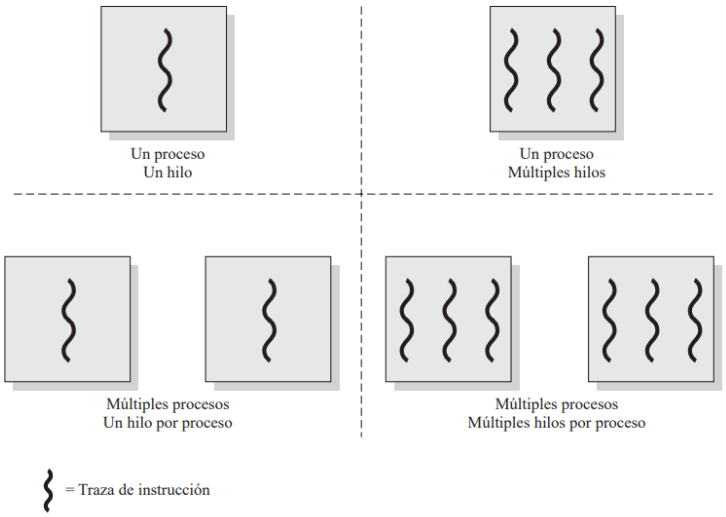
En los sistemas operativos tradicionales, cada proceso tiene un espacio de direcciones y un solo hilo de ejecución. De hecho, ésa es casi la definición de un proceso. Sin embargo, con frecuencia hay situaciones en las que es conveniente tener varios hilos de control en el mismo espacio de direcciones que se ejecuta en cuasiparalelo, como si fueran procesos (casi) separados (excepto por el espacio de direcciones compartido).



Multihilo se refiere a la capacidad de un sistema operativo de dar soporte a múltiples hilos de ejecución en un solo proceso.

Esa ejecución en paralelo/cuasiparalelo es la que permite que una aplicación moderna (como un navegador web) pueda descargar un archivo en un hilo, renderizar una animación en otro y responder a tus clics en un tercero, todo mientras comparten la misma memoria para ser eficientes y rápidos.

El enfoque tradicional de un solo hilo de ejecución por proceso, se conoce como estrategia ***monohilo***.



Un ejemplo de sistema operativo que soporta un único proceso de usuario y un único hilo por proceso es MS-DOS. Otros sistemas operativos, como algunas variedades de UNIX, soportan múltiples procesos de usuario, pero sólo un hilo por proceso.

El uso de múltiples procesos, cada uno de los cuales soporta múltiples hilos es el enfoque de sistemas como Windows o Solaris[4].

Dentro de un proceso puede haber uno o más hilos, cada uno con:

- Un estado de ejecución por hilo (Ejecutando, Listo, etc.).
- Un contexto de hilo que se almacena cuando no está en ejecución; una forma de ver a un hilo es como un contador de programa independiente dentro de un proceso.
- Una pila de ejecución.
- Acceso a la memoria y recursos de su proceso, compartido con todos los hilos de su mismo proceso.

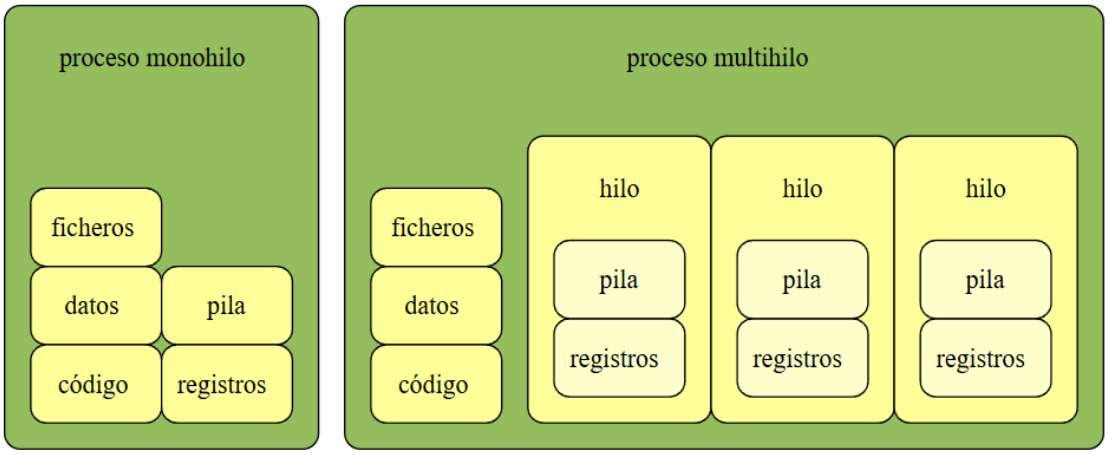


Ilustración 10 - Modelos de proceso con un único hilo y multihilo.

En un modelo de proceso ***monohilo***, la representación de un proceso incluye su bloque de control de proceso (BCP) y el espacio de direcciones de usuario, además de las pilas de usuario y núcleo para gestionar el comportamiento de las llamadas/retornos en la ejecución de los procesos. Mientras el proceso está ejecutando, los registros del procesador se controlan por ese proceso y, cuando el proceso no se está ejecutando, se almacena el contenido de estos registros.

En un entorno ***multihilo***, sigue habiendo un único BCP y un espacio de direcciones de usuario asociado al proceso, pero ahora hay varias pilas separadas para cada hilo, así como un bloque de control para cada hilo (TCB - Thread Control Block / ***BCH*** - Bloque de Control de Hilo) que contiene los valores de los registros, la prioridad, y otra información relativa al estado del hilo.

[4] Solaris es un sistema operativo de tipo Unix desarrollado desde 1992 inicialmente por Sun Microsystems y luego por Oracle Corporation tras la adquisición de Sun por parte de esta.

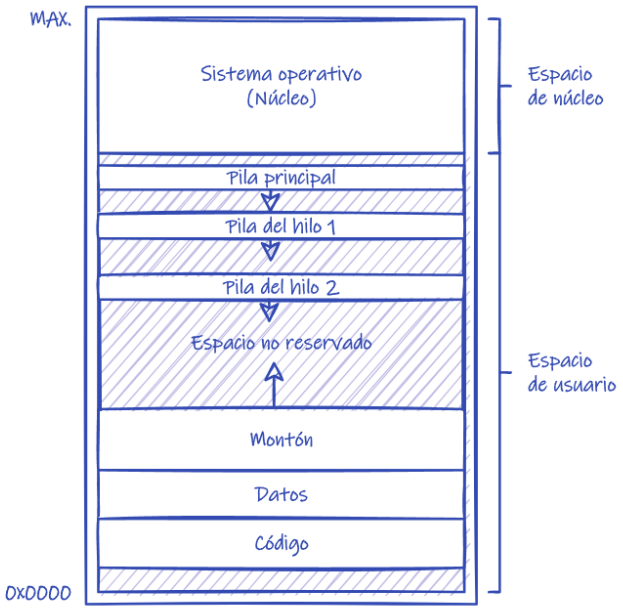


Ilustración 11 - Un proceso multihilo en memoria.

De esta forma, todos los hilos de un proceso comparten el estado y los recursos de ese proceso, residen en el mismo espacio de direcciones y tienen acceso a los mismos datos.

Entonces, a diferencia de un proceso completo, los hilos comparten el mismo espacio de memoria, lo que los hace rápidos, pero se requiere cuidado para evitar que "choquen" al escribir en las mismas variables.

Cuando un hilo cambia determinados datos en memoria, otros hilos ven los resultados cuando acceden a estos datos. Si un hilo abre un archivo con permisos de lectura, los demás hilos del mismo proceso pueden también leer ese archivo.

Todos los hilos del mismo proceso pueden ver las variables globales. Si dos hilos intentan sumar +1 a la misma variable al mismo tiempo, el resultado puede ser erróneo (Condición de Carrera).

En un procesador con varios núcleos, los hilos pueden ejecutarse físicamente al mismo tiempo (paralelismo). Con un solo núcleo, el procesador salta tan rápido entre hilos que parece que ocurren a la vez (concurrencia).

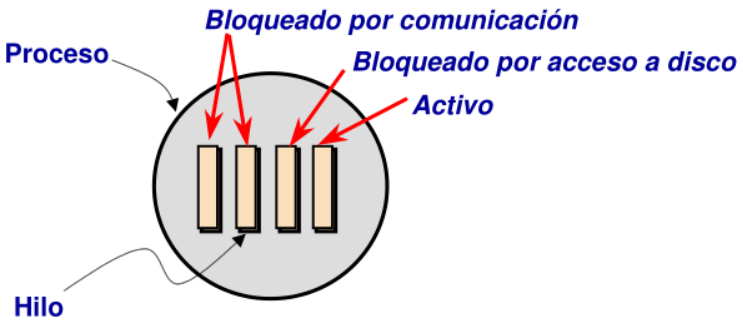
Estados de los hilos.

Los principales estados de los hilos son: ***Ejecución***, ***Listo*** y ***Bloqueado***. Generalmente, no tiene sentido aplicar otro tipo de estados a un hilo (ejemplo, estado de suspensión), ya que, generalmente, dichos estados son conceptos de nivel de proceso. En particular, si se expulsa un proceso, todos sus hilos se deben expulsar porque comparten el espacio de direcciones del proceso. Hay cuatro operaciones básicas relacionadas con los hilos que están asociadas con un cambio de estado del hilo:

- **Creación**. Cuando se crea un nuevo proceso, también se crea un hilo de dicho proceso. Posteriormente, un hilo del proceso puede crear otro hilo dentro del mismo proceso,

proporcionando un puntero a las instrucciones y los argumentos para el nuevo hilo. Al nuevo hilo se le proporciona su propio registro de contexto y espacio de pila y se coloca en la cola de Listos.

- **Bloqueo**. Cuando un hilo necesita esperar por un evento se bloquea, almacenando los registros de usuario, contador de programa y punteros de pila. El procesador puede pasar a ejecutar otro hilo en estado Listo, dentro del mismo proceso o en otro diferente.
- **Desbloqueo**. Cuando sucede el evento por el que el hilo está bloqueado, el hilo se pasa a la cola de Listos.
- **Finalización**. Cuando se completa un hilo, se liberan su registro de contexto y pilas.



Uso de hilos.

Hay varios motivos para tener miniprocesos, conocidos como **hilos**, dentro de otros procesos. La principal razón de tener hilos es que en muchas aplicaciones se desarrollan varias actividades a la vez. Algunas de ésas se pueden bloquear de vez en cuando. Al descomponer una aplicación en varios hilos secuenciales que se ejecutan en cuasiparalelo, el modelo de programación se simplifica.

Si se desea implementar una aplicación o función como un conjunto de unidades de ejecución relacionadas, es mucho más eficiente hacerlo con un conjunto de hilos que con un conjunto de procesos independientes.

Como ya vimos, en vez de pensar en interrupciones, temporizadores y conmutaciones de contexto, podemos pensar en procesos paralelos. Sólo que ahora con los hilos agregamos un nuevo elemento: la habilidad de las entidades en paralelo de compartir un espacio de direcciones y todos sus datos entre ellas. Esta habilidad es esencial para ciertas aplicaciones, en las cuales no sería optimo el tener varios procesos (con sus espacios de direcciones separados).

Un segundo argumento para tener hilos es que, como son más ligeros que los procesos, son más fáciles de crear y destruir (es decir, son más rápidos). En muchos sistemas, la creación de un hilo es de 10 a 100 veces más rápida que la de un proceso. Cuando el número de hilos necesarios cambia de manera dinámica y rápida, es útil tener esta propiedad.

Una tercera razón de tener hilos es también un argumento relacionado con el rendimiento. Los hilos no producen un aumento en el rendimiento cuando todos ellos están ligados a la CPU,

pero cuando hay una cantidad considerable de cálculos y operaciones de E/S, al tener hilos estas actividades se pueden solapar, con lo cual se agiliza la velocidad de la aplicación.

Por último, los hilos son útiles en los sistemas con varias CPUs, en donde es posible el verdadero paralelismo.

Velocidad	Menor sobrecarga (overhead) al iniciar tareas.
Solapamiento	La aplicación no se "congela" durante las operaciones lentas de disco o red.
Escalabilidad	El rendimiento mejora linealmente al añadir más núcleos de CPU.

Como ejemplo consideremos un procesador de texto. Por lo general, los procesadores de texto muestran el documento que se va crear en la pantalla exactamente como aparecerá en la página impresa. En especial, todos los saltos de línea y de página están en sus posiciones correctas y finales, de manera que el usuario pueda inspeccionarlas y cambiar el documento si es necesario.

Supongamos que el usuario está escribiendo un libro. Desde el punto de vista del autor, es más fácil mantener todo el libro en un solo archivo para facilitar la búsqueda de temas, realizar sustituciones globales, etc. También, cada capítulo podría estar en un archivo separado; sin embargo, tener cada sección y subsección como un archivo separado puede ser una verdadera molestia si hay que realizar cambios globales en todo el libro, ya que entonces tendrían que editarse cientos de archivos en forma individual.

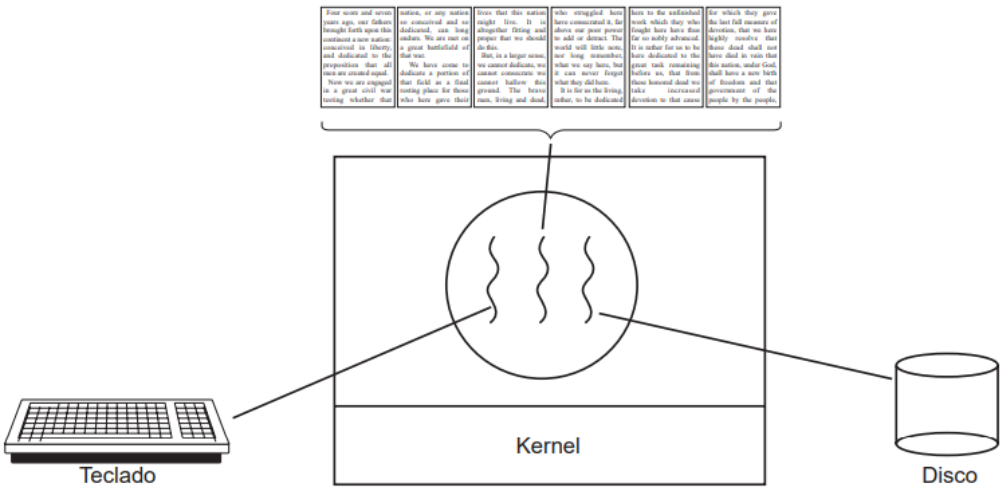
Si todo el libro está en un archivo, por lo general un solo comando puede realizar todas las sustituciones. Por el contrario, si el libro está esparcido en más de 300 archivos, cada uno se debe editar por separado.

Ahora consideremos lo que ocurre cuando el usuario repentinamente elimina un enunciado de la página 1 de un documento de 800 páginas. Después de revisar que la página modificada esté correcta, el usuario desea realizar otro cambio en la página 600 y escribe un comando que indica al procesador de palabras que vaya a esa página (posiblemente mediante la búsqueda de una frase que sólo esté en esa página). Entonces, el procesador de palabras tiene que volver a dar formato a todo el libro hasta la página 600 en ese momento, debido a que no sabe cuál será la primera línea de la página 600 sino hasta que haya procesado las demás páginas.

Puede haber un retraso considerable antes de que pueda mostrar la página 600 y el usuario estaría descontento. Aquí pueden ayudar los hilos. Supongamos que el procesador de palabras se escribe como un programa con dos hilos. Un hilo interactúa con el usuario y el otro se encarga de volver a dar formato en segundo plano. Tan pronto como se elimina el enunciado de la página 1, el hilo interactivo indica al hilo encargado de volver a dar formato que aplique de nuevo formato a todo el libro. Mientras tanto, el hilo interactivo sigue esperando las acciones del teclado y del mouse y responde a comandos simples como desplazarse por la página 1, mientras el otro hilo está realizando cálculos intensivos en segundo plano.

Con algo de suerte, el proceso de volver a dar formato se completará antes de que el usuario pida ver la página 600, para que pueda mostrarse al instante.

Podríamos complementar este ejemplo imaginando que se agrega un tercer hilo al programa. Muchos procesadores de palabras tienen la característica de guardar de manera automática todo el archivo cada cierto número de minutos. El tercer hilo se puede encargar de los respaldos en disco sin interferir con los otros dos.



Si el programa tuviera sólo un hilo, entonces cada vez que iniciara un respaldo en el disco se ignorarían los comandos del teclado y el ratón hasta que se terminara el respaldo. De manera alternativa, los eventos de teclado y ratón podrían interrumpir el respaldo en disco, permitiendo un buen rendimiento, pero produciendo un modelo de programación complejo, controlado por interrupciones. Con tres hilos, el modelo de programación es mucho más simple. El primer hilo interactúa sólo con el usuario, el segundo proceso vuelve a dar formato al documento cuando se le indica y el tercero escribe el contenido de la RAM al disco en forma periódica.

Debemos aclarar que en este caso no funcionaría tener tres procesos separados, ya que los tres hilos necesitan operar en el documento. Al tener tres hilos en vez de tres procesos, comparten una memoria común y por ende todos tienen acceso al documento que se está editando.

Ahora consideremos otro ejemplo, un servidor web. Las solicitudes de páginas llegan y la página solicitada se envía de vuelta al cliente. En la mayoría de los sitios web, algunas páginas se visitan con más frecuencia que otras. Por ejemplo, la página de inicio. Los servidores web utilizan este hecho para mejorar el rendimiento, al mantener una colección de páginas de uso frecuente en la memoria principal para eliminar la necesidad de ir al disco a obtenerlas. A dicha colección se le conoce como caché y se utiliza en muchos otros contextos también.

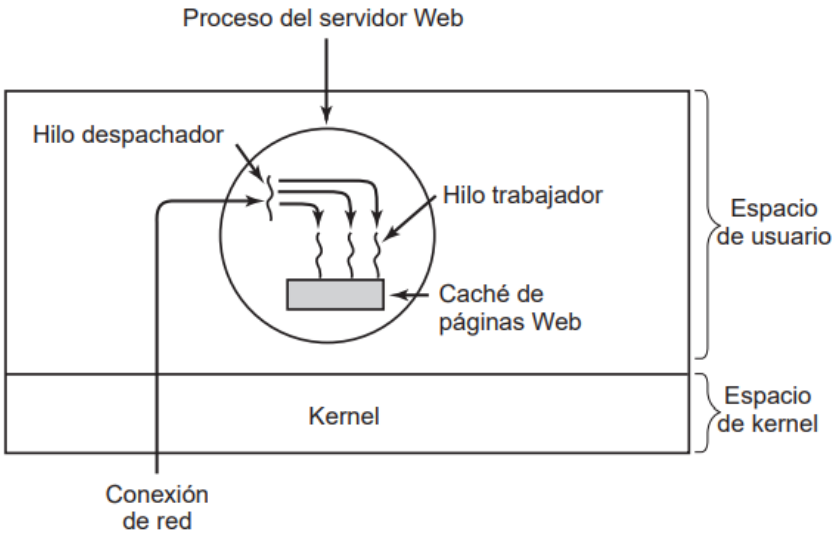


Ilustración 12 - Un servidor Web con múltiples hilos.

El hilo despachador lee las peticiones entrantes de trabajo de la red. Después de examinar la solicitud, selecciona un hilo trabajador inactivo (es decir, bloqueado) y le envía la solicitud, tal vez escribiendo un apuntador al mensaje en una palabra especial asociada con cada hilo. Después, el despachador despierta al hilo trabajador inactivo y lo pasa del estado bloqueado al estado listo.

Cuando el trabajador se despierta, comprueba si la petición se puede satisfacer desde la caché de páginas web, a la que todos los hilos tienen acceso. De no ser así, inicia una operación read para obtener la página del disco y se bloquea hasta que se complete la operación de disco.

Cuando el hilo se bloquea en la operación de disco, se selecciona otro hilo para ejecutarlo, posiblemente el despachador, para adquirir más trabajo o tal vez otro trabajador que queda listo para ejecutarse.

Este modelo permite escribir en el servidor en forma de una colección de hilos secuenciales. El programa del despachador consiste en un ciclo infinito para obtener una petición de trabajo y enviarla a un trabajador. El código de cada trabajador consiste en un ciclo infinito que se encarga de aceptar una petición del despachador y comprobar la caché de páginas web para ver si la página está presente. De ser así, se devuelve al cliente y el trabajador se bloquea para esperar una nueva petición.

Pensemos la forma en que podría escribirse el servidor web sin hilos. Una posibilidad es hacer que opere como un solo hilo. El ciclo principal del servidor Web recibe una petición, la examina y la lleva a cabo hasta completarla antes de obtener la siguiente. Mientras espera al disco, el servidor está inactivo y no procesa otras peticiones entrantes. Si el servidor Web se está ejecutando en una máquina dedicada, la CPU simplemente está inactiva mientras el servidor web espera al disco. El resultado neto es que se pueden procesar menos peticiones/segundo.

Por último, supongamos que el servidor web no puede utilizar múltiples hilos, pero que los diseñadores del sistema consideran que la pérdida de rendimiento debido a un solo hilo es inaceptable. Si hay una versión sin bloqueo de la llamada al sistema read disponible, es posible

un tercer diseño. Cuando entra una petición, el único hilo existente la examina. Si se puede satisfacer desde la caché está bien, pero si no es posible, se inicia una operación de disco sin bloqueo.

El servidor registra el estado de la petición actual en una tabla y después pasa a obtener el siguiente evento. Éste puede ser una petición para un nuevo trabajo o una respuesta del disco acerca de una operación anterior. Si es un nuevo trabajo, se inicia; si es una respuesta del disco, la información relevante se obtiene de la tabla y se procesa la respuesta. Con E/S de disco sin bloqueo, una respuesta probablemente tendrá que tomar la forma de una señal o interrupción.

En este diseño, el modelo de ***proceso secuencial*** que tuvimos en los primeros dos casos se pierde. El estado del cálculo debe guardarse y restaurarse de manera explícita en la tabla, cada vez que el servidor cambia de trabajar en una petición a otra. En efecto, estamos simulando los hilos y sus pilas de la manera difícil. Un diseño como éste, en el que cada cálculo tiene un estado de guardado y existe cierto conjunto de eventos que pueden ocurrir para cambiar el estado, se conoce como ***máquina de estados finitos***. Este concepto se utiliza ampliamente en las ciencias computacionales.

Los hilos posibilitan el concepto de procesos secuenciales que realizan llamadas al sistema con bloqueo (por ejemplo, para la E/S de disco) y de todas formas logran un paralelismo. Las llamadas al sistema con bloqueo facilitan el proceso de programación y el paralelismo mejora el rendimiento. El servidor de un solo hilo retiene la simpleza de las llamadas al sistema con bloqueo, pero pierde rendimiento.

El tercer método logra un alto rendimiento a través del paralelismo, pero utiliza llamadas e interrupciones sin bloqueo y, por ende, es difícil de programar.

Modelo	Características
Hilos	Paralelismo, llamadas al sistema con bloqueo
Proceso con un solo hilo	Sin paralelismo, llamadas al sistema con bloqueo
Máquina de estados finitos	Paralelismo, llamadas al sistema sin bloqueo, interrupciones

El modelo clásico de hilo.

El modelo de procesos se basa en dos conceptos independientes: agrupamiento de recursos y ejecución. Algunas veces es útil separarlos; aquí es donde entran los hilos.

Una manera de ver a un proceso es como si fuera una forma de agrupar recursos relacionados. Un proceso tiene un espacio de direcciones que contiene texto y datos del programa, así como otros recursos. Estos pueden incluir archivos abiertos, procesos hijos, alarmas pendientes, manejadores de señales, información contable y mucho más. Al reunirlos en forma de un proceso, pueden administrarse con más facilidad.

El otro concepto que tiene un proceso es un hilo de ejecución, al que por lo general sólo se le llama hilo. El hilo tiene un contador de programa que lleva el registro de cuál instrucción se va

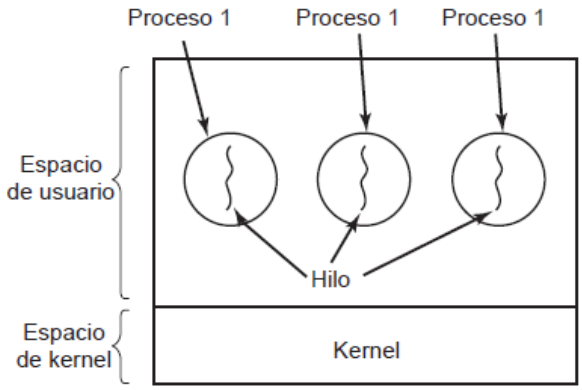
a ejecutar a continuación. Tiene registros que contienen sus variables de trabajo actuales. Tiene una pila, que contiene el historial de ejecución, con un conjunto de valores para cada procedimiento al que se haya llamado, pero del cual no se haya devuelto todavía.

Aunque un hilo se debe ejecutar en cierto proceso, el hilo y su proceso son conceptos distintos y pueden tratarse por separado. Los procesos se utilizan para agrupar los recursos.

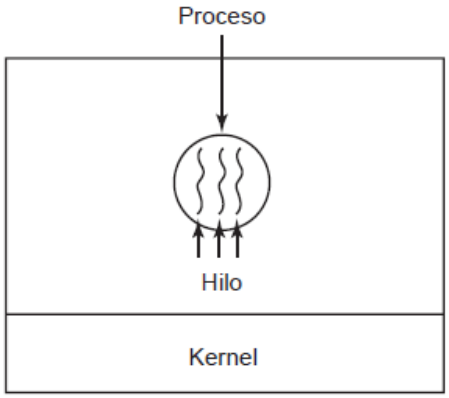
Lo que agregan los hilos al modelo de procesos es permitir que se lleven a cabo varias ejecuciones en el mismo entorno del proceso, que son en gran parte independientes unas de las otras.

Tener varios subprocesos ejecutándose en paralelo en un proceso es algo similar a tener varios procesos ejecutándose en paralelo en una computadora. En el primer caso, los hilos comparten un espacio de direcciones y otros recursos; en el segundo, los procesos comparten la memoria física, los discos, las impresoras y otros recursos. Como los hilos tienen algunas de las propiedades de los procesos, algunas veces se les llama ***procesos ligeros***.

El término multihilamiento también se utiliza para describir la situación de permitir varios hilos en el mismo proceso. Algunas CPUs tienen soporte directo en el hardware para el multihilamiento y permiten que las conmutaciones de hilos ocurran en una escala de tiempo en nanosegundos.



En la figura podemos ver tres procesos tradicionales. Cada proceso tiene su propio espacio de direcciones y un solo hilo de control.



Acá vemos un solo proceso con tres hilos de control. Aunque en ambos casos tenemos tres hilos, en la primera figura, cada uno de ellos opera en un espacio de direcciones distinto, mientras que en la segunda los tres comparten el mismo espacio de direcciones.

Cuando se ejecuta un proceso con multihilamiento en un sistema con una CPU, los hilos toman turnos para ejecutarse. Al conmutar de un proceso a otro entre varios procesos, el sistema da la apariencia de que hay procesos secuenciales separados ejecutándose en paralelo.

El multihilamiento funciona de la misma manera. La CPU conmuta rápidamente entre un hilo y otro, dando la ilusión de que los hilos se ejecutan en paralelo, aunque en una CPU más lenta que la verdadera. Con tres hilos limitados a cálculos en un proceso, los hilos parecerían ejecutarse en paralelo, cada uno en una CPU con un tercio de la velocidad de la CPU verdadera.

Los distintos hilos en un proceso no son tan independientes como los procesos. Todos los hilos tienen el mismo espacio de direcciones, lo cual significa que también comparten las mismas variables globales. Como cada hilo puede acceder a cada dirección de memoria dentro del espacio de direcciones del proceso, un hilo puede leer, escribir o incluso borrar la pila de otro hilo. No hay protección entre los hilos debido a que (1) es imposible y (2) no debe ser necesario.

A diferencia de tener procesos diferentes, que pueden ser de distintos usuarios y hostiles entre sí, un proceso siempre es propiedad de un solo usuario, quien se supone que ha creado varios hilos para que puedan cooperar, no pelear.

Además de compartir un espacio de direcciones, todos los hilos pueden compartir el mismo conjunto de archivos abiertos, procesos hijos, alarmas y señales, etc.

Si un hilo abre un archivo, ese archivo está visible para los demás hilos en el proceso y pueden leer y escribir datos en él. Esto es lógico, ya que ***el proceso es la unidad de administración de recursos, no el hilo***. Si cada hilo tuviera su propio espacio de direcciones, archivos abiertos, alarmas pendientes, etcétera, sería un proceso separado. Lo que estamos tratando de lograr con el concepto de los hilos es la habilidad de que varios hilos de ejecución compartan un conjunto de recursos, de manera que puedan trabajar en conjunto para realizar cierta tarea.

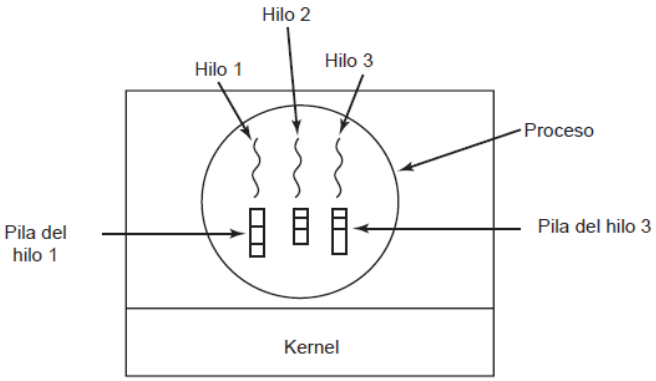
Al igual que un proceso tradicional (es decir, un proceso con sólo un hilo), un hilo puede estar en uno de varios estados: en ejecución, bloqueado, listo o terminado.

Un hilo en ***ejecución*** tiene la CPU en un momento dado y está activo.

Un hilo ***bloqueado*** está esperando a que cierto evento lo desbloquee. Por ejemplo, cuando un hilo realiza una llamada al sistema para leer datos del teclado, se bloquea hasta que se escribe la entrada. Un hilo también puede bloquearse en espera de que ocurra algún evento externo o que algún otro hilo lo desbloquee.

Un hilo ***listo*** se programa para ejecutarse y lo hará tan pronto como sea su turno. Las transiciones entre los estados de los hilos son las mismas que las transiciones entre los estados de los procesos en muchos sentidos.

Es importante tener en cuenta que cada hilo tiene su propia pila.



La pila de cada hilo contiene un conjunto de valores para cada procedimiento llamado, pero del que todavía no se ha regresado. Este conjunto de valores contiene las variables locales del procedimiento y la dirección de retorno que se debe utilizar cuando haya terminado la llamada al procedimiento.

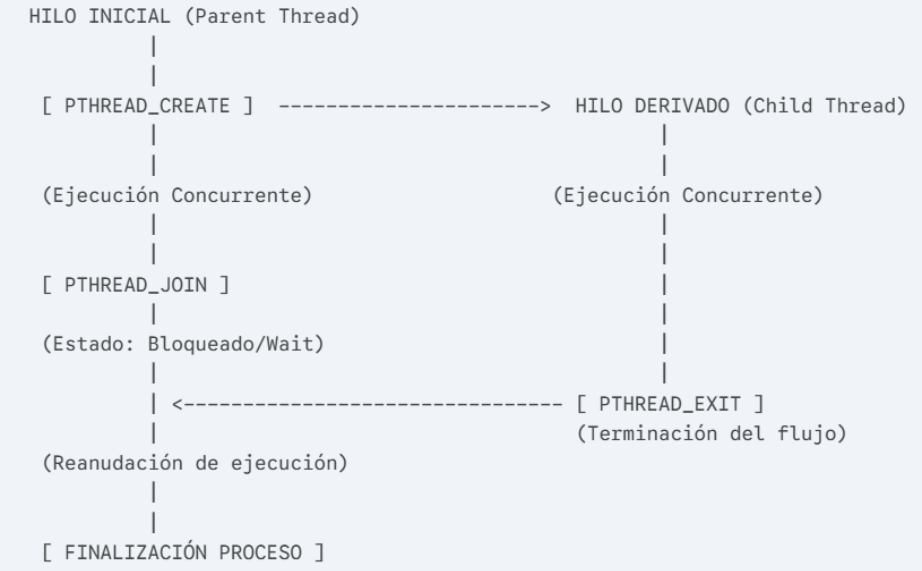
Por ejemplo, si el procedimiento *X* llama al procedimiento *Y* y *Y* llama al procedimiento *Z*, entonces mientras se ejecuta *Z*, los conjuntos de valores para *X*, *Y* y *Z* estarán todos en la pila. Por lo general, cada hilo llama a distintos procedimientos y, por ende, tiene un historial de ejecución diferente. Esta es la razón por la cual cada hilo necesita su propia pila.

Cuando hay multihilamiento, por lo general los procesos empiezan con un solo hilo presente. Este hilo tiene la habilidad de crear hilos mediante la llamada a un procedimiento de biblioteca, como ***thread_create***. Comúnmente, un parámetro para *thread_create* especifica el nombre de un procedimiento para que se ejecute el nuevo hilo.

No es necesario (ni posible) especificar nada acerca del espacio de direcciones del nuevo hilo, ya que se ejecuta de manera automática en el espacio de direcciones del hilo creador. Algunas veces los hilos son jerárquicos, con una relación padre-hijo, pero a menudo no existe dicha relación y todos los hilos son iguales. Con o sin una relación jerárquica, el hilo creador generalmente recibe un identificador de hilo que da nombre al nuevo hilo.

Cuando un hilo termina su trabajo, puede salir mediante la llamada a un procedimiento de biblioteca, como ***thread_exit***. Después desaparece y ya no puede planificarse para volver a ejecutarse.

En algunos sistemas con hilos, un hilo puede esperar a que otro hilo (específico) termine mediante la llamada a un procedimiento, por ejemplo, ***thread_join*.** Este procedimiento bloquea al hilo llamador hasta que un hilo (específico) haya terminado. En este aspecto, la creación y terminación de hilos es algo muy parecido a la creación y terminación de procesos, conteniendo casi las mismas opciones.



Otra llamada de hilos común es ***thread_yield***, que permite a un hilo entregar voluntariamente la CPU para dejar que otro hilo se ejecute. Dicha llamada es importante, ya que no hay una interrupción de reloj para implementar en realidad la multiprogramación, como en los procesos. Por ende, es importante que los hilos sean amables y entreguen de manera voluntaria la CPU de vez en cuando, para dar a otros hilos la oportunidad de ejecutarse.

Llamada de hilo	Descripción
Pthread_create	Crea un nuevo hilo
Pthread_exit	Termina el hilo llamador
Pthread_join	Espera a que un hilo específico termine
Pthread_yield	Libera la CPU para dejar que otro hilo se ejecute
Pthread_attr_init	Crea e inicializa la estructura de atributos de un hilo
Pthread_attr_destroy	Elimina la estructura de atributos de un hilo

Ilustración 13 - Algunas de las llamadas a funciones de Pthreads (POSIX).

Aunque los hilos son útiles a menudo, también introducen cierto número de complicaciones en el modelo de programación. Por ejemplo, tomemos en cuenta el hecho de que los hilos

comparten muchas estructuras de datos. Sería inconveniente que un hilo cierre un archivo mientras otro aún está leyendo datos de él.

En un segundo ejemplo, un hilo detecta que hay muy poca memoria y empieza a asignar más. A mitad del proceso, ocurre una conmutación de hilos y el nuevo hilo también detecta que hay muy poca memoria y también empieza a asignar más memoria. Es muy probable que se asigne memoria dos veces.

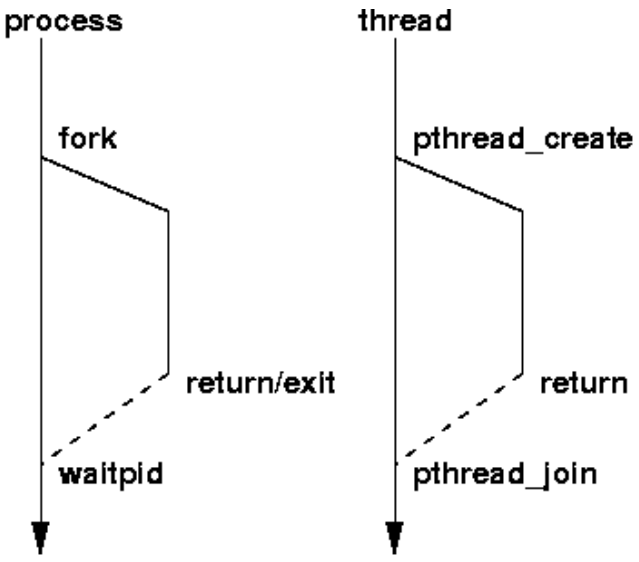
Estos problemas se pueden resolver con cierto esfuerzo de los programadores.

Hilos en POSIX.

Para que sea posible escribir programas con hilos portátiles, el IEEE ha definido un estándar para los hilos. El paquete de hilos que define se conoce como ***Pthreads***. La mayoría de los sistemas UNIX aceptan este paquete. El estándar define más de 60 llamadas a funciones (ya repasamos algunas anteriormente).

Todos los hilos Pthreads tienen ciertas propiedades. Cada uno tiene un identificador, un conjunto de registros (incluyendo el contador de programa) y un conjunto de atributos, que se almacenan en una estructura. Los atributos incluyen el tamaño de la pila, parámetros de planificación y otros elementos necesarios para utilizar el hilo.

Para crear un hilo se utiliza la llamada a ***pthread_create***. El identificador de hilo o el hilo recién creado se devuelven como el valor de la función. La llamada es intencionalmente muy parecida a la llamada al sistema fork, donde el identificador del hilo juega el papel del PID, en buena medida para identificar a los hilos referenciados en otras llamadas.



Cuando un hilo ha terminado el trabajo que se le asignó, puede terminar llamando a ***thread_exit***. Esta llamada detiene el hilo y libera su pila.

A veces, un hilo necesita esperar a que otro termine su trabajo y salga para poder continuar. El hilo que está esperando llama a ***pthread_join*** para esperar a que otro hilo específico termine. El identificador de hilo del hilo al que se va a esperar se proporciona como parámetro. Algunas veces sucede que un hilo no está lógicamente bloqueado, pero determina que se ha ejecutado el tiempo suficiente y desea dar a otro hilo la oportunidad de ejecutarse. Para lograr este objetivo hace una llamada a ***pthread_yield***. No hay una llamada así para los procesos, debido a que se supone que los procesos son en extremo competitivos y cada uno desea tener a la CPU todo el tiempo que pueda. Sin embargo, como los hilos de un proceso están trabajando en conjunto y su código fue escrito indudablemente por el mismo programador, algunas veces éste desea darles a otros hilos una oportunidad.

Las siguientes dos llamadas a hilos tratan con los atributos. ***Pthread_attr_init*** crea la estructura de atributos asociada con un hilo y la inicializa con los valores predeterminados. Estos valores (como la prioridad) se pueden modificar mediante la manipulación de campos en la estructura de atributos. Por último, ***pthread_attr_destroy*** elimina la estructura de atributos de un hilo, liberando su memoria.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMERO_DE_HILOS             10

void *imprimir_hola_mundo(void *tid)
{
    /* Esta funcion imprime el identificador del hilo y después termina. */
    printf("Hola mundo. Saludos del hilo %d0, tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    /* El programa principal crea 10 hilos y después termina. */
    pthread_t hilos[NUMERO_DE_HILOS];
    int estado, i;

    for(i=0; i < NUMERO_DE_HILOS; i++) {
        printf("Aqui main. Creando hilo %d0, i);
        estado = pthread_create(%hilos[i], NULL, imprimir_hola_mundo, (void *)i);

        if (estado != 0) {
            printf("Ups. pthread_create devolvió el codigo de error %d0, estado);
            exit(-1);
        }
    }
    exit(NULL);
}
```

El código de C anterior itera NUMERO_DE_HILOS (10) veces, creando un nuevo hilo en cada iteración, después de anunciar su intención. Si falla la creación del hilo, imprime un mensaje de error y después termina. Después de crear todos los hilos, el programa principal termina.

Cuando se crea un hilo, éste imprime un mensaje de una línea anunciándose a sí mismo y después termina. El orden en el que se intercalan los diversos mensajes no es determinado, y puede variar entre una ejecución del programa y otra.

Las llamadas de Pthreads antes descritas no son en las únicas; hay muchas más.

Referencias y bibliografía de la sección.

- Tanenbaum 2-2 / 2-2-1 / 2-2-2 / 2-2-3.
- Stallings 4.1.

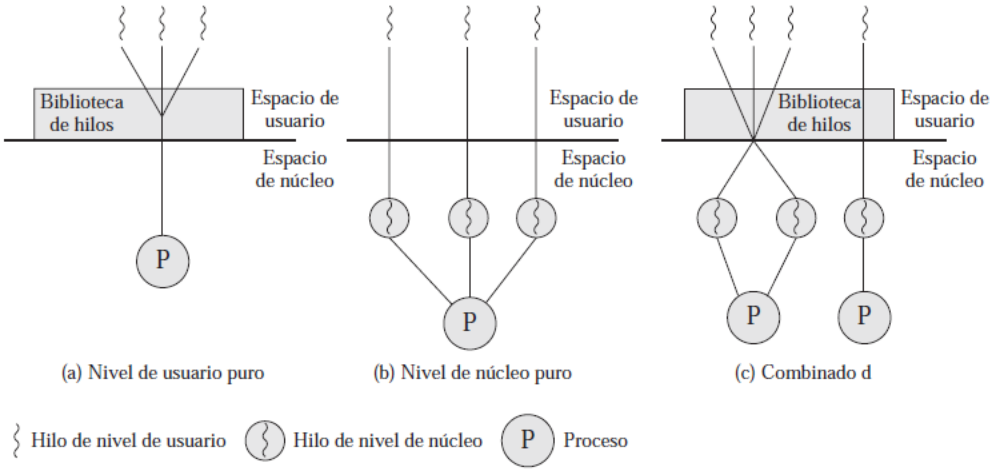
Implementación de hilos a nivel núcleo y a nivel usuario.

Las dos formas fundamentales de implementar un paquete de hilos: en espacio de usuario (user-level threads, ***ULT***) y en el kernel (kernel-level threads, ***KLT***). Los últimos son también conocidos en la literatura como hilos soportados por el núcleo (kernel-supported threads) o procesos ligeros (lightweight processes).

Hilos a nivel de usuario: son implementados en alguna librería. Estos hilos se gestionan sin soporte del SO, el cual solo reconoce un hilo de ejecución.

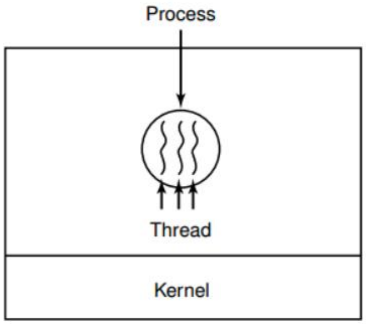
Hilos a nivel de kernel: el SO es quien crea, planifica y gestiona los hilos. Se reconocen tantos hilos como se hayan creado.

La elección de la forma de implementación (ULT vs KLT) puede generar controversia. Es también es posible una implementación ***híbrida***.



Implementación de hilos en el espacio de usuario.

El primer método es colocar el paquete de hilos completamente en espacio de usuario. La aplicación gestiona todo el trabajo de los hilos y el kernel no es consciente de la existencia de los mismos. En lo que al kernel concierne, está administrando procesos ordinarios con un solo hilo.

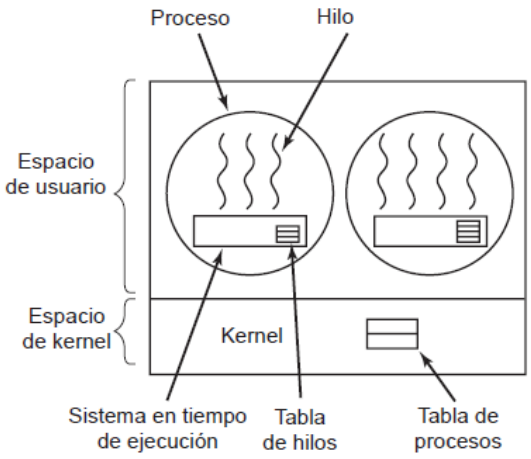


En otras palabras, el Sistema Operativo ni siquiera sabe que existen hilos; solo ve un proceso único.

Quién manda en estos casos es una librería interna del programa que actúa como "gestor de hilos", llamado ***sistema en tiempo de ejecución***. También se lo conoce por los términos ***planificador en espacio de usuario*** (User-Level Scheduler / User-Space Scheduler), ***biblioteca de hilos*** (Thread Library) o ***manejador de hilos*** (Thread Manager).

La primera ventaja, la más obvia, es que un paquete de hilos de nivel usuario puede implementarse en un sistema operativo que no acepte hilos.

En un principio, todos los sistemas operativos solían entrar en esta categoría. Como dijimos, con este método, los hilos se implementan mediante una biblioteca. Todas estas implementaciones tienen la misma estructura general.

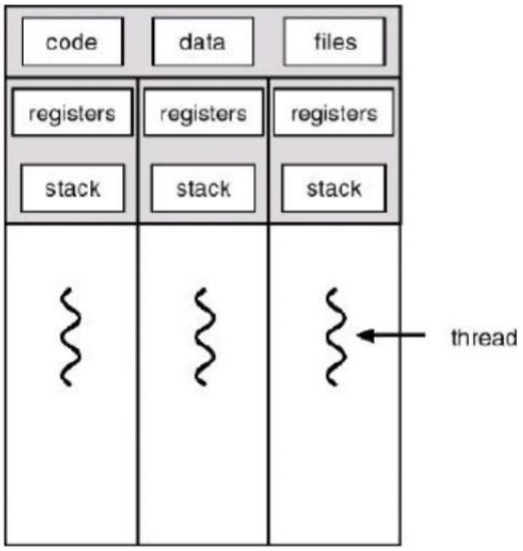


Los hilos se ejecutan encima del ***sistema en tiempo de ejecución***, el cual es una colección de procedimientos que administran hilos.

Cuando los hilos se administran en espacio de usuario, cada proceso necesita su propia tabla de hilos privada para llevar la cuenta de los hilos en ese proceso.



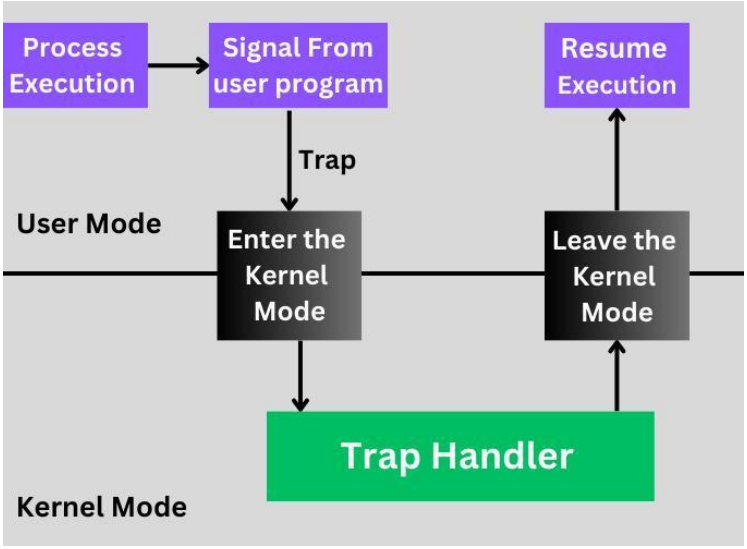
La tabla de hilos es similar a la tabla de procesos del kernel, excepto porque sólo lleva la cuenta de las propiedades por cada hilo, como el contador de programa, apuntador de pila, registros, estado, etc.



La tabla de hilos es administrada por el ***sistema en tiempo de ejecución***. Cuando un hilo pasa al estado listo o bloqueado, la información necesaria para reiniciarlo se almacena en la tabla de hilos, en la misma forma en que el kernel almacena la información acerca de los procesos en la tabla de procesos.

Cuando un hilo hace algo que pueda ponerlo en estado bloqueado dentro del proceso (por ejemplo, esperar a que otro hilo, del mismo proceso, complete cierto trabajo); llama a un procedimiento del sistema en tiempo de ejecución. Este procedimiento comprueba si el hilo debe ponerse en estado bloqueado. De ser así, almacena los registros del hilo (es decir, sus propios registros) en la tabla de hilos, busca en la tabla un hilo listo para ejecutarse y vuelve a cargar los registros de la máquina con los valores guardados del nuevo hilo. Tan pronto como se conmutan el apuntador de pila y el contador de programa, el nuevo hilo vuelve otra vez a la vida de manera automática.

Si la máquina tiene una instrucción especial para guardar todos los registros y otra para cargarlos de una sola vez, toda la conmutación de hilos se puede realizar con sólo unas cuantas instrucciones. Realizar una conmutación de hilos como esta es más veloz que hacer el ***TRAP*** al kernel y es un sólido argumento a favor de los paquetes de hilos de nivel usuario.



Los hilos de nivel usuario también tienen otras ventajas. Permiten que cada proceso tenga su propio ***algoritmo de planificación***[5] personalizado. Por ejemplo, para algunas aplicaciones, las que tienen un hilo recolector de basura, es una ventaja no tener que preocuparse porque un hilo se detenga en un momento inconveniente. También se escalan mejor, ya que los hilos del kernel requieren sin duda algo de espacio en la tabla y en la pila del kernel, lo cual puede ser un problema si hay una gran cantidad de hilos.

Es importante aclarar que la resistencia a los bloqueos por parte del esquema ***ULT*** solo es válido para el caso de "bloqueos internos". Si el hilo llamara a un procedimiento que involucra al Kernel (como una E/S de disco o teclado), se produciría, invariablemente, un ***TRAP***.

De hecho, el problema crítico del esquema ***ULT***, es que si un hilo hace una llamada bloqueante (como el ***scanf***), todo el proceso se detiene. En otras palabras, el punto débil de este esquema es la manera en que se implementan las llamadas al sistema de bloqueo[6].

Supongamos que un hilo lee del teclado antes de que se haya oprimido una tecla. Es inaceptable permitir que el hilo realice la llamada al sistema, ya que esto detendrá a todos los hilos.

Uno de los principales objetivos de tener hilos en primer lugar era permitir que cada uno utilizara llamadas de bloqueo, pero para evitar que un hilo bloqueado afectara a los demás. Con las llamadas al sistema de bloqueo, es difícil ver cómo se puede lograr este objetivo sin problemas.

Todas las llamadas al sistema se podrían cambiar para que quedaran sin bloqueo (por ejemplo, un ***read*** en el teclado sólo devolvería 0 bytes si no hubiera caracteres en el búfer), pero es inconveniente requerir cambios en el sistema operativo. Además, uno de los argumentos para los hilos de nivel usuario era precisamente que se podían ejecutar con los sistemas operativos

[5] Son los encargados de ordenar y dirigir los procesos para asegurar que ninguno de ellos monopolice el uso de la CPU.
[6] Las llamadas al sistema de bloqueo son aquellas que bloquean el hilo/proceso esperando que se disponga del recurso solicitado.

existentes. Además, si se cambia la semántica de ***read*** se requerirán cambios en muchos programas de usuario.

Otro problema con los paquetes de hilos de nivel usuario es que, si un hilo empieza a ejecutarse, ningún otro hilo en ese proceso se ejecutará a menos que el primero renuncie de manera voluntaria a la CPU. Dentro de un solo proceso no hay interrupciones de reloj, lo cual hace que sea imposible planificar procesos en el formato round robin[7] (tomando turnos). A menos que un hilo entre al sistema en tiempo de ejecución por su propia voluntad, el planificador nunca tendrá una oportunidad.

Una posible solución al problema de los hilos que se ejecutan en forma indefinida es hacer que el sistema en tiempo de ejecución solicite una señal de reloj (interrupción) una vez por segundo para dar el control, pero esto también es crudo y complicado para un programa. No siempre son posibles las interrupciones periódicas de reloj a una frecuencia más alta e incluso si lo son, la sobrecarga total podría ser considerable. Lo que es peor: un hilo podría requerir también una interrupción de reloj, interfiriendo con el uso que el sistema en tiempo de ejecución da al reloj.

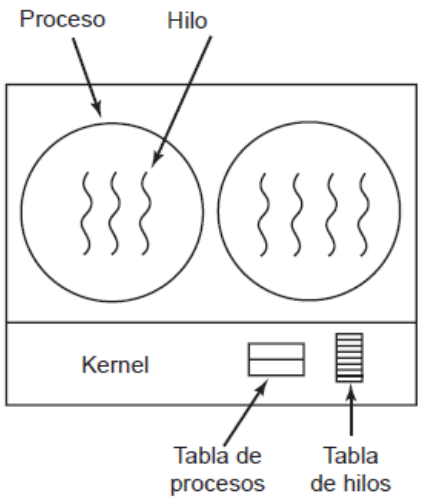
Otro argumento contra los hilos de nivel usuario es que, por lo general, los programadores desean hilos precisamente en aplicaciones donde éstos se bloquean con frecuencia, como, por ejemplo, un servidor Web con multihilado. Estos hilos están realizando llamadas al sistema en forma constante. Una vez que ocurre un trap al kernel, de manera que lleve a cabo la llamada al sistema, no le cuesta mucho al kernel conmutar hilos si el anterior está bloqueado y, al hacer esto el kernel, se elimina la necesidad de realizar llamadas al sistema select[8] en forma constante que comprueben si las llamadas al sistema read son seguras.

Implementación de hilos en el kernel.

Ahora vamos a considerar el caso en que el kernel sabe acerca de los hilos y los administra. No se necesita un sistema en tiempo de ejecución para ninguna de las dos acciones.

[7] Round Robin es un algoritmo que asigna tareas o recursos de manera equitativa. Se utiliza en sistemas operativos y de red.

[8] La llamada select espera a que uno o varios descriptores estén listos para un tipo de actividad de E/S.



Además, no hay tabla de hilos en cada proceso. En vez de ello, el kernel tiene una tabla de hilos que lleva la cuenta de todos los hilos en el sistema.

Cuando un hilo desea crear un nuevo hilo o destruir uno existente, realiza una llamada al kernel[9], la cual se encarga de la creación o destrucción mediante una actualización en la tabla de hilos del kernel.

```
+----------------------+
|   Tabla de Procesos  |
+----------------------+
          |
          v
     +---------+
     |  PCB    |  (cada proceso)
     +---------+
          |
          v
+----------------------+
|   Tabla de Hilos     |  (mantenida en kernel)
+----------------------+
  |     |     |
  v     v     v
+---+ +---+ +---+
|TCB| |TCB| |TCB|   (un TCB por cada hilo)
+---+ +---+ +---+
```

[9] En algunos sistemas operativos se diferencian las llamadas al sistema de las llamadas el kernel. La llamada al kernel no necesariamente proviene de un programa en modo usuario, también puede ser usada por el propio kernel.

La tabla de hilos del kernel contiene los registros, el estado y demás información de cada hilo. Esta información es la misma que con los hilos de nivel usuario, pero ahora se mantiene en el kernel, en vez de hacerlo en espacio de usuario (dentro del sistema en tiempo de ejecución).

Esta información es un subconjunto de la información que mantienen tradicionalmente los kernels acerca de sus procesos con un solo hilo; es decir, el estado del proceso.

Además, el kernel también mantiene la tabla de procesos tradicional para llevar la cuenta de los procesos. Todas las llamadas que podrían bloquear un hilo se implementan como llamadas al sistema, a un costo considerablemente mayor que una llamada a un procedimiento o funcion del sistema en tiempo de ejecución.

Operación	Hilos a nivel de usuario	Hilos a nivel de núcleo	Procesos
Crear Proceso Nulo	34	948	11.300
Señalizar-Esperar	37	441	1.840

Ilustración 14 - Latencia de las Operaciones en Hilos y Procesos (mS).

Cuando un hilo se bloquea, el kernel, según lo que decida, puede ejecutar otro hilo del mismo proceso (si hay uno listo) o un hilo de un proceso distinto.

Esta mecánica es similar a la utilizada por sistemas operativos tipo Linux como Red Hat, en especial cuando se usa ***pthread_create*** (la función de la biblioteca ***POSIX Threads*** encargada de la creación de un nuevo hilo de ejecución dentro de un proceso). En estos casos, quien manda es el planificador del Sistema Operativo.

Similar es el caso de Windows. En C, en lugar de ***pthread_create***, se utiliza la función de la API de Windows llamada ***CreateThread***. Al igual que en Red Hat, cada hilo que se crea con ***CreateThread*** es una entidad que el Kernel de Windows conoce.

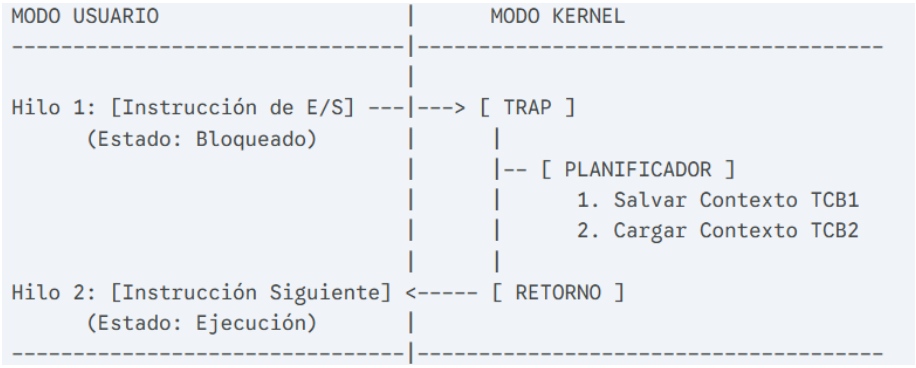
Con los hilos de nivel usuario, el ***sistema en tiempo de ejecución*** ejecuta hilos de su propio proceso hasta que el kernel le quita la CPU (o cuando ya no hay hilos para ejecutar). Debido al costo considerablemente mayor de crear y destruir hilos en el kernel, algunos sistemas optan por un método ambientalmente correcto, reciclando sus hilos.

Cuando se destruye un hilo, se marca como no ejecutable pero las estructuras de datos de su kernel no se ven afectadas de ninguna otra forma. Más adelante, cuando debe crearse un hilo, se reactiva uno anterior, lo que ahorra cierta sobrecarga.

El reciclaje de hilos también es posible para los hilos de nivel usuario, pero como la sobrecarga de la administración de los hilos es mucho menor, hay menos incentivos para hacer esto.

Los hilos de kernel no requieren de nuevas llamadas al sistema sin bloqueo. Además, si un hilo en un proceso produce un fallo de página, el kernel puede comprobar con facilidad si el proceso tiene otros hilos que puedan ejecutarse y de ser así, ejecuta uno de ellos mientras espera a que se traiga la página requerida desde el disco.

Otra ventaja es que, si un hilo se bloquea (por ejemplo, esperando el teclado con ***scanf*** en C), el kernel sabe que solo ese hilo está bloqueado y permite que los demás de ese proceso sigan corriendo en otros núcleos del procesador.



Su principal desventaja es que el costo de una llamada al sistema es considerable, por lo que si las operaciones de hilos (de creación o terminación, por ejemplo) son comunes, se incurrirá en una mayor sobrecarga.

Una cuestión problemática a tener en cuenta (y no es la única) está relacionada con las señales. Las señales se envían a los procesos, no a los hilos, por lo menos en el modelo clásico. Cuando entra una señal, es difícil determinar que hilo se hará cargo de ella. Es posible que los hilos puedan registrar su interés en ciertas señales, de manera que cuando llegue una señal se envíe al hilo que la está esperando. Pero si dos o más hilos se registran para la misma señal podría producirse un conflicto.

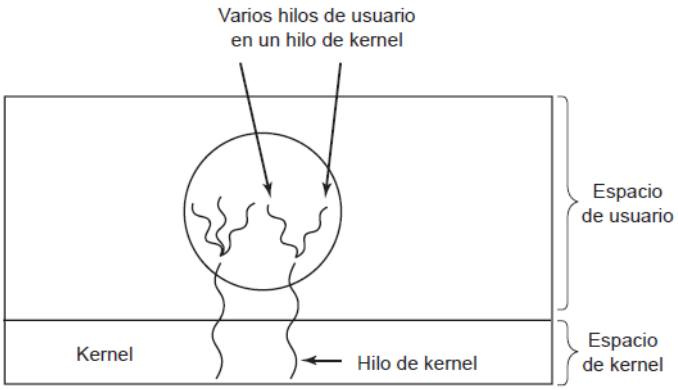
Implementaciones híbridas/combinadas.

Se han investigado varias formas de tratar de combinar las ventajas de los hilos de nivel usuario con los hilos de nivel kernel. Una de esas formas es utilizar hilos de nivel kernel y después multiplexar[10] los hilos de nivel usuario con alguno o con todos los hilos de nivel kernel.

Cuando se utiliza este método, el programador puede determinar cuántos hilos de kernel va a utilizar y cuántos hilos de nivel usuario va a multiplexar en cada uno. Este modelo proporciona mayor flexibilidad.

Solaris es el principal ejemplo de sistema operativo de implementación hibrida. Fue diseñado para combinar la eficiencia de los hilos de usuario con la robustez de los hilos de kernel.

[10] Multiplexar hilos y multithreading son técnicas que permiten procesar varias tareas simultáneamente.



Con este método, el kernel está consciente sólo de los hilos de nivel kernel y los planifica. Algunos de esos hilos pueden tener varios hilos de nivel usuario multiplexados encima de ellos; los hilos de nivel de usuario se crean, destruyen y planifican de igual forma que los hilos de nivel usuario en un proceso que se ejecuta en un sistema operativo sin capacidad de multihilamiento. En este modelo, cada hilo de nivel kernel tiene algún conjunto de hilos de nivel usuario que toman turnos para utilizarlo.

En los enfoques combinados, múltiples hilos de la misma aplicación pueden ejecutar en paralelo en múltiples procesadores, y una llamada al sistema bloqueante no necesita bloquear el proceso completo.

Si el sistema está bien diseñado, este enfoque debería combinar las ventajas de los enfoques puros ULT y KLT, minimizando las desventajas.

Actualmente, sistemas operativo tipo Linux como Red Hat utilizan la ***NPTL*** (Native POSIX Thread Library), que sigue un Modelo 1:1 (KLT puro). Sin embargo, se puede implementar un comportamiento híbrido bajo las ciertas condiciones.

A nivel de aplicación, se pueden utilizar bibliotecas que gestionen hilos de usuario sobre los hilos de kernel que provee Red Hat.

Tanto ***Go***[11] como ***Erlang***[12] son los ejemplos modernos destacados de lo que se denomina un modelo híbrido o modelo ***M:N***. Aunque el núcleo de Red Hat opere internamente con un modelo ***1:1*** (KLT), estos lenguajes ejecutan su propia infraestructura sobre él para simular un comportamiento híbrido.

Relación entre ULT y KLT.

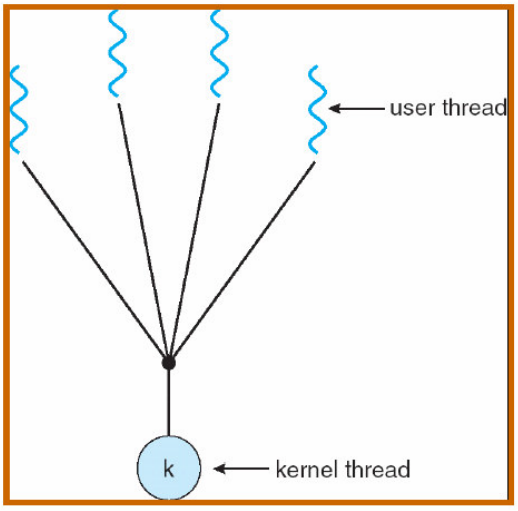
[11] Go (o Golang) es un lenguaje de programación de código abierto, compilado y concurrente, desarrollado por Google, conocido por su simplicidad, alto rendimiento y seguridad de memoria. Diseñado para la nube, redes y microservicios, destaca por su rápida compilación y manejo eficiente de la concurrencia, popularizándose en infraestructuras modernas.

[12] Erlang es un lenguaje de programación funcional, concurrente y de propósito general, diseñado por Ericsson para sistemas distribuidos de tiempo real, alta disponibilidad y tolerancia a fallos.

Modelo Mx1 (Many to one).

El modelo asigna múltiples hilos de usuario a un hilo del kernel. Este caso se corresponde a los hilos implementados a nivel de usuario, ya que el sistema solo reconoce un hilo de control para el proceso.

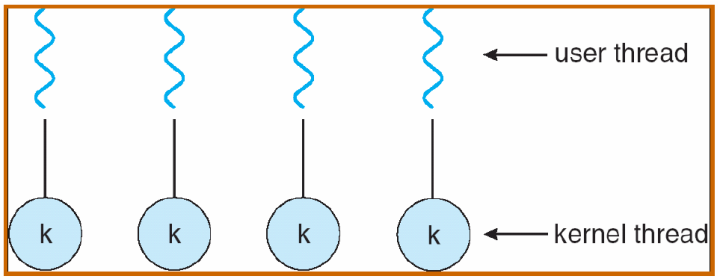
Tiene como inconveniente que, si un hilo se bloquea, todo el proceso se bloquea. También, dado que solo un hilo puede acceder al kernel cada vez, no podrán ejecutarse varios hilos en paralelo en múltiples CPUs.



Modelo 1x1 (One to one).

El modelo asigna cada hilo de usuario a un hilo del kernel. Proporciona una mayor concurrencia que el modelo anterior, permitiendo que se ejecute otro hilo si uno se bloqueó.

Tiene como inconveniente que cada vez que se crea un hilo a nivel de usuario, se crea un hilo a nivel del kernel, y la cantidad de hilos a nivel del kernel están restringidos en la mayoría de los sistemas.

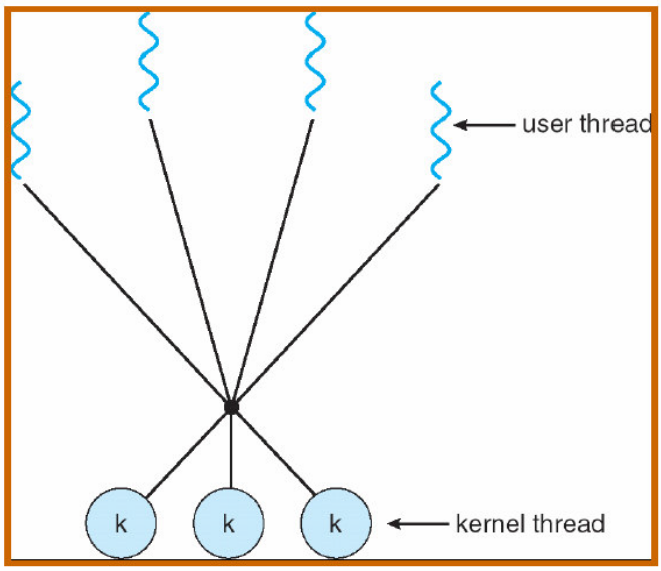


Modelo MxM (Many to many).

El modelo multiplexa muchos hilos de usuario sobre un número menor o igual de hilos del kernel. Cada proceso tiene asignado un conjunto de hilos de kernel, independientemente de la cantidad de hilos de usuario que haya creado.

No posee ninguno de los inconvenientes de los dos modelos anteriores, ya que saca lo mejor de cada uno. El usuario puede crear tantos hilos como necesite y los hilos de kernel pueden ejecutar en paralelo. Asimismo, cuando un hilo se bloquea, el kernel puede planificar otro hilo para su ejecución.

Entonces, el planificador a nivel de usuario asigna los hilos de usuario a los hilos de kernel, y el planificador a nivel de kernel asigna los hilos de kernel a los procesadores.



Referencias y bibliografía de la sección.

- Tanenbaum 2-2-4 / Tanenbaum 2-2-5.
- Stallings 4.1. pág. 165.

Comunicación entre procesos.

Con frecuencia, los procesos necesitan comunicarse con otros procesos. De esta necesidad surgen tres cuestiones fundamentales a tener en cuenta. La primera se alude a cómo un proceso puede pasar información a otro.

La segunda está relacionada con hacer que dos o más procesos no se interpongan entre sí; por ejemplo, dos procesos en un sistema de reservaciones de una aerolínea, cada uno de los cuales trata de obtener el último asiento en un avión para un cliente distinto.

La tercera trata acerca de obtener la secuencia apropiada cuando hay dependencias presentes: si el proceso *A* produce datos y el proceso *B* los imprime, *B* tiene que esperar hasta que *A* haya producido algunos datos antes de empezar a imprimir.

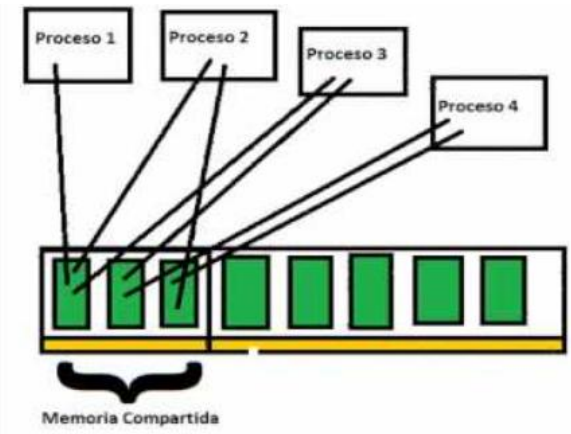
Vale la pena analizar estas cuestiones también a nivel de hilos. La primera (el paso de información) es fácil para los hilos, ya que comparten un espacio de direcciones común (los hilos en distintos espacios de direcciones que necesitan comunicarse entran en la categoría de los procesos en comunicación).

Sin embargo, las otras dos (evitar que un hilo entre en conflicto con los demás y obtener la secuencia apropiada) se aplican de similar forma a los hilos que a los procesos. Existen los mismos problemas y se aplican las soluciones parecidas.

Los procesos que se ejecutan concurrentemente pueden ser procesos independientes o procesos cooperativos.

Un proceso es independiente si no puede afectar o verse afectado por los restantes procesos que se ejecutan en el sistema. Cualquier proceso que no comparte datos con ningún otro proceso es un proceso independiente.

Un proceso es cooperativo si puede afectar o verse afectado por los demás procesos que se ejecutan en el sistema. Evidentemente, cualquier proceso que comparte datos con otros procesos es un proceso cooperativo.



Hay varias razones para proporcionar un entorno que permita la cooperación entre procesos:

- **Compartir información.** Dado que varios procesos pueden estar interesados en la misma información (por ejemplo, un archivo compartido), debemos proporcionar un entorno que permita el acceso concurrente a dicha información.
- **Acelerar los cálculos.** Si deseamos que una determinada tarea se ejecute rápidamente, podemos dividirla en subtareas, ejecutándose cada una de ellas de forma paralela. Hay

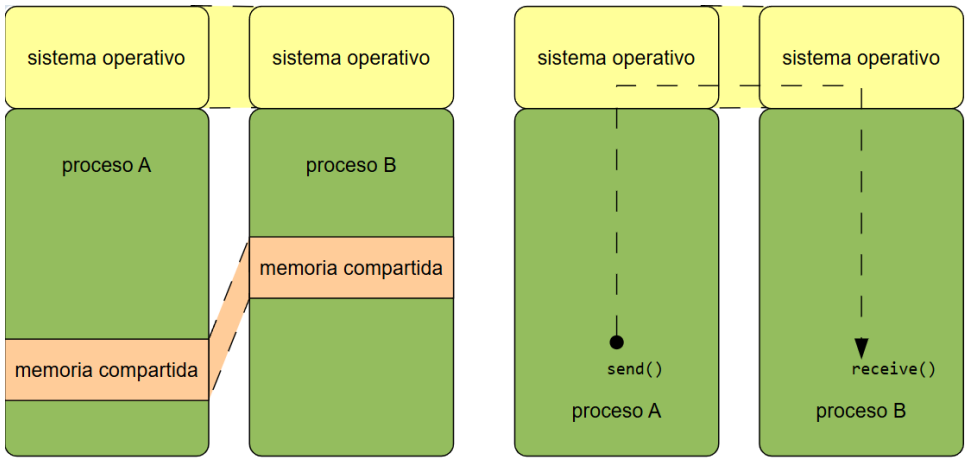
que aclarar que tal aceleración sólo se puede conseguir si la computadora tiene múltiples elementos de procesamiento, como por ejemplos varias CPU.

- **Modularidad.** Podría ser útil construir el sistema de forma modular, dividiendo las funciones del sistema en diferentes procesos o hilos.

La cooperación entre procesos requiere mecanismos de comunicación interprocesos (***IPC***, interprocess communication) que les permitan intercambiar datos e información. Existen dos modelos fundamentales de comunicación interprocesos: (1) memoria compartida y (2) paso de mensajes.

En el modelo de ***memoria compartida***, se establece una región de la memoria para que sea compartida por los procesos cooperativos. De este modo, los procesos pueden intercambiar información leyendo y escribiendo datos en la zona compartida.

En el modelo de ***paso de mensajes***, la comunicación tiene lugar mediante el intercambio de mensajes, entre los procesos cooperativos.



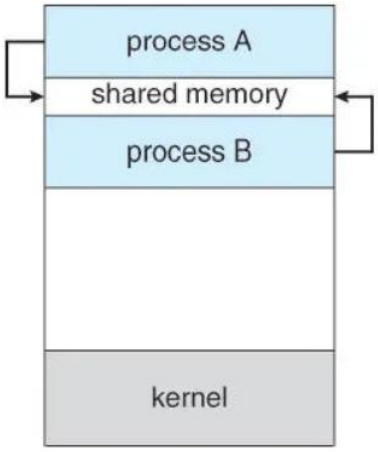
Los dos modelos son bastante comunes en los distintos sistemas operativos y muchos sistemas implementan ambos.

El paso de mensajes resulta útil para intercambiar pequeñas cantidades de datos, ya que no existe la necesidad de evitar conflictos. El paso de mensajes también es más fácil de implementar que el modelo de memoria compartida como mecanismo de comunicación entre diferentes computadoras.

La memoria compartida permite una velocidad máxima y una mejor comunicación, ya que puede realizarse a velocidades de memoria cuando se hace en una misma computadora. La memoria compartida es más rápida que el paso de mensajes, ya que este último método se implementa normalmente usando llamadas al sistema y, por tanto, requiere que intervenga el kernel, lo que consume más tiempo. Por el contrario, en los sistemas de memoria compartida, las llamadas al sistema sólo son necesarias para establecer las zonas de memoria compartida. Una vez establecida la memoria compartida, todos los accesos se tratan como accesos a memoria rutinarios y no se precisa la ayuda del kernel.

Sistemas de memoria compartida.

La comunicación interprocesos que emplea memoria compartida requiere que los procesos que se estén comunicando establezcan una región de memoria compartida. Normalmente, una región de memoria compartida reside en el espacio de direcciones del proceso que crea el segmento de memoria compartida. Otros procesos que deseen comunicarse usando este segmento de memoria compartida deben conectarse a su espacio de direcciones.



Este esquema requiere que el programador de la aplicación escriba explícitamente el código para acceder y manipular la memoria compartida.

Hay que tener en cuenta que habitualmente, el sistema operativo intenta evitar que un proceso acceda a la memoria de otro proceso. Por este motivo, se requiere que dos o más procesos acuerden eliminar esta restricción. Entonces podrán intercambiar información leyendo y escribiendo datos en las áreas compartidas.

El formato de los datos y su ubicación están determinados por estos procesos, y no se encuentran bajo el control del sistema operativo. Los procesos también son responsables de verificar que no escriben en la misma posición simultáneamente.

Consideremos el problema del ***productor-consumidor***, el cual es un paradigma comúnmente utilizado para los procesos cooperativos. Un proceso productor genera información que consume un proceso consumidor. Por ejemplo, un compilador puede generar código ensamblado, que consume un ensamblador. El ensamblador, a su vez, puede generar módulos objeto, que consume el cargador. El problema del productor-consumidor también proporciona una metáfora muy útil para el paradigma cliente-servidor.

Generalmente, pensamos en un servidor como en un productor y en un cliente como en un consumidor. Por ejemplo, un servidor web produce (es decir, proporciona) archivos HTML e imágenes, que consume (es decir, lee) el explorador web cliente que solicita el recurso.

Una solución para el problema del productor-consumidor es utilizar mecanismos de memoria compartida.

Para permitir que los procesos productor y consumidor se ejecuten de forma concurrente, debemos tener disponible un búfer de elementos que pueda rellenar el productor y vaciar el consumidor. Este búfer residirá en una región de memoria que será compartida por ambos procesos, consumidor y productor.

Un productor puede generar un elemento mientras que el consumidor consume otro. El productor y el consumidor deben estar sincronizados, de modo que el consumidor no intente consumir un elemento que todavía no haya sido producido.

Pueden emplearse dos tipos de búferes. El sistema de búfer no limitado no pone límites al tamaño de esa memoria compartida. El consumidor puede tener que esperar para obtener elementos nuevos, pero el productor siempre puede generar nuevos elementos.

El sistema de búfer limitado establece un tamaño de búfer fijo. En este caso, el consumidor tiene que esperar si el búfer está vacío y el productor tiene que esperar si el búfer está lleno.

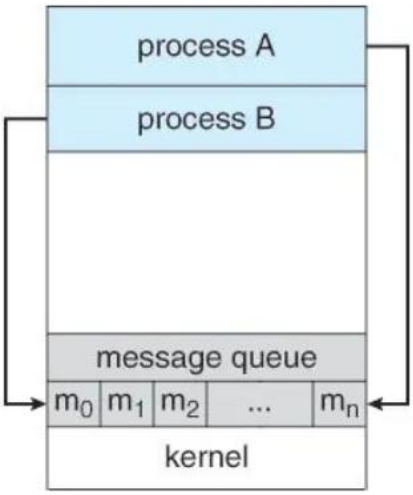
Un problema a tener en cuenta es la situación en la que tanto el proceso productor como el consumidor intentan acceder al búfer compartido de forma concurrente. Los programadores tendrán que considerar cómo puede implementarse la sincronización entre procesos cooperativos de forma efectiva en un entorno de memoria compartida.

Al igual que en otros sistemas basados en el estándar POSIX, la implementación de la memoria compartida puede ser llevada a cabo por herramientas suministradas por medio de librerías (ej.: la función ***shm_open()*** crea o abre un objeto de memoria compartida).

Sistemas de paso de mensajes.

Otra forma de conseguir la comunicación entre procesos es que el sistema operativo proporcione los medios para que los procesos cooperativos se comuniquen entre sí a través de una facilidad de paso de mensajes.

El paso de mensajes proporciona un mecanismo que permite a los procesos comunicarse y sincronizar sus acciones sin compartir el mismo espacio de direcciones, y es especialmente útil en un entorno distribuido, en el que los procesos que se comunican pueden residir en diferentes computadoras conectadas en red. Por ejemplo, un programa de chat utilizado en la World Wide Web podría diseñarse de modo que los procesos participantes en la conversación se comunicaran entre sí intercambiando mensajes.

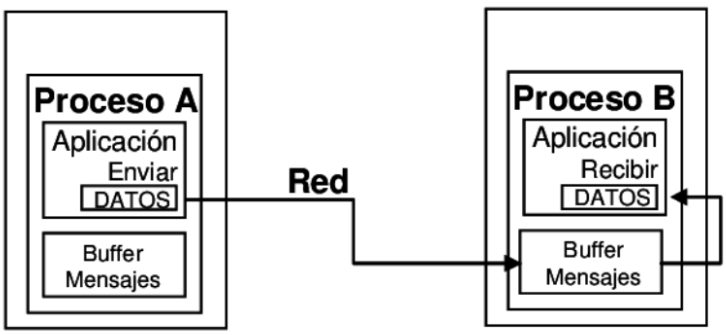


El paso de mensajes proporciona al menos dos operaciones: envío de mensajes (***send***) y recepción de mensajes (***receive***). Los mensajes enviados por un proceso pueden tener un tamaño fijo o variable. Si sólo se pueden enviar mensajes de tamaño fijo, la implementación en el nivel de sistema es directa. Sin embargo, esta restricción hace que la tarea de programación sea más complicada. Por el contrario, los mensajes de tamaño variable requieren una implementación más compleja en el nivel de sistema, pero la tarea de programación es más sencilla.

Éste es un tipo de compromiso que se encuentra muy habitualmente en el diseño de sistemas operativos. Si los procesos P y Q desean comunicarse, tienen que enviarse mensajes entre sí; debe existir un enlace de comunicaciones entre ellos. Este enlace se puede implementar de diferentes formas.

Existen varios métodos para implementar lógicamente (no nos estamos refiriendo al aspecto físico, hardware) un enlace y las operaciones de envío y recepción:

- Comunicación directa o indirecta.
- Comunicación síncrona o asíncrona.
- Almacenamiento en búfer explícito o automático.



En sistemas Linux modernos, en general, el paso de mensajes se puede implementar a través de la API de POSIX. En sistemas operativos como Red Hat, esta implementación podría realizarse con relativa facilidad con las herramientas como la librería ***mqueue***.

Nombrado.

Los procesos que se van a comunicar deben disponer de un modo de referenciarse entre sí. Pueden usar comunicación directa o indirecta.

En el caso de la comunicación ***directa***, cada proceso que desea establecer una comunicación debe nombrar de forma explícita al receptor o transmisor de la comunicación. En este esquema, las primitivas[13] send () y receive () se definen del siguiente modo:

- send (P, mensaje) - Envía un mensaje al proceso P.
- receive (Q, mensaje) - Recibe un mensaje del proceso Q.

Un enlace de comunicaciones, según este esquema, tiene las siguientes propiedades:

- Los enlaces se establecen de forma automática entre cada par de procesos que quieran comunicarse. Los procesos sólo tienen que conocer la identidad del otro para comunicarse.
- Cada enlace se asocia con exactamente dos procesos.
- Entre cada par de procesos existe exactamente un enlace.

Este esquema presenta ***simetría*** en lo que se refiere al direccionamiento, es decir, tanto el proceso transmisor como el proceso receptor deben nombrar al otro para comunicarse.



Existe una variante de este esquema que emplea ***asimetría*** en el direccionamiento. En este caso, sólo el transmisor nombra al receptor; el receptor no tiene que nombrar al transmisor. En este esquema, las primitivas send () y receive () se definen del siguiente modo:

- send (P, mensaje)— Envía un mensaje al proceso P.
- receive (id, mensaje)— Recibe un mensaje de cualquier proceso; a la variable id se le asigna el nombre del proceso con el que se ha llevado a cabo la comunicación.

[13] Una primitiva es uno de los elementos fundamentales del lenguaje de programación y proporcionan los bloques de construcción básicos para llevar a cabo operaciones específicas. Es uno de los elementos más simples disponibles en un lenguaje de programación. En sistemas operativos, una primitiva es una operación o concepto fundamental que actúa como un bloque de construcción para tareas más complejas, como primitivas de sincronización (semáforos, monitores, etc.) para coordinar hilos, y primitivas de comunicación para intercambiar datos entre procesos.

La desventaja de estos dos esquemas (simétrico y asimétrico) es la limitada modularidad de las definiciones de procesos resultantes. Cambiar el identificador de un proceso puede requerir que se modifiquen todas las restantes definiciones de procesos. Deben localizarse todas las referencias al identificador antiguo, para poder sustituirlas por el nuevo identificador.

En general, cualquier técnica de precodificación, en la que los identificadores deban establecerse explícitamente, es menos deseable que las técnicas basadas en la indirección.

Con el modelo de comunicación ***indirecta***, los mensajes se envían y reciben en buzones de correo o puertos. Un buzón de correo puede verse de forma abstracta como un objeto en el que los procesos pueden colocar mensajes y del que pueden eliminar mensajes. Cada buzón de correo tiene asociada una identificación unívoca. Por ejemplo, las colas de mensajes de POSIX usan un valor entero para identificar cada buzón de correo. En este esquema, un proceso puede comunicarse con otros procesos a través de una serie de buzones de correo diferentes. Sin embargo, dos procesos sólo se pueden comunicar si tienen un buzón de correo compartido. Las primitivas send () y receive () se definen del siguiente modo:

- send(A, mensaje)— Envía un mensaje al buzón de correo A.
- receive(A, mensaje)— Recibe un mensaje del buzón de correo A.



En este esquema, un enlace de comunicaciones tiene las siguientes propiedades:

- Puede establecerse un enlace entre un par de procesos sólo si ambos tienen un buzón de correo compartido.
- Un enlace puede asociarse con más de dos procesos.
- Entre cada par de procesos en comunicación, puede haber una serie de enlaces diferentes, correspondiendo cada enlace a un buzón de correo.

Un buzón de correo puede ser propiedad de un proceso o del sistema operativo. Si es propiedad de un proceso, es decir, si el buzón de correo forma parte del espacio de direcciones del proceso, entonces podemos diferenciar entre el propietario (aquél que sólo recibe mensajes a través de este buzón) y el usuario (aquél que sólo puede enviar mensajes a dicho buzón de correo). Puesto que cada buzón de correo tiene un único propietario, no puede haber confusión acerca de quién recibirá un mensaje enviado a ese buzón de correo. Cuando un proceso que posee un buzón de correo termina, dicho buzón desaparece. A cualquier proceso que con posterioridad envíe un mensaje a ese buzón debe notificársele que dicho buzón ya no existe.

Por el contrario, un buzón de correo que sea propiedad del sistema operativo tiene existencia propia, es independiente y no está asociado a ningún proceso concreto. El sistema operativo debe proporcionar un mecanismo que permita a un proceso hacer lo siguiente:

- Crear un buzón de correo nuevo.
- Enviar y recibir mensajes a través del buzón de correo.
- Eliminar un buzón de correo.

Por omisión, el proceso que crea un buzón de correo nuevo es el propietario del mismo. Inicialmente, el propietario es el único proceso que puede recibir mensajes a través de este buzón. Sin embargo, la propiedad y el privilegio de recepción se pueden pasar a otros procesos mediante las apropiadas llamadas al sistema. Por supuesto, esta medida puede dar como resultado que existan múltiples receptores para cada buzón de correo.

Sincronización.

La comunicación entre procesos tiene lugar a través de llamadas a las primitivas send() y receive(). Existen diferentes opciones de diseño para implementar cada primitiva. El paso de mensajes puede ser con bloqueo o sin bloqueo, mecanismos también conocidos como síncrono y asíncrono.

- **Envío con bloqueo.** El proceso que envía se bloquea hasta que el proceso receptor o el buzón de correo reciben el mensaje.
- **Envío sin bloqueo.** El proceso transmisor envía el mensaje y continúa operando.
- **Recepción con bloqueo.** El receptor se bloquea hasta que hay un mensaje disponible.
- **Recepción sin bloqueo.** El receptor extrae un mensaje válido o un mensaje nulo.

Son posibles diferentes combinaciones de las operaciones send() y receive(). Cuando ambas operaciones se realizan con bloqueo, tenemos lo que se denomina un ***rendezvous***[14] entre el transmisor y el receptor. La solución al problema del productor-consumidor es trivial cuando se usan instrucciones send () y receive () con bloqueo. El productor simplemente invoca la llamada send() con bloqueo y espera hasta que el mensaje se entrega al receptor o al buzón de correo. Por otro lado, cuando el consumidor invoca la llamada receive(), se bloquea hasta que haya un mensaje disponible.

Almacenamiento en búfer.

Sea la comunicación directa o indirecta, los mensajes intercambiados por los procesos que se están comunicando residen en una cola temporal. Básicamente, tales colas se pueden implementar de tres maneras:

- **Capacidad cero.** La cola tiene una longitud máxima de cero; por tanto, no puede haber ningún mensaje esperando en el enlace. En este caso, el transmisor debe bloquearse hasta que el receptor reciba el mensaje.

[14] Rendezvous es un mecanismo de sincronización que permite a dos procesos concurrentes, el solicitante y el llamado, intercambiar datos de forma coordinada. El proceso que solicita el rendezvous debe esperar en el punto de reencuentro hasta que el proceso llamado llegue allí.

- **Capacidad limitada.** La cola tiene una longitud finita n; por tanto, puede haber en ella n mensajes como máximo. Si la cola no está llena cuando se envía un mensaje, el mensaje se introduce en la cola (se copia el mensaje o se almacena un puntero al mismo), y el transmisor puede continuar la ejecución sin esperar. Sin embargo, la capacidad del enlace es finita. Si el enlace está lleno, el transmisor debe bloquearse hasta que haya espacio disponible en la cola.
- **Capacidad ilimitada.** La longitud de la cola es potencialmente infinita; por tanto, puede haber cualquier cantidad de mensajes esperando en ella. El transmisor nunca se bloquea.

En ocasiones, se dice que el caso de capacidad cero es un sistema de mensajes sin almacenamiento en búfer; los otros casos se conocen como sistemas con almacenamiento en búfer automático.

Referencias y bibliografía de la sección.

- Silberschatz 3-4 / 3-4-1 / 3-4-2.
- Tanenbaum 2-3.
- Luis Castellanos – Sistemas Operativos 3-2.

Más herramientas de sincronización y comunicación en UNIX.

Dependiendo del sistema operativo que se utilice, estarán disponibles o no diferentes herramientas de sincronización de procesos. ***UNIX*** proporciona diversos mecanismos de comunicación y sincronización, entre los más importantes podemos destacar:

- Tuberías (*pipes*).
- Mensajes.
- Memoria compartida.
- Semáforos (visto anteriormente).
- Señales.

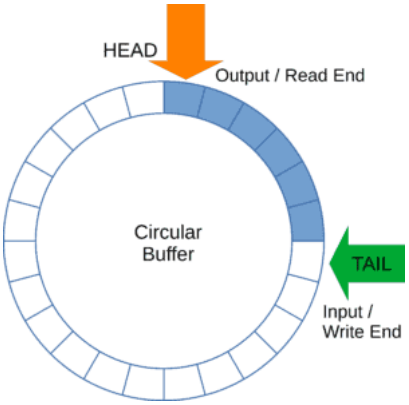
Las tuberías, los mensajes y la memoria compartida pueden utilizarse para comunicar datos entre procesos, mientras que las señales se utilizan para disparar acciones en otros procesos.

Vale remarcar que muchas de estos mecanismos no son más que versiones de las herramientas de sincronización vistas anteriormente adaptadas a los sistemas operativos tipo UNIX.

Tuberías.

Una de las contribuciones más significativas de UNIX en el desarrollo de los sistemas operativos es la ***tubería***. Una tubería es un ***buffer*** circular que permite que dos procesos se

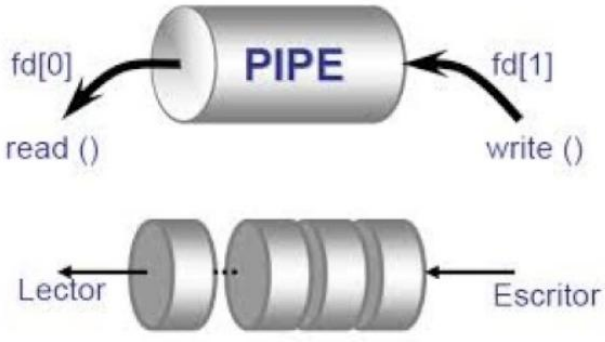
comuniquen siguiendo el modelo productor-consumidor. Por tanto, se trata de una cola en la que el primero en entrar es el primero en salir.



Cuando se crea una tubería, se le establece un tamaño fijo en bytes. Cuando un proceso intenta escribir en la tubería, la petición de escritura se ejecuta inmediatamente si hay suficiente espacio; en caso contrario, el proceso se bloquea. De manera similar, un proceso que lee se bloquea si intenta leer más bytes de los que están actualmente en la tubería; en caso contrario, la petición de lectura se ejecuta inmediatamente.

El sistema operativo asegura la exclusión mutua: es decir, en cada momento sólo puede acceder a una tubería un único proceso.

Hay dos tipos de tuberías: con nombre y sin nombre. Sólo los procesos relacionados pueden compartir tuberías sin nombre, mientras que, los procesos pueden compartir tuberías con nombre tanto si están relacionados como si no.



Mensajes.

Un mensaje es un conjunto de bytes con un tipo asociado. UNIX proporciona las llamadas al sistema ***msgsnd*** y ***msgrcv*** para que los procesos puedan realizar la transferencia de mensajes. Asociada con cada proceso existe una ***cola de mensajes***, que funciona como un buzón.

El emisor del mensaje especifica el tipo en cada mensaje que envía, de manera que este tipo puede utilizarse como un criterio de selección por el receptor. El receptor puede recuperar los mensajes tanto en orden de llegada o por su tipo.

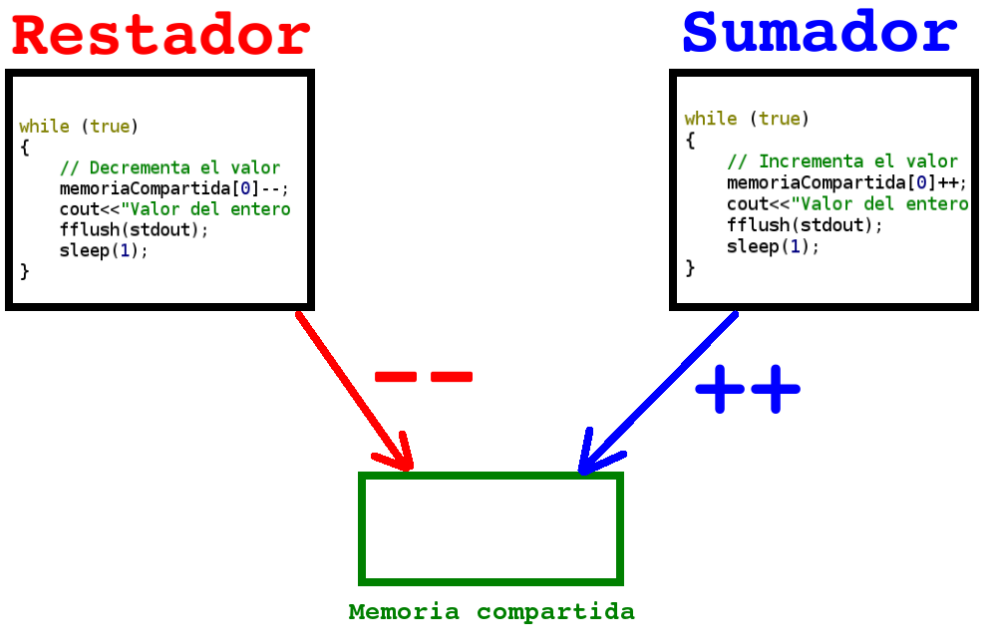
Un proceso se bloqueará cuando intente enviar un mensaje a una cola llena. Un proceso también se bloqueará cuando intente leer un mensaje de una cola vacía.

Si un proceso intenta leer un mensaje de un cierto tipo y no es posible debido a que no está presente ningún mensaje de este tipo, el proceso no se bloquea.

Memoria compartida.

La forma más rápida de comunicación entre procesos proporcionada en UNIX es la ***memoria compartida***. Se trata de un bloque de ***memoria virtual*** compartido por múltiples procesos. Los procesos leen y escriben en la memoria compartida utilizando las mismas instrucciones de máquina que se utilizan para leer y escribir otras partes de su espacio de memoria virtual.

El permiso para un determinado proceso puede ser de sólo lectura o de lectura y escritura, estableciéndose de forma individual para cada proceso. Las restricciones de exclusión mutua no son parte de este mecanismo de memoria compartida, sino que las deben proporcionar los procesos que utilizan la memoria compartida.



Señales.

Una señal es un mecanismo software que informa a un proceso de la existencia de eventos asíncronos.

Una señal es similar a las interrupciones hardware, pero no emplea prioridades. Es decir, todas las señales se tratan por igual; las señales que ocurren al mismo tiempo se le presentan al proceso una detrás de otra, sin ningún orden en particular.

Los procesos pueden enviarse señales entre sí, o puede ser el núcleo quien envíe señales internamente.



Para entregar una señal, se actualiza un campo en la tabla de procesos correspondiente al proceso al que se le está enviando la señal. Dado que por cada señal se mantiene un único bit, las señales de un determinado tipo no pueden encolarse.

Una señal sólo se procesa después de que un proceso se despierte para ejecutar o cuando el proceso está retornando de una llamada al sistema.

Un proceso puede responder a una señal realizando alguna acción por defecto (por ejemplo, su terminación), ejecutando una función de manejo de la señal o ignorando la señal.

Valor	Nombre	Descripción
01	SIGHUP	Desconexión; enviada al proceso cuando el núcleo asume que el usuario de ese proceso no está haciendo trabajo útil
02	SIGINT	Interrupción
03	SIGQUIT	Abandonar; enviada por el usuario para provocar la parada del proceso y la generación de un volcado de su memoria (*core dump*)
04	SIGILL	Instrucción ilegal
05	SIGTRAP	*Trap* de traza; activa la ejecución de código para realizar un seguimiento de la ejecución del proceso
06	SIGIOT	Instrucción IOT
07	SIGEMT	Instrucción EMT
08	SIGFPE	Excepción de coma flotante
09	SIGKILL	Matar; terminar el proceso
10	SIGBUS	Error de bus
11	SIGSEGV	Violación de segmento; el proceso intenta acceder a una posición fuera de su espacio de direcciones
12	SIGSYS	Argumento erróneo en una llamada al sistema
13	SIGPIPE	Escritura sobre una tubería que no tiene lectores asociados
14	SIGALRM	Alarma; emitida cuando un proceso desea recibir una señal cuando transcurra un determinado periodo de tiempo
15	SIGTERM	Terminación por software
16	SIGUSR1	Señal 1 definida por el usuario
17	SIGUSR2	Señal 2 definida por el usuario
18	SIGCHLD	Muerte de un proceso hijo
19	SIGPWR	Interrupción en el suministro de energía

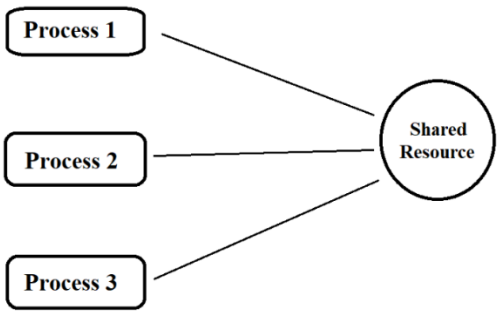
Referencias y bibliografía de la sección.

- Stallings 6-7 pág. 280.

Competencia entre procesos.

Los procesos concurrentes entran en conflicto entre ellos cuando compiten por el uso del mismo recurso. Hablamos de competencia cuando dos o más procesos necesitan acceso a un recurso durante el curso de su ejecución.

Ningún proceso se percata de la existencia de los otros procesos y ninguno debe verse afectado por la ejecución de los otros procesos. Esto conlleva que cada proceso debe dejar inalterado el estado de cada recurso que utilice. Ejemplos de recursos son los dispositivos de E/S, la memoria, el tiempo de procesador y el reloj.

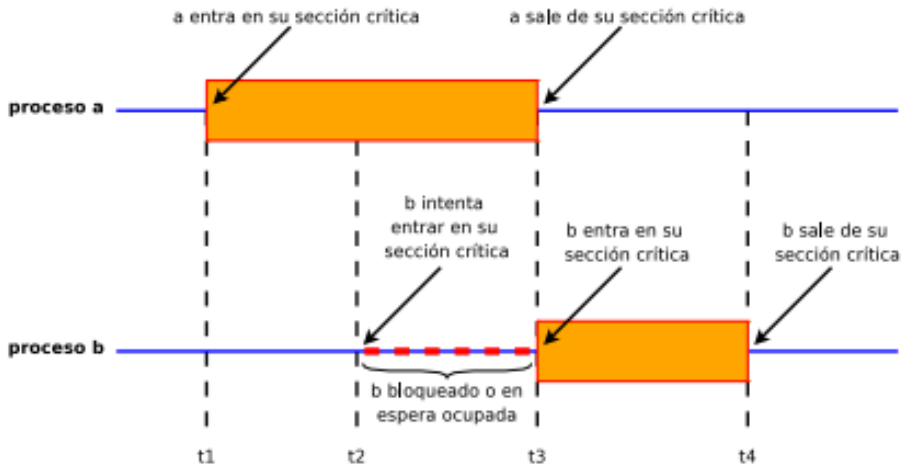


La ejecución de un proceso puede afectar al comportamiento de los procesos en competencia. En concreto, si dos procesos desean acceder al mismo recurso único, entonces, el sistema operativo reservará el recurso para uno de ellos, y el otro tendrá que esperar. Por lo tanto, el proceso al que se le deniega el acceso será ralentizado. En un caso extremo, el proceso bloqueado puede no conseguir jamás el recurso y por tanto no terminar nunca satisfactoriamente.

En el caso de procesos en competencia, deben afrontarse tres problemas de control. Primero está la necesidad de ***exclusión mutua***. Supongamos que dos o más procesos requieren acceso a un recurso único no compartible, como una impresora. Durante el curso de la ejecución, cada proceso estará enviando comandos al dispositivo de E/S, recibiendo información de estado, enviando datos o recibiendo datos.

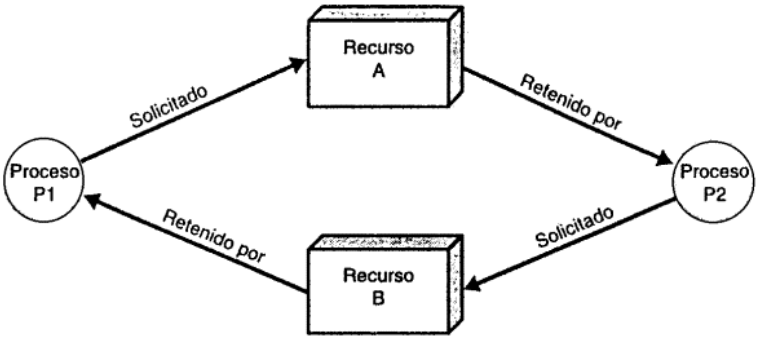
En este caso, podríamos referirnos a este recurso como un ***recurso crítico***, y a la porción del programa que lo utiliza como la ***sección crítica*** del programa.

Es importante que sólo se permita un programa, en un instante determinando, ejecutando su sección crítica. No podemos simplemente delegar en el sistema operativo la tarea de entender y aplicar esta restricción porque los detalles de los requisitos pueden no ser obvios.



En el caso de una impresora, por ejemplo, queremos que cualquier proceso individual tenga el control de la impresora mientras imprime el fichero completo. De otro modo, las líneas de los procesos en competencia se intercalarían.

La aplicación de la ***exclusión mutua*** crea dos problemas de control adicionales. Uno es el del ***interbloqueo*** (deadlock). Por ejemplo, consideremos dos procesos, P1 y P2, y dos recursos, R1 y R2. Supongamos que cada proceso necesita acceder a ambos recursos para realizar parte de su función. Entonces es posible encontrarse la siguiente situación: el sistema operativo asigna R1 a P2, y R2 a P1. Cada proceso está esperando por uno de los dos recursos. Ninguno liberará el recurso que ya posee hasta haber conseguido el otro recurso y realizado la función que requiere ambos recursos. Los dos procesos están ***interbloqueados***.



Un último problema de control es la ***inanición*** (starvation). Supongamos que tres procesos (P1, P2, P3) requieren todos accesos periódicos al recurso R. Consideremos la situación en la cual P1 está en posesión del recurso y P2 y P3 están ambos retenidos, esperando por ese recurso. Cuando P1 termine su sección crítica, debería permitírsele acceso a R a P2 o P3. Asumiendo que el sistema operativo le concede acceso a P3 y que P1 solicita acceso otra vez antes de completar su sección crítica. Si el sistema operativo le concede acceso a P1 después de que P3 haya terminado, y posteriormente concede alternativamente acceso a P1 y a P3, entonces a P2 puede denegársele indefinidamente el acceso al recurso, aunque no suceda un interbloqueo.

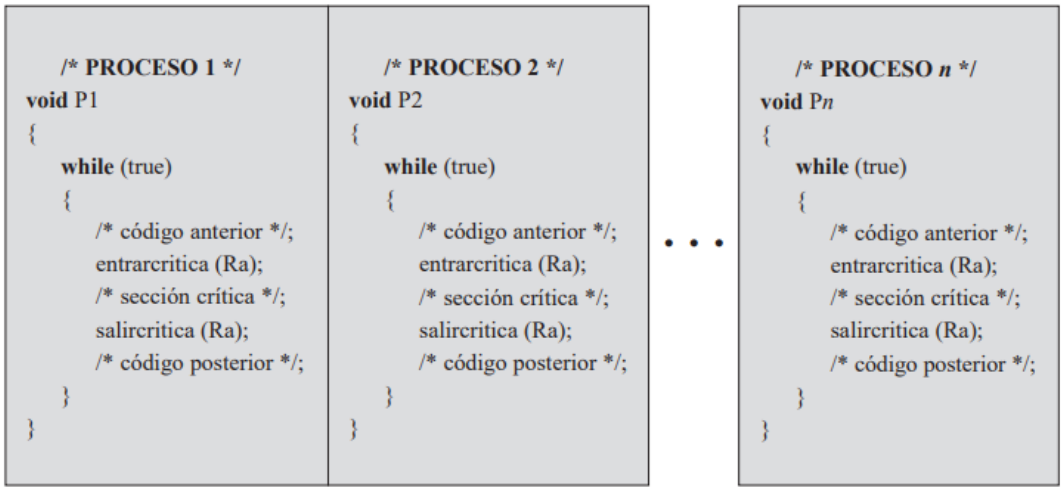
```
Cola de listos (ordenada por prioridad)

[ P1 (alta) ] --> se ejecuta
[ P2 (media) ] --> espera
[ P3 (baja) ] --> espera indefinidamente

Tiempo pasa...

Nuevos procesos de alta prioridad llegan:
[ P1' (alta) ] --> se ejecuta
[ P1'' (alta) ] --> se ejecuta
[ P2 (media) ] --> sigue esperando
[ P3 (baja) ] --> nunca ejecuta ❌ (inanición)
```

El control de la competencia involucra inevitablemente al sistema operativo ya que es quien administra los recursos. Además, los procesos necesitarán ser capaces por sí mismos de expresar de alguna manera el requisito de exclusión mutua, como bloqueando el recurso antes de usarlo. Cualquier solución involucrará algún apoyo del sistema operativo, como proporcionar un servicio de bloqueo.



```
/* PROCESO 1 */
void P1
{
    while (true)
    {
        /* código anterior */;
        entrarcritica (Ra);
        /* sección crítica */;
        salircritica (Ra);
        /* código posterior */;
    }
}
```

```
/* PROCESO 2 */
void P2
{
    while (true)
    {
        /* código anterior */;
        entrarcritica (Ra);
        /* sección crítica */;
        salircritica (Ra);
        /* código posterior */;
    }
}
```

```
/* PROCESO n */
void Pn
{
    while (true)
    {
        /* código anterior */;
        entrarcritica (Ra);
        /* sección crítica */;
        salircritica (Ra);
        /* código posterior */;
    }
}
```

La figura ilustra el mecanismo de exclusión mutua en términos abstractos. Hay n procesos para ser ejecutados concurrentemente. Cada proceso incluye (1) una sección crítica que opera sobre algún recurso Ra, y (2) código adicional que precede y sucede a la sección crítica y que no involucra acceso a Ra.

Dado que todos los procesos acceden al mismo recurso Ra, se desea que sólo un proceso esté en su sección crítica al mismo tiempo. Para aplicar exclusión mutua se proporcionan dos funciones: entrarcritica() y salircritica(). Cada función toma como un argumento el nombre del recurso sujeto de la competencia. A cualquier proceso que intente entrar en su sección crítica mientras otro proceso está en su sección crítica, por el mismo recurso, se le hace esperar.

sección crítica (***critical section***)	Sección de código dentro de un proceso que requiere acceso a recursos compartidos y que no puede ser ejecutada mientras otro proceso esté en una sección de código correspondiente.
interbloqueo (***deadlock***)	Situación en la cual dos o más procesos son incapaces de actuar porque cada uno está esperando que alguno de los otros haga algo.
círculo vicioso (***livelock***)	Situación en la cual dos o más procesos cambian continuamente su estado en respuesta a cambios en los otros procesos, sin realizar ningún trabajo útil.
exclusión mutua (***mutual exclusion***)	Requisito de que cuando un proceso esté en una sección crítica que accede a recursos compartidos, ningún otro proceso pueda estar en una sección crítica que acceda a ninguno de esos recursos compartidos.
condición de carrera (***race condition***)	Situación en la cual múltiples hilos o procesos leen y escriben un dato compartido y el resultado final depende de la coordinación relativa de sus ejecuciones.
inanición (***starvation***)	Situación en la cual un proceso preparado para avanzar es soslayado indefinidamente por el planificador; aunque es capaz de avanzar, nunca se le escoge.

Mas sobre exclusión mutua.

Para evitar situaciones problemáticas en las que se involucran la memoria compartida, los archivos compartidos y todo lo demás compartido, se debe buscar alguna manera de prohibir que más de un proceso lea y escriba en estos datos al mismo tiempo. Se necesita ***exclusión mutua***.

Debemos cumplir con cuatro condiciones para proveer exclusión mutua:

- No puede haber dos procesos de manera simultánea dentro de sus secciones críticas.
- No pueden hacerse suposiciones acerca de las velocidades o el número de CPUs.
- Ningún proceso que se ejecute fuera de su región crítica puede bloquear otros procesos.
- Ningún proceso tiene que esperar para siempre para entrar a su sección crítica.

Condición de carrera.

Una condición de carrera sucede cuando múltiples procesos o hilos leen y escriben datos de manera que el resultado final depende del orden de ejecución de las instrucciones en los múltiples procesos.

Consideremos dos casos sencillos. Como primer ejemplo, supongamos que dos procesos, P1 y P2, comparten la variable global A. En algún punto de su ejecución, P1 actualiza A al valor 1 y, en el mismo punto en su ejecución, P2 actualiza A al valor 2. Así, las dos tareas compiten en una carrera por escribir la variable A.

En este ejemplo el “perdedor” de la carrera (el último proceso que actualiza la variable) determina el valor de A.

Para nuestro segundo ejemplo, considere dos procesos, P3 y P4, que comparten las variables globales b y c, con valores iniciales b = 1 y c = 2. En algún punto de su ejecución, P3 ejecuta la asignación b = b + c y, en algún punto de su ejecución, P4 ejecuta la asignación c = b + c.

Observemos que los dos procesos actualizan diferentes variables. Sin embargo, los valores finales de las dos variables dependen del orden en que los dos procesos ejecuten estas dos asignaciones. Si P3 ejecuta su sentencia de asignación primero, entonces los valores finales serán b = 3 y c = 5. Si P4 ejecuta su sentencia de asignación primero, entonces los valores finales serán b = 4 y c = 3.

Referencias y bibliografía de la sección.

- Stallings pág. 206.

Sección crítica.

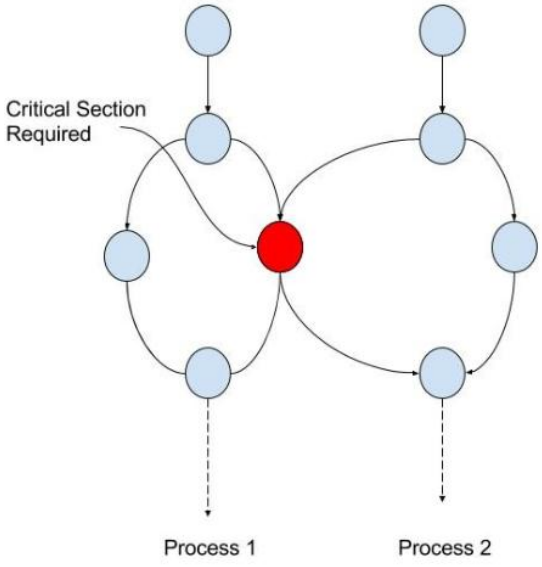
La clave para evitar problemas como las condiciones de carrera, en las que se involucran la memoria compartida, los archivos compartidos, etc., es buscar alguna manera de prohibir que más de un proceso lea y escriba los datos compartidos al mismo tiempo.

Dicho en otras palabras, lo que necesitamos es ***exclusión mutua***, cierta forma de asegurar que, si un proceso está utilizando una variable o archivo compartido, los demás procesos se excluirán de hacer lo mismo.

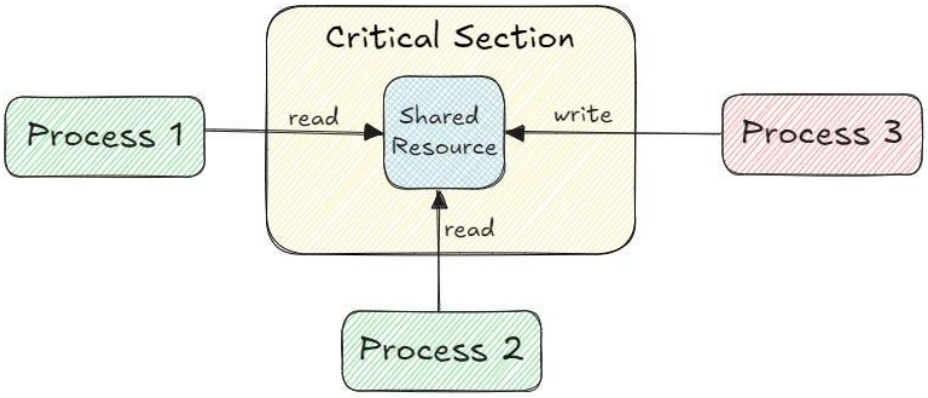
La elección de operaciones primitivas apropiadas para lograr la exclusión mutua es una cuestión de diseño importante en cualquier sistema operativo.

El problema de evitar las condiciones de carrera también se puede formular de una manera abstracta. Parte del tiempo, un proceso está ocupado realizando cálculos internos y otras cosas que no producen condiciones de carrera. Sin embargo, algunas veces un proceso tiene que acceder a la memoria compartida o a archivos compartidos, o hacer otras cosas críticas que pueden producir condiciones de carrera.

Esa parte del programa en la que se accede a la memoria compartida se conoce como ***región crítica*** o ***sección crítica***. Si pudiéramos ordenar las cosas de manera que dos procesos nunca estuvieran en sus regiones críticas al mismo tiempo, podríamos evitar las condiciones de carrera.



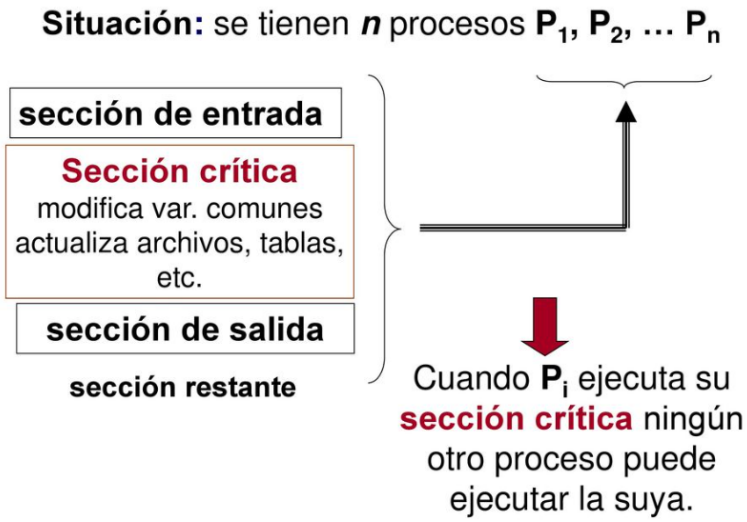
Consideremos un sistema que consta de n procesos. Cada proceso tiene un segmento de código, llamado ***sección crítica***, en el que el proceso puede modificar variables comunes, actualizar una tabla, escribir en un archivo, etc.



Una característica importante de este sistema es que, cuando un proceso está ejecutando su sección crítica, ningún otro proceso puede ejecutar su correspondiente sección crítica. Es decir, dos procesos no pueden ejecutar su sección crítica al mismo tiempo.

El problema de la sección crítica consiste en diseñar un protocolo que los procesos puedan usar para cooperar. Cada proceso debe solicitar permiso para entrar en su sección crítica; la sección de código que implementa esta solicitud es la ***sección de entrada***.

La sección crítica puede ir seguida de una ***sección de salida***. El código restante se encuentra en la ***sección restante***.



La estructura general de un proceso típico es la que se muestra en la figura anterior. Cualquier solución al problema de la sección crítica deberá satisfacer los tres requisitos siguientes:

- **Exclusión mutua.** Si un proceso está ejecutándose en su sección crítica, los demás procesos no pueden estar ejecutando sus secciones críticas.
- **Progreso.** Si ningún proceso está ejecutando su sección crítica y algunos procesos desean entrar en sus correspondientes secciones críticas, sólo aquellos procesos que no estén ejecutando sus secciones restantes pueden participar en la decisión de cuál será el siguiente que entre en su sección crítica, y esta selección no se puede posponer indefinidamente.
- **Espera limitada.** Existe un límite en el número de veces que se permite que otros procesos entren en sus secciones críticas después de que un proceso haya hecho una solicitud para entrar en su sección crítica y antes de que la misma haya sido concedida.

Estamos suponiendo que cada proceso se ejecuta a una velocidad distinta de cero. Sin embargo, no podemos hacer ninguna suposición sobre la velocidad relativa de los procesos. En un instante de tiempo determinado, pueden estar activos muchos procesos en modo kernel en el sistema operativo. Como resultado, el código que implementa el sistema operativo (el código del kernel) está sujeto a varias posibles ***condiciones de carrera***.

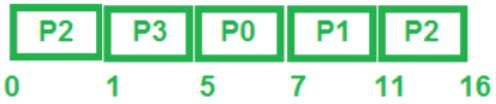
Consideremos por ejemplo una estructura de datos del kernel que mantenga una lista de todos los archivos abiertos en el sistema. Esta lista debe modificarse cuando se abre un nuevo archivo o se cierra un archivo abierto (añadiendo el archivo a la lista o eliminándolo de la misma). Si dos procesos abrieran archivos simultáneamente, las actualizaciones de la lista podrían llevar a una condición de carrera.

Otras estructuras de datos del kernel propensas a posibles condiciones de carrera son aquellas empleadas para controlar la asignación de memoria, las listas de procesos y para gestionar las interrupciones. Es responsabilidad de los desarrolladores del kernel asegurar que el sistema operativo esté libre de tales condiciones de carrera.

Se usan dos métodos generales para gestionar las secciones críticas en los sistemas operativos: los ***kernels apropiativos*** (también llamados expulsivos o preventivos) y los ***kernels no apropiativos*** (también conocidos como cooperativos).

Un kernel apropiativo permite que un proceso sea desalojado mientras se está ejecutando.

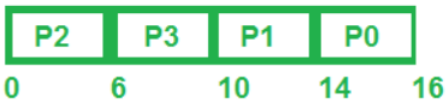
Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Preemptive Scheduling

Un kernel no apropiativo no permite que un proceso que se esté ejecutando sea desalojado; el proceso se ejecutará hasta que se bloquee o hasta que ceda voluntariamente el control de la CPU.

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Non-Preemtive Scheduling

Obviamente, un kernel no apropiativo está esencialmente libre de condiciones de carrera en lo que respecta a las estructuras de datos del kernel, ya que sólo hay un proceso activo en el kernel en cada momento. No podemos decir lo mismo acerca de los kernels apropiativos, por lo que deben ser diseñados cuidadosamente para asegurar que los datos compartidos del kernel no se vean afectados por posibles condiciones de carrera.

Los kernel apropiativos son especialmente difíciles de diseñar en arquitecturas SMP[15], dado que en estos entornos es posible que dos procesos se ejecuten simultáneamente en procesadores diferentes. De todas formas, un kernel apropiativo podría ser conveniente en este tipo de arquitectura porque que es más adecuado para la programación en tiempo real, ya que permite a un proceso en tiempo real desalojar a un proceso que se esté ejecutando en ese momento.

Además, un kernel apropiativo puede tener una mejor capacidad de respuesta, ya que existe menos riesgo de que un proceso se ejecute durante un período de tiempo arbitrariamente largo antes de ceder el procesador a los procesos que estén a la espera.

Por supuesto, este efecto puede minimizarse diseñando código de kernel que no presente este tipo de comportamiento. Windows XP y Windows 2000 son kernels no apropiativos, al igual que el kernel tradicional de UNIX.

Antes de Linux 2.6, el kernel de Linux también era no apropiativo. Sin embargo, con la versión del kernel 2.6, Linux cambió al modelo apropiativo. Varias versiones comerciales de UNIX usan un kernel apropiativo, incluyendo Solaris e IRIX[16].

Referencias y bibliografía de la sección.

- Silberschatz 6-2.
- Luis Castellanos – Sistemas Operativos 3-2.

Intentos de solución al problema de la sección crítica.

Deshabilitar interrupciones.

El problema de la sección crítica podría resolverse de forma simple en un entorno de un solo procesador si pudiéramos impedir que se produjeran interrupciones mientras se está modificando una variable compartida (sección critica).

De este modo, podríamos asegurar que la secuencia actual de instrucciones se ejecute por orden. Ninguna otra instrucción se ejecutará, por lo que no se producirán modificaciones inesperadas de la variable compartida. Éste es el método que emplean los ***kernels no apropiativos***.

Con las interrupciones deshabilitadas, no pueden ocurrir interrupciones de reloj. Después de todo, la CPU sólo se conmuta de un proceso a otro como resultado de una interrupción del reloj o de otro tipo, y con las interrupciones desactivadas la CPU no se conmutará a otro

[15] La arquitectura SMP (multiprocesamiento simétrico) es una arquitectura de computadoras que conecta dos o más procesadores a una memoria centralizada. Se utiliza en entornos informáticos que requieren alto rendimiento, como servidores. Los procesadores son idénticos y comparten la misma responsabilidad en la ejecución de las tareas.
[16] IRIX es un sistema operativo compatible con Unix, creado por Silicon Graphics para su plataforma MIPS de 64 bits.

proceso. Por ende, una vez que un proceso ha deshabilitado las interrupciones, puede examinar y actualizar la memoria compartida sin temor de que algún otro proceso intervenga.

Vale aclarar que mientras el proceso está haciendo cálculos internos, leyendo datos que no comparte con nadie o ejecutando lógica propia (sección restante), las interrupciones están ***habilitadas***. El sistema operativo puede quitarle la CPU en cualquier momento para dársela a otro. Esto permite que el sistema sea fluido.

Sección de Entrada	**Deshabilitar Interrupciones**
Sección Crítica	**Ejecución Atómica**
Sección de Salida	**Habilitar Interrupciones**

Sin embargo, por lo general el método de desactivar interrupciones es poco atractivo, ya que no es conveniente dar a los procesos de usuario el poder para desactivar las interrupciones. Supongamos que uno de ellos lo hiciera y nunca las volviera a activar, ese podría ser el fin del sistema.

Además, esta solución no resulta tan adecuada en un entorno multiprocesador. Desactivar las interrupciones en un sistema multiprocesador puede consumir mucho tiempo, ya que hay que pasar el mensaje a todos los procesadores. Este paso de mensajes retarda la entrada en cada sección crítica y la eficiencia del sistema disminuye.

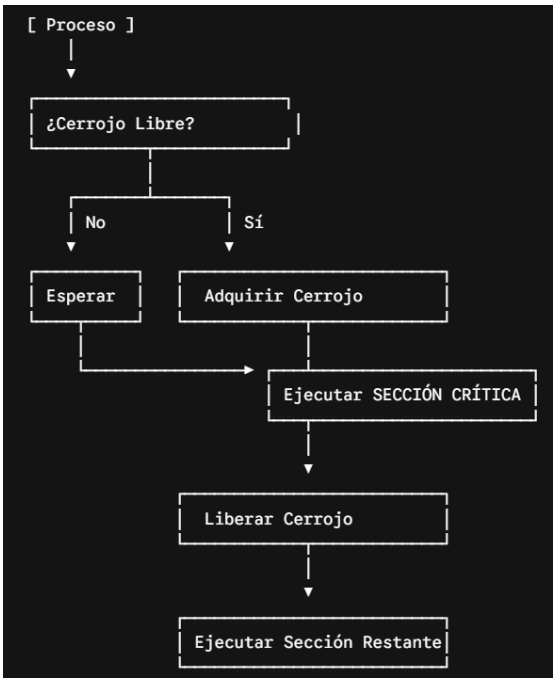
La posibilidad de lograr la exclusión mutua al deshabilitar las interrupciones ha ido disminuyendo día con día debido al creciente número de chips multinúcleo que se encuentran hasta en las PCs de bajo rendimiento.

En un multinúcleo (es decir, sistema con multiprocesadores) al deshabilitar las interrupciones de una CPU no se evita que las demás CPUs interfieran con las operaciones que la primera CPU está realizando. En consecuencia, se requieren esquemas más sofisticados.

Cerrojos y hardware de sincronización.

En general, podemos afirmar que cualquier solución al problema de la sección crítica requiere una herramienta muy simple, un ***cerrojo***.

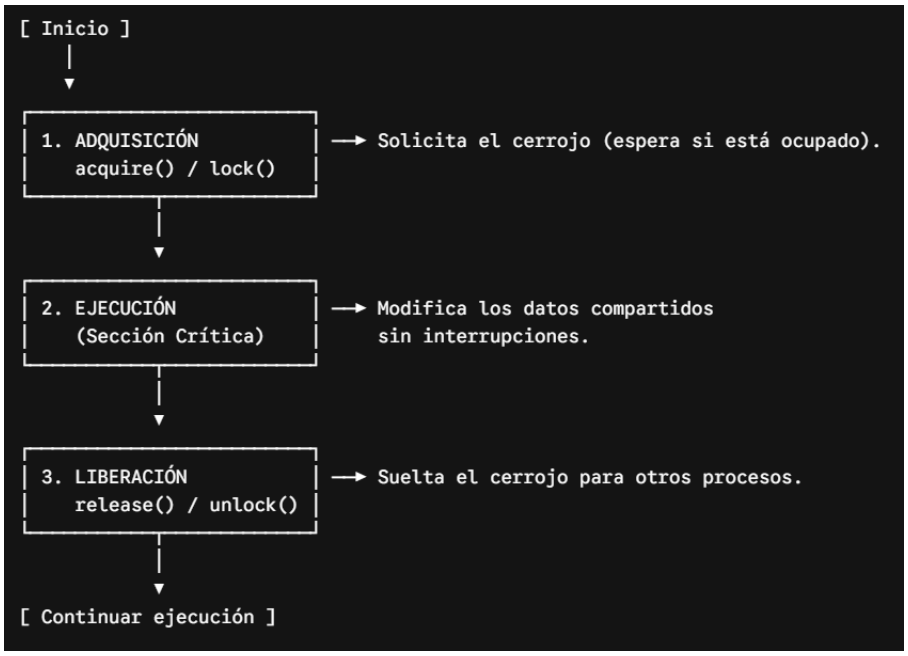
Un ***cerrojo (lock)*** es una herramienta de sincronización provista por software y que puede ser respaldada por hardware que se utiliza en sistemas operativos para resolver el problema de la sección crítica y evitar condiciones de carrera en entornos concurrentes.



Funciona como una variable lógica (booleana) con dos estados (libre u ocupado).

- **Adquisición (acquire / lock):** Antes de ingresar a la sección crítica, el proceso solicita el cerrojo. Si está libre, lo toma y cambia su estado a ocupado; si está ocupado, el proceso debe esperar.
- **Ejecución:** El proceso ejecuta las instrucciones de la sección crítica sabiendo que ningún otro proceso puede intervenir.
- **Liberación (release / unlock):** Al finalizar la sección crítica, el proceso libera el cerrojo, dejándolo disponible para que otro proceso pueda adquirirlo.

Las ***condiciones de carrera*** se evitan requiriendo que las regiones críticas se protejan mediante cerrojos. Es decir, un proceso debe adquirir un cerrojo antes de entrar en sección crítica y liberarlo cuando salga de la misma.



El ***soporte de hardware*** puede facilitar cualquier tarea de programación y mejorar la eficiencia del sistema.

Muchos sistemas informáticos modernos proporcionan ***instrucciones de hardware*** especiales que nos permiten consultar y modificar el contenido de una palabra o intercambiar los contenidos de dos palabras atómicamente, es decir, como una unidad de trabajo ininterrumpible.

Podemos usar estas instrucciones de la CPU especiales para resolver el problema de la sección crítica de una forma relativamente simple, permitiendo que los procesos compartan recursos sin interferencias. Se conocen como ***primitivas de sincronización de hardware***.

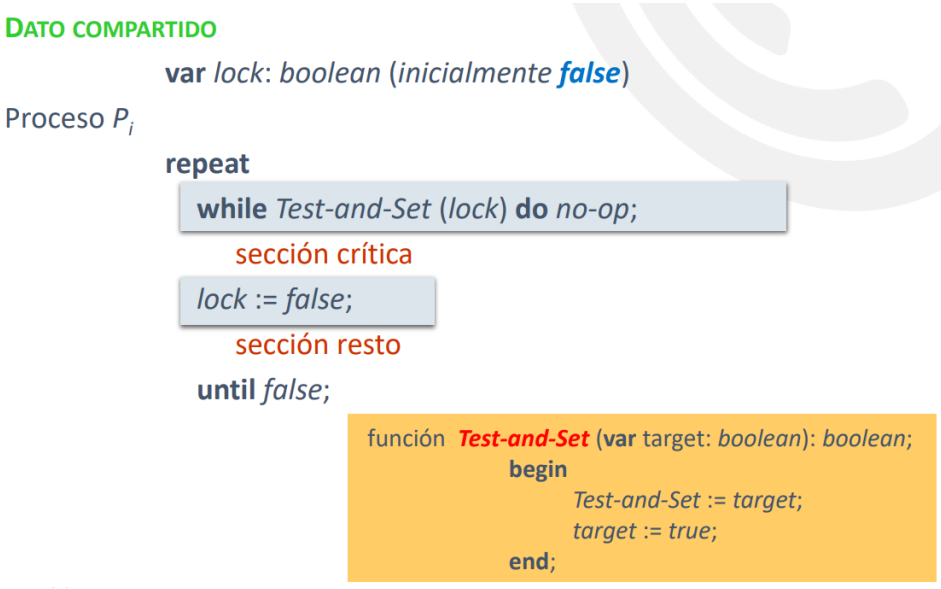
Estas primitivas constituyen la base física más elemental sobre la cual el sistema operativo construye herramientas de sincronización de mayor nivel, como cerrojos y semáforos.

La instrucción ***TestAndSet*** sirve para leer y modificar ***atómicamente***[17] una variable.

Un procesador realizando una instrucción ***TestAndSet*** sobre una dirección de memoria impide que cualquier otro procesador pueda realizarla cerrando el bus de memoria.

La característica importante es que esta instrucción se ejecuta atómicamente (no interrumpible). Por lo tanto, si dos instrucciones TestAndSet se ejecutan simultáneamente (cada una en una CPU diferente), se ejecutarán secuencialmente en un orden arbitrario. Si la máquina soporta la instrucción TestAndSet, entonces podemos implementar la exclusión mutua declarando una variable booleana ***lock*** inicializada con el valor false.

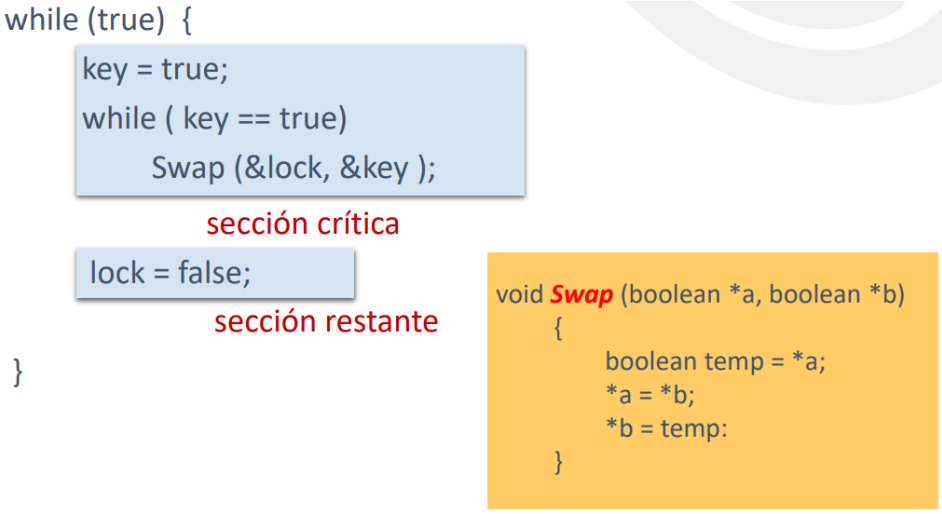
[17] Una instrucción atómica en un sistema operativo es un conjunto de instrucciones que se ejecutan como una unidad indivisible. Esto significa que la operación se ejecuta por completo o no se realiza en absoluto.



La instrucción ***Swap*** para intercambiar el valor de dos variables, a diferencia de la instrucción TestAndSet, opera sobre los contenidos de dos palabras.

Como la instrucción TestAndSet, se ejecuta atómicamente. Si la máquina soporta la instrucción Swap, la exclusión mutua se proporciona así: se declara una variable global booleana ***lock*** y se inicializa como false; además, cada proceso tiene una variable local booleana ***key***.

Aunque estos algoritmos satisfacen el requisito de exclusión mutua, no satisfacen el requisito de espera limitada.



Solución de Peterson.

Una solución clásica basada en software al problema de la sección crítica es la solución de ***Peterson***.

Debido a la forma en que las arquitecturas informáticas modernas ejecutan las instrucciones básicas en lenguaje máquina, como load y store, no hay garantías de que la solución de Peterson funcione correctamente en tales arquitecturas. Sin embargo, esta solución proporciona una buena descripción ***algorítmica*** de la resolución del problema de la sección crítica e ilustra algunas de las complejidades asociadas al diseño de software que satisfaga los requisitos de exclusión mutua, progreso y tiempo de espera limitado.

Este algoritmo demuestra que es posible lograr la exclusión mutua mediante software puro, sin necesidad de deshabilitar interrupciones ni usar instrucciones especiales de hardware.



La solución de Peterson se restringe a dos procesos que van alternando la ejecución de sus secciones críticas y de sus secciones restantes. Los procesos se numeran como P0 y P1.

La solución de Peterson requiere que los dos procesos compartan dos elementos de datos:

```
int turn;
boolean flag[2];
```

La variable turn indica qué proceso va a entrar en su sección crítica. Es decir, si turn == i, entonces el proceso Pi puede ejecutar su sección crítica.

La matriz flag se usa para indicar si un proceso está preparado para entrar en su sección crítica. Por ejemplo, si flag[i] es verdadera, este valor indica que Pi está preparado para entrar en su sección crítica.

```
do {
    // 1. SECCIÓN DE ENTRADA
    flag[i] = true; // Se declara que el Proceso i quiere entrar en su sección critica
    turn = j; // Se le cede el turno al Proceso j

    // Espera activa/ocupada: mientras el otro proceso quiera entrar y sea su turno, nos
quedamos esperando
    while (flag[j] && turn == j);

    // 2. SECCIÓN CRÍTICA
    // Acceso a los datos compartidos de forma segura

    // 3. SECCIÓN DE SALIDA
    flag[i] = false; // Se indica que el Proceso i ya salió de la sección crítica

    // 4. SECCIÓN RESTANTE
    // Resto del código independiente

} while (true);
```

Para entrar en la sección crítica, el proceso Pi primero asigna el valor verdadero a flag[i] y luego asigna a turn el valor j, confirmando así que, si el otro proceso desea entrar en la sección crítica, puede hacerlo.

Si ambos procesos intentan entrar al mismo tiempo, a la variable turn se le asignarán los valores tanto i como j aproximadamente al mismo tiempo. Sólo una de estas asignaciones permanecerá; la otra tendrá lugar, pero será sobreescrita inmediatamente.

El valor que adopte turn decidirá cuál de los dos procesos podrá entrar primero en su sección crítica.

Componente	Definición Técnica	Función en el Control
`flag[i]`	Variable booleana	Indica si el proceso P_i está listo para entrar a la Sección Crítica.
`turn`	Variable entera	Determina cuál de los dos procesos tiene la prioridad de paso.
Entrada	Protocolo de acceso	P_i asigna `flag[i] = true` y cede el turno: `turn = j`.
Espera	Bucle `while`	P_i se bloquea si P_j tiene su `flag` activo **y** posee el `turn`.
Salida	Protocolo de liberación	P_i asigna `flag[i] = false`, permitiendo el progreso de P_j.

Una de las desventajas de la solución de Peterson es que está limitada a solo dos procesos. Aunque existen extensiones teóricas para N procesos, estas incrementan drásticamente la complejidad y degradan el rendimiento.

Otra desventaja a tener en cuenta es el uso de la espera activa. Mientras un proceso aguarda para entrar a la sección crítica, permanece atrapado en un bucle while consumiendo ciclos de CPU continuamente.

Semáforos.

Las diversas soluciones hardware al problema de la sección crítica, basadas en las instrucciones TestAndSet () y Swap () son complicadas de utilizar por los programadores de aplicaciones. Para superar esta dificultad, podemos usar una herramienta de sincronización denominada ***semáforo***.

Un semáforo es una variable entera (S) a la que, dejando aparte la inicialización, sólo se accede mediante dos operaciones atómicas estándar: ***wait*** () y ***signal*** ().

```
wait (S)
    while S≤ 0 do no-op;
    S := S – 1;

signal (S)
    S := S + 1;
```

Todas las modificaciones del valor entero del semáforo en las operaciones wait () y signal () deben ejecutarse de forma indivisible. Es decir, cuando un proceso modifica el valor del semáforo, ningún otro proceso puede modificar simultáneamente el valor de dicho semáforo. Además, en el caso de wait (), la prueba del valor entero de S (S =<0), y su posible modificación (S --) también se deben ejecutar sin interrupción.

```
struct semaphore {
    int cuenta;
    queueType cola;
}
void semWait(semaphore s)
{
    s.cuenta—;
    if (s.cuenta < 0)
    {
        poner este proceso en s.cola;
        bloquear este proceso;
    }
}
void semSignal(semaphore s)
{
    s.cuenta++;
    if (s.cuenta <= 0)
    {
        extraer un proceso P de s.cola;
        poner el proceso P en la lista de listos;
    }
}
```

Ilustración 15 - Una definición de las primitivas del semáforo.

```
struct binary_semaphore {
    enum {cero, uno} valor;
    queueType cola;
};
void semWaitB(binary_semaphore s)
{
    if (s.valor == 1)
        s.valor = 0;
    else
        {
            poner este proceso en s.cola;
            bloquear este proceso;
    }
}
void semSignalB(binary_semaphore s)
{
    if (esta_vacia(s.cola))
        s.valor = 1;
    else
    {
            extraer un proceso P de s.cola;
            poner el proceso P en la lista de listos;
    }
}
```

Ilustración 16 - Una definición de las primitivas del semáforo binario.

Utilización.

Los sistemas operativos diferencian a menudo entre semáforos contadores y semáforos binarios.

El valor de un semáforo contador puede variar en un dominio no restringido, mientras que el valor de un semáforo binario sólo puede ser 0 o 1.

En algunos sistemas, además, los semáforos binarios y los cerrojos mutex se identifican como una misma herramienta, ya que actúan como cerrojos que proporcionan exclusión mutua. Sin embargo, vale aclarar que también se relacionan a los mutex como un mecanismo de exclusión mutua ligado a un dueño (el proceso que bloquea el mutex debe ser el mismo que lo libera).

Característica	Semáforo Contador	Semáforo Binario	Cerrojo Mutex
Rango Valor	0 a N	0 o 1	0 o 1
Uso Base	Recursos múltiples	Exclusión mutua	Exclusión mutua
Dueño	No tiene	No tiene	**Sí (Obligatorio)**
Liberación	Cualquier proceso	Cualquier proceso	**Solo el dueño**
Contexto	Gestión de cupos	Sincronización	Sección Crítica

Podemos usar semáforos binarios para abordar el problema de la sección crítica en el caso de múltiples procesos. Los n procesos comparten un semáforo binario, inicializado con el valor 1.

```
Semáforo S;   //  inicializado en 1
wait (S);
        Sección Crítica
 signal (S);
```

Los semáforos contadores se pueden usar para controlar el acceso a un determinado recurso formado por un número finito de instancias. El semáforo se inicializa con el número de recursos disponibles. Cada proceso que desee usar un recurso ejecuta una operación wait () en el semáforo (decrementando la cuenta).

Cuando un proceso libera un recurso, ejecuta una operación signal () (incrementando la cuenta). Cuando la cuenta del semáforo llega a 0, todos los recursos estarán en uso. Después, los procesos que deseen usar un recurso se bloquearán hasta que la cuenta sea mayor que 0.

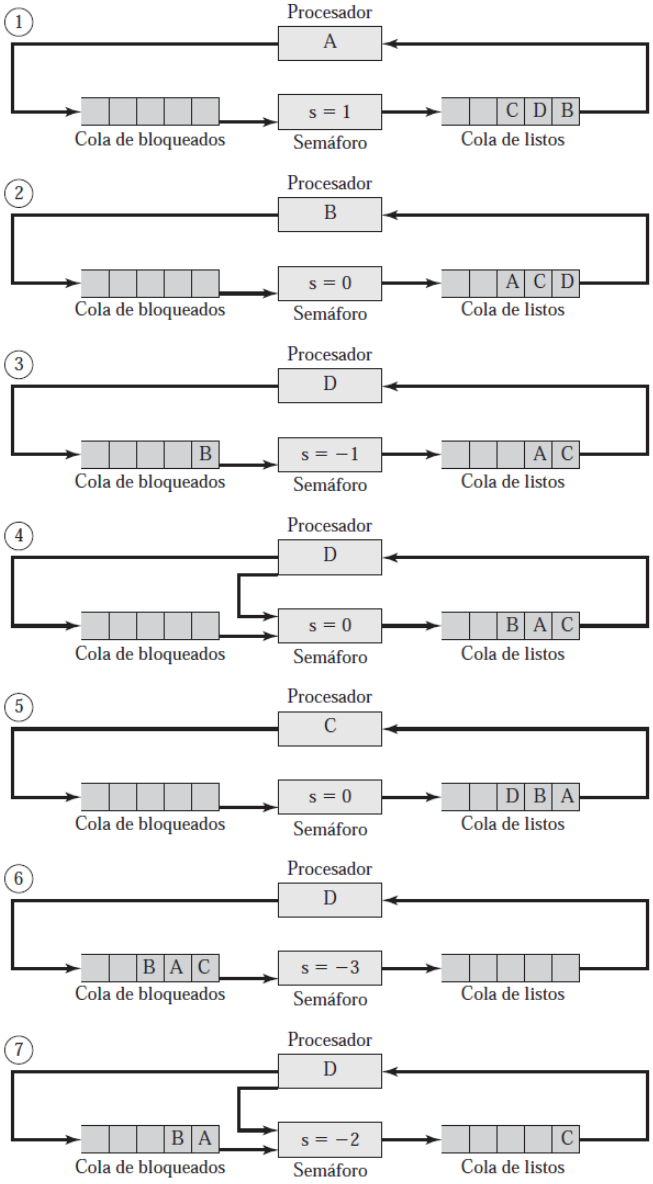


Ilustración 17 - Ejemplo de mecanismo semáforo.

Implementación

La principal desventaja de la definición de semáforo dada aquí es que requiere una espera activa. Mientras un proceso está en su sección crítica, cualquier otro proceso que intente entrar en su sección crítica debe ejecutar continuamente un bucle en el código de entrada. Este bucle continuo plantea claramente un problema en un sistema real de multiprogramación, donde una sola CPU se comparte entre muchos procesos.

La espera activa desperdicia ciclos de CPU que algunos otros procesos podrían usar de forma productiva. Este tipo de semáforo también se denomina ***cerrojo mediante bucle sin fin*** (spinlock), ya que el proceso “permanece en un bucle sin fin” en espera de adquirir el cerrojo.

Cuando se espera que los cerrojos se mantengan durante un período de tiempo corto, los cerrojos mediante bucle sin fin pueden resultar útiles; por eso se emplean a menudo en los sistemas multiprocesador, donde un hilo puede “ejecutar un bucle” sobre un procesador mientras otro hilo ejecuta su sección crítica en otro procesador.

Para salvar la necesidad de la espera activa, podemos modificar la definición de las operaciones de semáforo wait () y signal (). Cuando un proceso ejecuta la operación wait () y determina que el valor del semáforo no es positivo, tiene que esperar. Sin embargo, en lugar de entrar en una espera activa, el proceso puede bloquearse a sí mismo. La operación de bloqueo coloca al proceso en una cola de espera asociada con el semáforo y el estado del proceso pasa al estado de espera. A continuación, el control se transfiere al planificador de la CPU, que selecciona otro proceso para su ejecución.

Un proceso bloqueado, que está esperando en un semáforo S, debe reiniciarse cuando algún otro proceso ejecuta una operación signal (). El proceso se reinicia mediante una operación wakeup (), que cambia al proceso del estado de espera al estado de preparado. El proceso se coloca en la cola de procesos preparados.

```
wait(S):
        S.value := S.value – 1;
        if S.value < 0
          then begin
                 agregue este proceso a S.L;
                 block;
          end;
signal(S):
        S.value := S.value + 1;
        if S.value ≤ 0
          then begin
                 remueva un proceso P de S.L;
                 wakeup(P);
           end;
```

Monitores.

Un ***monitor*** es un mecanismo de sincronización diseñado para resolver problemas de concurrencia. Fue introducido como una forma más segura y estructurada que los semáforos para coordinar procesos o hilos.

Los semáforos proporcionan una herramienta potente y flexible para conseguir la exclusión mutua y para la coordinación de procesos. Sin embargo, puede ser difícil producir un programa correcto utilizando semáforos.

La dificultad es que las operaciones semWait (wait) y semSignal (signal) pueden estar dispersas a través de un programa y no resulta fácil ver el efecto global de estas operaciones sobre los semáforos a los que afectan.

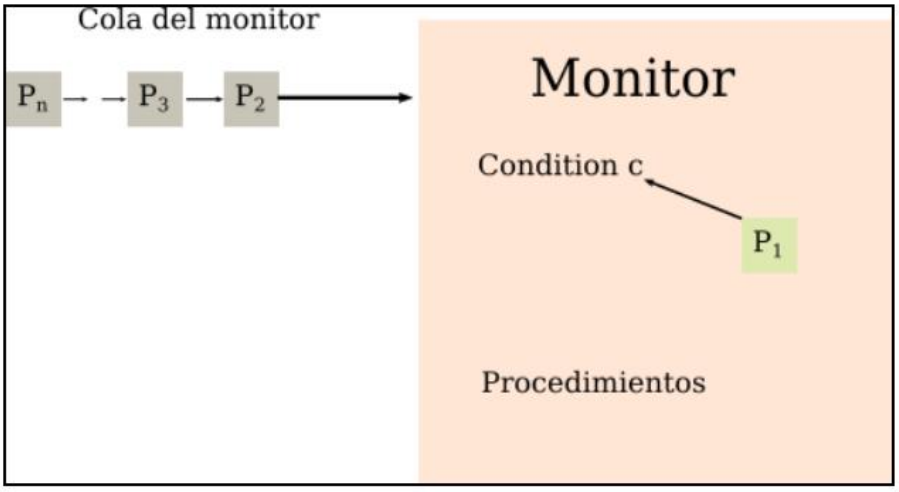
El ***monitor*** es una construcción del lenguaje de programación que proporciona una funcionalidad equivalente a la de los semáforos, pero es más fácil de controlar.

La construcción monitor ha sido implementada en cierto número de lenguajes de programación, incluyendo Pascal Concurrente, Pascal-Plus, Modula-2, Modula-3 y Java.

La gran ventaja es que la exclusión mutua es ***automática***: el compilador o el tiempo de ejecución del lenguaje garantiza que solo un proceso esté "dentro" del monitor en un momento dado.

- **Error con Semáforos:** Si un programador olvida un `signal(S)` o intercambia el orden por error (`signal` antes de `wait`), el sistema puede corromperse o bloquearse (Deadlock) sin previo aviso, ya que el control está esparcido por todo el código.
- **Seguridad del Monitor:** Al estar todo el código de sincronización centralizado en un solo lugar (dentro del monitor), es mucho más sencillo verificar la corrección del algoritmo.

También ha sido implementada como una biblioteca de programa. Esto permite a los programadores poner ***cerrojos monitor*** sobre cualquier objeto. En concreto, para algo como una lista encadenada, puede quererse tener un único cerrojo para todas las listas, por cada lista o por cada elemento de cada lista.



Los monitores tienen una importante propiedad que los hace útiles para lograr la exclusión mutua: sólo puede haber un proceso activo en un monitor en cualquier instante.

Por lo general, cuando un proceso llama a un procedimiento de monitor, las primeras instrucciones del procedimiento comprobarán si hay algún otro proceso activo en un momento dado dentro del monitor. De ser así, el proceso invocador se suspenderá hasta que el otro proceso haya dejado el monitor. Si no hay otro proceso utilizando el monitor, el proceso invocador puede entrar.

La persona que escribe el monitor no tiene que saber acerca de cómo el compilador hace los arreglos para la exclusión mutua. Basta con saber que, al convertir todas las regiones críticas en procedimientos de monitor, nunca habrá dos procesos que ejecuten sus regiones críticas al mismo tiempo.

Como las estructuras de datos solo se pueden acceder desde el propio monitor, y solo un proceso puede estar en un monitor en cada momento dado, nunca puede un proceso estar esperando algún recurso teniendo otro tomado.

```
monitor modelo {
    int i;          //variable local del monitor
    condition c;    //variable de condición

    procedimiento P1() { ...
    }

    procedimiento P2() { ...
    }

    procedimiento P3() { ...
    }

    //inicialización de variables
}
```

Semáforos:

- Recursos globales.
- Código disperso por los procesos.
- Las operaciones sobre los recursos no están restringidas.

Monitores:

- Recursos locales.
- Código concentrado.
- Los procesos invocan los procedimientos públicos.

Cuando los procesos desean comunicarse o sincronizarse utilizan los recursos privados del Monitor mediante invocación de los procedimientos públicos (exportados).

- **Monitor libre:** Si un proceso invoca un procedimiento de un monitor y nadie posee el monitor, este proceso bloquea y ocupa el monitor (ejecuta el procedimiento).
- **Monitor ocupado:** Si un proceso invoca algún procedimiento del monitor y éste está ocupado entonces el proceso queda bloqueado en una cola asociada al monitor. Cuando el proceso poseedor del monitor finaliza la ejecución de un procedimiento del monitor, se libera el primer proceso bloqueado en ella.

```
monitor EjemploMonitor {

    // Variables compartidas (estado interno)
    int recursoCompartido;
    condition c;   // variable de condición

    // Procedimiento 1
    procedure operacion1() {
        if (condicion_no_cumplida) {
            wait(c);   // el hilo se bloquea aquí
        }
        // ... trabajar con recursoCompartido ...
    }

    // Procedimiento 2
    procedure operacion2() {
        // ... modificar recursoCompartido ...
        signal(c);    // despierta a un hilo esperando en 'c'
    }
}
```

Las principales características de un monitor son las siguientes:

1. Las variables locales de datos son sólo accesibles por los procedimientos del monitor y no por ningún procedimiento externo.
2. Un proceso entra en el monitor invocando uno de sus procedimientos.
3. Sólo un proceso puede estar ejecutando dentro del monitor al tiempo; cualquier otro proceso que haya invocado al monitor se bloquea, en espera de que el monitor quede disponible.

Las dos primeras características guardan semejanza con las de los objetos en el software orientado a objetos. De hecho, en un sistema operativo o lenguaje de programación orientado a objetos puede implementarse inmediatamente un monitor como un objeto con características especiales.

Al cumplir la disciplina de sólo un proceso al mismo tiempo, el monitor es capaz de proporcionar ***exclusión mutua*** fácilmente. Las variables de datos en el monitor sólo pueden ser accedidas por un proceso a la vez. Así, una estructura de datos compartida puede ser protegida colocándola dentro de un monitor. Si los datos en el monitor representan cierto

recurso, entonces el monitor proporciona la función de exclusión mutua en el acceso al recurso.

Para ser útil para la programación concurrente, el monitor debe incluir herramientas de sincronización.

Por ejemplo, suponga un proceso que invoca a un monitor y mientras está en él, deba bloquearse hasta que se satisfaga cierta condición. Se precisa una funcionalidad mediante la cual el proceso no sólo se bloquee, sino que libere el monitor para que algún otro proceso pueda entrar en él. Más tarde, cuando la condición se haya satisfecho y el monitor esté disponible nuevamente, el proceso debe poder ser retomado y permitida su entrada en el monitor en el mismo punto en que se suspendió.

Un monitor soporta la sincronización mediante el uso de ***variables condición*** que están contenidas dentro del monitor y son accesibles sólo desde el monitor. Las variables condición son un tipo de datos especial en los monitores que se manipula mediante dos funciones:

- **cwait(c):** Suspende la ejecución del proceso llamante en la condición *c*. El monitor queda disponible para ser usado por otro proceso.
- **csignal(c):** Retoma la ejecución de algún proceso bloqueado por un ***cwait*** en la misma condición. Si hay varios procesos, elige uno de ellos; si no hay ninguno, no hace nada.

Nótese que las operaciones ***wait*** y ***signal*** de los monitores son diferentes de las de los semáforos.

Si un proceso en un monitor ***señala*** y no hay ningún proceso esperando en la variable condición, la señal se pierde.

```
monitor ProducerConsumer
     condition full, empty;
     integer count;

     procedure insert(item: integer);
     begin
          if count = N then wait(full);
          insert item(item);
          count := count + 1;
          if count = 1 then signal(empty)
     end;

     function remove: integer;
     begin
          if count = 0 then wait(empty);
          remove = remove item;
          count := count - 1;
          if count = N - 1 then signal(full)
     end;

     count := 0;

end monitor;
```

Aunque un proceso puede entrar en el monitor invocando cualquiera de sus procedimientos, puede entenderse que el monitor tiene un único punto de entrada que es el protegido, de ahí que sólo un proceso pueda estar en el monitor a la vez. Otros procesos que intenten entrar en el monitor se unirán a una cola de procesos bloqueados esperando por la disponibilidad del monitor. Una vez que un proceso está en el monitor, puede temporalmente bloquearse a sí mismo; en tal caso, el proceso será añadido a una cola de procesos esperando a reentrar en el monitor cuando cambie la condición y retomar la ejecución.

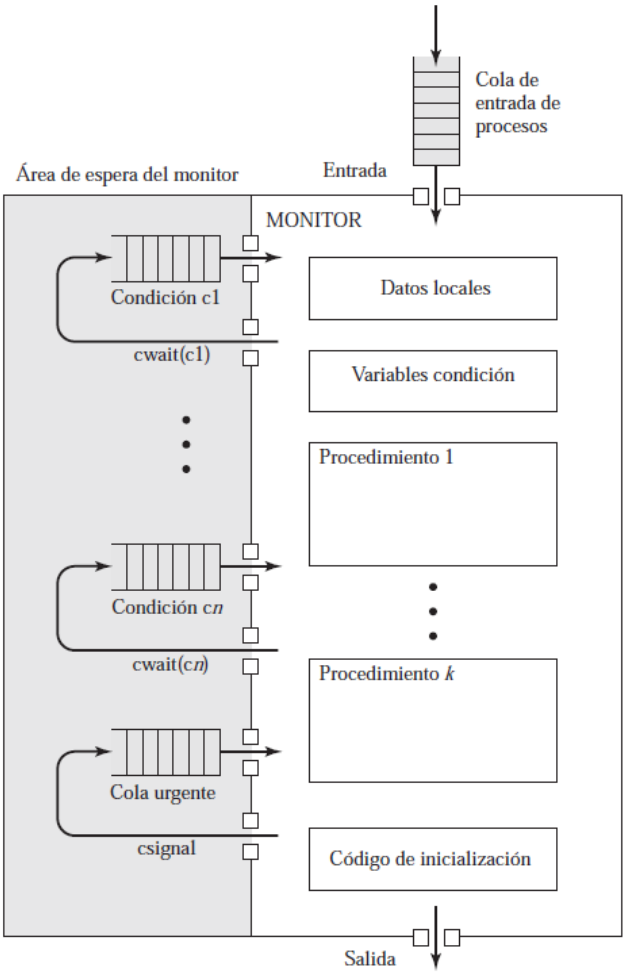


Ilustración 18 - Estructura de un monitor.

Desventajas del modelo de monitor.

La definición de monitor anterior (monitor de Hoare o monitor con señal) requiere que, si hay al menos un proceso en una cola de una condición, un proceso de dicha cola ejecuta inmediatamente cuando otro proceso realice un ***csignal*** sobre dicha condición. Así, el proceso que realiza el ***csignal*** debe salir inmediatamente del monitor o bloquearse dentro del monitor.

Hay dos desventajas en esta solución:

1. Si el proceso que realiza el ***csignal*** no ha terminado con el monitor, entonces se necesitarán dos cambios de proceso adicionales: uno para bloquear este proceso y otro para retomarlo cuando el monitor quede disponible.

2. La planificación de procesos asociada con una señal debe ser perfectamente fiable. Cuando se realiza un ***csignal***, un proceso de la cola de la correspondiente condición debe ser activado inmediatamente y el planificador debe asegurar que ningún otro proceso entra en el monitor antes de la activación. De otro modo, la condición bajo la cual el proceso fue activado podría cambiar (ejemplo: un proceso productor puede añadir un carácter a un *buffer* vacío y entonces fallar justo antes de la señalización; los procesos de la cola no-vacio estarían permanentemente colgados).

Espera activa e inactiva.

El término ***espera activa*** (*busy waiting*), o ***espera cíclica*** (*spin waiting*) se refiere a una técnica en la cual un proceso no puede hacer nada hasta obtener permiso para entrar en su sección crítica, pero continúa ejecutando una instrucción o conjunto de instrucciones que comprueban la variable apropiada para conseguir entrar. Cuando un proceso abandona su sección crítica, restablece la variable de cerrojo a 0; en este punto, a uno y sólo a uno de los procesos en espera se le concederá acceso a su sección crítica.

La elección del proceso depende de cuál de los procesos es el siguiente que ejecuta la instrucción ***TestAndSet***.

Puede ser una estrategia válida en algunas circunstancias especiales, sobre todo en la sincronización de procesos en los sistemas con múltiples procesadores. Aunque en general, debe ser evitada, ya que consume tiempo de CPU sin realizar ninguna operación.

Ventajas

- Es simple y rápido de entender e implementar.
- Es útil para implementar spinlock[18].

Desventajas

- Es ineficiente porque el bucle consume recursos de la computadora.
- Inactividad mientras espera.

```
while (!C) {
  // no hacer nada
}
// ¡Aquí se cumple C!
```

[18] Un spinlock es un mecanismo de sincronización que hace que un hilo o proceso espere en un ciclo mientras comprueba si un bloqueo está disponible. Se utiliza en software para controlar el acceso a recursos compartidos.

Esta técnica no solo es útil en el ámbito de los semáforos, también podría utilizarse para, por ejemplo, esperar una entrada de teclado o generar una demora arbitraria.

Alternativas

- Se pueden utilizar semáforos para sincronizar procesos sin usar busy waiting.
- Se pueden modificar las definiciones de las operaciones de wait y signal del semáforo.

Por su parte, la ***espera inactiva*** se refiere a una implementación en la que, en lugar de desperdiciar tiempo de CPU en espera activa, el proceso se bloquea y queda en espera hasta que el valor de, por ejemplo, un semáforo sea positivo. Esto se logra mediante funciones proporcionadas por el sistema operativo.

Característica	Espera Activa (Spinlock)	Espera Inactiva (Bloqueo)
Consumo CPU	Máximo (bucle infinito).	Mínimo (proceso suspendido).
Context Switch	No existe.	Ocurre al bloquear y despertar.
Uso Ideal	Secciones críticas muy cortas.	Secciones críticas largas o E/S.

Hay varias implementaciones de semáforos con espera inactiva, como la que comprueba si la lista de procesos en espera está vacía o la que utiliza el valor negativo del semáforo para llevar cuenta de los procesos en espera.

El planificador de Linux, por ejemplo, utiliza la espera inactiva como norma general para gestionar la eficiencia del sistema. En un sistema operativo moderno, mantener un proceso en espera activa (quemando ciclos de CPU) es inaceptable a menos que sea estrictamente necesario.

```
Ejecución ——(espera un evento)——> Bloqueado ——(evento ocurre)——> Listo ——> Ejecución
```

Referencias y bibliografía de la sección.

- Stallings pág. 213 / 5-3 pág. 215 / 5-4 pág. 229 / 5-5 pág. 235.
- Tanenbaum 2-3-7.
- Silberschatz 6-3 / 6-4.

Otros problemas de comunicación y sincronización entre procesos.

Existe una gran cantidad de desafición e inconvenientes a la hora de encarar la sincronización de procesos. Vamos a repasar algunos de los problemas más conocidos.

El problema de los filósofos comensales.

En 1965, Dijkstra[19] propuso y resolvió un problema de sincronización al que llamó el problema de los ***filósofos comensales***. Sirve para ilustrar los desafíos de la ***concurrencia***, la ***exclusión mutua***, la ***inanición*** y el ***interbloqueo***.

Este problema se puede enunciar simplemente de la siguiente manera. Cinco filósofos están sentados alrededor de una mesa circular. Cada filósofo tiene un plato de espagueti. El espagueti es tan resbaloso, que un filósofo necesita dos tenedores para comerlo. Entre cada par de platos hay un tenedor.

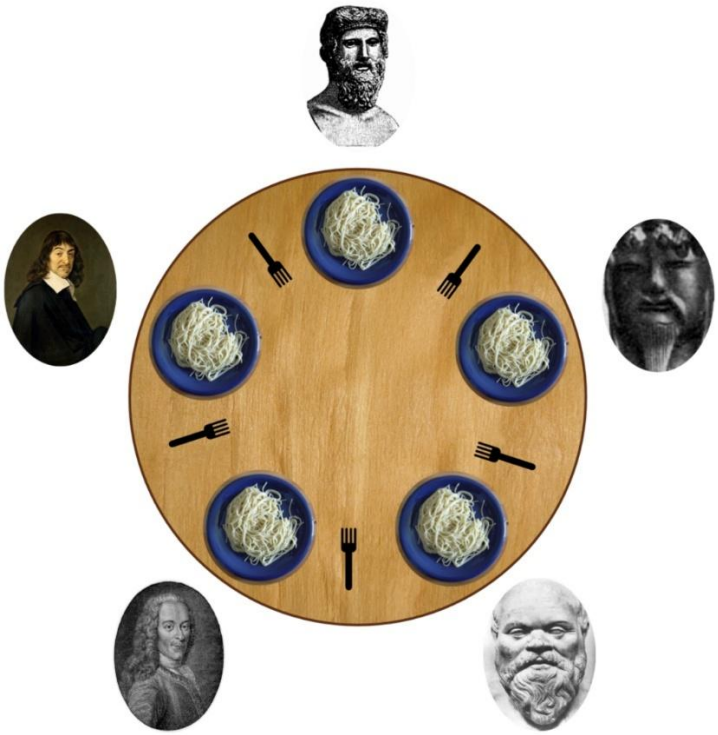


Ilustración 19 - La distribución de la mesa de los filósofos (Platón, Konfuzius, Sócrates, Voltaire y Descartes).

Cada uno de los filosofo alterna entre comer y pensar. Cuando un filósofo tiene hambre, trata de adquirir sus tenedores izquierdo y derecho, uno a la vez, en cualquier orden. Si tiene éxito

[19] Edsger Wybe Dijkstra fue un científico de la computación de los Países Bajos.

al adquirir dos tenedores, come por un momento, después deja los tenedores y continúa pensando.

La cuestión clave es analizar si sería posible escribir un algoritmo para cada filósofo, que haga lo que se supone que debe hacer y nunca se trabe.

```
#define N 5                         /* número de filósofos */

void filosofo(int i)                /* i: número de filósofo, de 0 a 4 */
{
  while(TRUE){
        pensar();                   /* el filósofo está pensando */
        tomar_tenedor(i);           /* toma tenedor izquierdo */
        tomar_tenedor((i+1) % N);   /* toma tenedor derecho; % es el operador módulo */
        comer();                    /* come espagueti */
        poner_tenedor(i);           /* pone tenedor izquierdo de vuelta en la mesa */
        poner_tenedor((i+1) % N);   /* pone tenedor derecho de vuelta en la mesa */
  }
}
```

El código en lenguaje C anterior muestra un primer intento de solución. El procedimiento ***tomar_tenedor*** espera hasta que el tenedor específico esté disponible y luego lo toma. Por desgracia, ***esta solución está mal***. Supongamos que los cinco filósofos toman sus tenedores izquierdos al mismo tiempo. Ninguno podrá tomar sus tenedores derechos y habrá un ***interbloqueo***.

Podríamos modificar el programa de manera que después de tomar el tenedor izquierdo, el programa compruebe para ver si el tenedor derecho está disponible. Si no lo está, el filósofo regresa el tenedor izquierdo, espera cierto tiempo y después repite todo el proceso. ***Esta proposición falla también***, aunque por una razón distinta. Con un poco de mala suerte, todos los filósofos podrían iniciar el algoritmo en forma simultánea, tomarían sus tenedores izquierdos, verían que sus tenedores derechos no están disponibles, regresarían sus tenedores izquierdos, esperarían, volverían a tomar sus tenedores izquierdos al mismo tiempo y así en lo sucesivo, eternamente. Una situación como ésta, en la que todos los programas continúan ejecutándose en forma indefinida pero no progresan se conoce como ***inanición*** (*starvation*).



Ahora podríamos pensar que, si los filósofos sólo esperan por un tiempo aleatorio en vez de esperar durante el mismo tiempo al no poder adquirir el tenedor derecho, la probabilidad de que todo continúe bloqueado durante incluso una hora es muy pequeña. Esta observación es verdad y en casi todas las aplicaciones intentar de nuevo en un tiempo posterior no representa un problema. Por ejemplo, en la red de área local Ethernet, si dos computadoras envían un paquete al mismo tiempo, cada una espera durante un tiempo aleatorio e intenta de nuevo; en la práctica esta solución funciona bien.

Sin embargo, en algunas cuantas aplicaciones sería preferible una solución que funcione siempre y que no pueda fallar debido a una serie improbable de números aleatorios. Pensemos acerca del control de la seguridad en una planta de energía nuclear.

Una mejora al código en C que vimos anteriormente que no tiene interbloqueo ni inanición es proteger las cinco instrucciones que siguen de la llamada a *pensar* mediante un ***semáforo binario***. Antes de empezar a adquirir tenedores, un filósofo realizaría una operación ***down***[20] en ***mutex***[21]. Después de regresar los tenedores, realizaría una operación ***up***[22] en ***mutex***.

Desde un punto de vista teórico, esta solución es adecuada. Desde un punto de vista práctico, tiene un error de rendimiento: ***sólo puede haber un filósofo comiendo en cualquier instante***.

Con cinco tenedores disponibles, deberíamos poder permitir que dos filósofos coman al mismo tiempo.

[20] En informática, la operación down (semáforo s) es una acción que decrementa el valor de un semáforo. Se utiliza para sincronizar procesos y resolver problemas de exclusión mutua y sincronización.
[21] Los mutex son mecanismos de bloqueo y desbloqueo que se utilizan para problemas de exclusión mutua y sección crítica.
[22] Permite que un proceso que estaba inactivo en el semáforo complete su operación down.

```
#define N                5                  /* número de filósofos */
#define IZQUIERDO        (i+N-1)%N          /* número del vecino izquierdo de i */
#define DERECHO          (i+1)%N            /* número del vecino derecho de i */
#define PENSANDO         0                  /* el filósofo está pensando */
#define HAMBRIENTO       1                  /* el filósofo trata de obtener los tenedores */
#define COMIENDO         2                  /* el filósofo está comiendo */
typedef int semaforo;                       /* los semáforos son un tipo especial de int */
int estado[N];                              /* arreglo que lleva registro del estado de todos*/
semaforo mutex = 1;                         /* exclusión mutua para las regiones críticas */
semaforo s[N];                              /* un semáforo por filósofo */

void filosofo(int i)                        /* i: número de filósofo, de 0 a N−1 */
{
     while(TRUE){                           /* se repite en forma indefinida */
          pensar();                         /* el filósofo está pensando */
          tomar_tenedores(i);               /* adquiere dos tenedores o se bloquea */
          comer();                          /* come espagueti */
          poner_tenedores(i);               /* pone de vuelta ambos tenedores en la mesa */
     }
}

void tomar_tenedores(int i)                 /* i: número de filósofo, de 0 a N−1 */
{
     down(&mutex);                          /* entra a la región crítica */
     estado[i] = HAMBRIENTO;                /* registra el hecho de que el filósofo i está hambriento */
     probar(i);                             /* trata de adquirir 2 tenedores */
     up(&mutex);                            /* sale de la región crítica */
     down(&s[i]);                           /* se bloquea si no se adquirieron los tenedores */
}

void poner_tenedores(i)                     /* i: número de filósofo, de 0 a N–1 */
{
     down(&mutex);                          /* entra a la región crítica */
     estado[i] = PENSANDO;                  /* el filósofo terminó de comer */
     probar(IZQUIERDO);                     /* verifica si el vecino izquierdo puede comer ahora */
     probar(DERECHO);                       /* verifica si el vecino derecho puede comer ahora */
     up(&mutex);                            /* sale de la región crítica */
}

void probar(i)              /* i: número de filósofo, de 0 a N−1 */
{
     if (estado[i] == HAMBRIENTO && estado[IZQUIERDO] != COMIENDO && estado[DERECHO] != COMIENDO) {
          estado[i] = COMIENDO;
          up(&s[i]);
     }
}
```

La solución que se presenta en el código anterior está libre de interbloqueos y permite el máximo paralelismo para un número arbitrario de filósofos. Se utiliza un arreglo llamado ***estado*** para llevar el registro de si un filósofo está comiendo, pensando o hambriento (tratando de adquirir tenedores).

Un filósofo sólo se puede mover al estado de comer si ningún vecino está comiendo. Los ***i*** vecinos del filósofo se definen mediante las macros *IZQUIERDO* y *DERECHO*. En otras palabras, si *i* es 2, *IZQUIERDO* es 1 y *DERECHO* es 3.

El código incorpora un arreglo de semáforos, uno por cada filósofo, de manera que los filósofos hambrientos puedan bloquearse si los tenedores que necesitan están ocupados.

Observemos que cada proceso ejecuta el procedimiento ***filosofo*** como su código principal, pero los demás procedimientos (*tomar_tenedores*, *pone_ tenedores* y *probar*) son ordinarios y no procesos separados.

El problema de los filósofos comensales es útil para modelar procesos que compiten por el acceso exclusivo a un número limitado de recursos, como los dispositivos de E/S.

Elemento del Problema	Analogía en Sistemas Operativos
Filósofos	Procesos o hilos de ejecución.
Palillos (Recurso)	Recursos no compartibles (impresoras, registros, archivos).
Comer	Ejecución de la Sección Crítica.
Pensar	Ejecución de la Sección Restante.

El problema de los lectores y escritores.

Otro problema famoso es el de los lectores y escritores, que modela a varios procesos que compiten por acceder a un recurso compartido.

Por ejemplo, imaginemos un sistema de reservación de aerolíneas, con muchos procesos en competencia que desean leer y escribir en él. Es aceptable tener varios procesos que lean la ***base de datos*** al mismo tiempo, pero si un proceso está actualizando (escribiendo) la base de datos, ningún otro proceso puede tener acceso a la base de datos, ni siquiera los lectores.



La cuestión clave en este caso es como programar los lectores y escritores.

```
typedef int semaforo;                   /* use su imaginación */
semaforo mutex=1;                       /* controla el acceso a 'cl' */
semaforo bd=1;                          /* controla el acceso a la base de datos */
int cl=0;                               /* # de procesos que leen o desean hacerlo */

void lector(void)
{
        while(TRUE){                    /* se repite de manera indefinida */
            down(&mutex);               /* obtiene acceso exclusivo a 'cl' */
            cl = cl + 1;                /* ahora hay un lector más */
            if (cl == 1) down(&bd);     /* si este es el primer lector ... */
            up(&mutex);                 /* libera el acceso exclusivo a 'cl' */
            leer_base_de_datos();       /* accede a los datos */
            down(&mutex);               /* obtiene acceso exclusivo a 'cl' */
            cl = cl – 1;                /* ahora hay un lector menos */
            if (cl == 0) up(&bd);       /* si este es el último lector ... */
            up(&mutex);                 /* libera el acceso exclusivo a 'cl' */
            usar_lectura_datos();       /* región no crítica */
        }
}

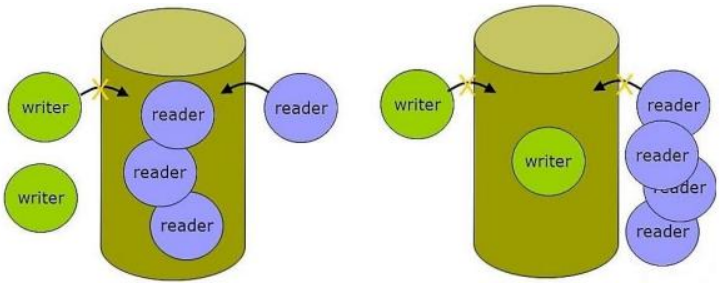
void escritor(void)
{
            while(TRUE){                /* se repite de manera indefinida */
            pensar_datos();             /* región no crítica */
            down(&bd);                  /* obtiene acceso exclusivo */
            escribir_base_de_datos();   /* actualiza los datos */
            up(&bd);                    /* libera el acceso exclusivo */
        }
}
```

En la solución que se muestra en el código anterior, el primer lector en obtener acceso a la base de datos realiza una operación ***down*** en el semáforo ***bd***. Los siguientes lectores simplemente incrementan un contador llamado ***cl***.

A medida que los lectores van saliendo, decrementan el contador y el último realiza una operación ***up*** en el semáforo, para permitir que un escritor bloqueado (si lo hay) entre.

La solución que se presenta aquí contiene en forma implícita una decisión sutil que vale la pena observar. Supongamos que mientras un lector utiliza la base de datos, llega otro lector. Como no es un problema tener dos lectores al mismo tiempo, el segundo lector es admitido. También se pueden admitir más lectores, si es que llegan.

Ahora supongamos que aparece un escritor. Tal vez éste no sea admitido a la base de datos, ya que los escritores deben tener acceso exclusivo y, por ende, el escritor se suspende. Más adelante aparecen lectores adicionales. Mientras que haya un lector activo, se admitirán los siguientes lectores.

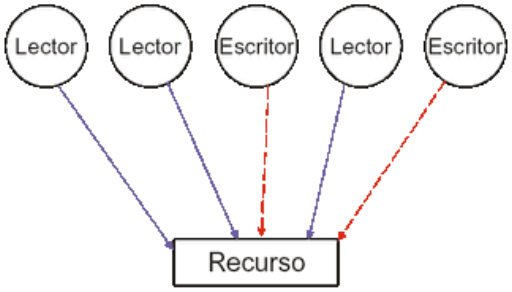


Como consecuencia de esta estrategia, mientras que haya un suministro continuo de lectores, todos entrarán tan pronto lleguen. El escritor estará suspendido hasta que no haya un lector presente. Si llega un nuevo lector, por decir cada 2 segundos y cada lector requiere 5 segundos para hacer su trabajo, el escritor nunca entrará.

Para evitar esta situación, el programa se podría escribir de una manera ligeramente distinta: cuando llega un lector y hay un escritor en espera, el lector se suspende detrás del escritor, en vez de ser admitido de inmediato.

De esta forma, un escritor tiene que esperar a que terminen los lectores que estaban activos cuando llegó, pero no tiene que esperar a los lectores que llegaron después de él.

La desventaja de esta solución es que logra una menor concurrencia y, por ende, un menor rendimiento.



El problema productor/consumidor.

Uno de los problemas más comunes afrontados en programación concurrente es el ***problema productor/consumidor***. El enunciado general es éste: hay uno o más procesos generando algún tipo de datos (registros, caracteres) y poniéndolos en un *buffer*. Hay un único consumidor que está extrayendo datos de dicho *buffer* de uno en uno. El sistema está obligado a impedir la superposición de las operaciones sobre los datos. Es decir, sólo un agente (productor o consumidor) puede acceder al *buffer* en un momento dado.

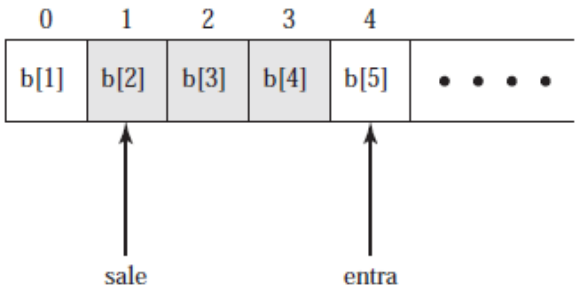
Hay varias soluciones a este problema que ilustran tanto la potencia como las dificultades de los semáforos.

Para empezar, asumiremos que el *buffer* es infinito y consiste en un vector de datos. En términos abstractos, las funciones ***productor*** y ***consumidor*** se definen así:

```
productor:
while (true)
{
    /* producir dato v */;
    b[entra] = v;
    entra++;
}
```

```
consumidor:
while (true)
{
    while (entra <= sale)
        /* no hacer nada */;
    w = b[sale];
    sale++;
    /* consumir dato w */
}
```

Y la siguiente figura ilustra la estructura del *buffer* b (definida previamente):

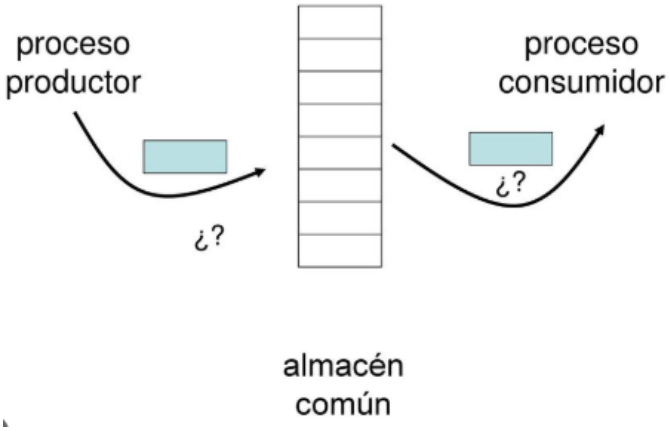


Nota: el área sombreada indica la porción del *buffer* que está ocupada

El productor puede generar datos y almacenarlos en el vector a su propio ritmo. Cada vez, se incrementa un índice sobre el vector (*entra*).

El consumidor procede de manera similar, pero debe asegurarse de que no intenta leer de una entrada vacía del vector. Por lo tanto, el consumidor se asegura de que el productor ha avanzado más allá que él (*entra > sale*) antes de seguir.

Este sistema se podría implementar utilizando ***semáforos binarios***. En cambio, si se utilizan semáforos generales (también denominados ***semáforos con contador***) puede obtenerse una solución algo más clara.


proceso
productor
¿?
proceso
consumidor
¿?
almacén
común

```
/* programa productor consumidor */
int n;
binary_semaphore s = 1;
binary_semaphore retardo = 0;
void productor()
{
    while (true)
    {
        producir();
        semWaitB(s);
        anyadir();
        n++;
        if (n==1) semSignalB(retardo);
        semSignalB(s);
    }
}
void consumidor()
{
    int m; /* una variable local */
    semWaitB(retardo);
    while (true)
    {
        semWaitB(s);
        extraer();
        n—;
        m = n;
        semSignalB(s);
        consumir();
        if (m==0) semWaitB(retardo);
    }
}
void main()
{
    n = 0;
    paralelos (productor, consumidor);
}
```

Ilustración 20 - Una solución al problema productor/consumidor con buffer infinito usando semáforos binarios.

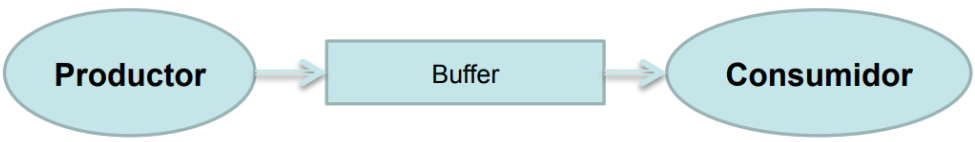
```
/* programa productor consumidor */
semaphore n = 0;
semaphore s = 1;
void productor()
{
    while (true)
    {
        producir();
        semWait(s);
        anyadir();
        semSignal(s);
        semSignal(n);
    }
}
void consumidor()
{
    while (true)
    {
        semWait(n);
        semWait(s);
        extraer();
        semSignal(s);
        consumir();
    }
}
void main()
{
    paralelos (productor, consumidor);
}
```

Ilustración 21 - Una solución al problema productor/consumidor con buffer infinito usando semáforos.

Problema del búfer limitado.

Ahora supongamos que el productor y el consumidor comparten un ***búfer de tamaño fijo;*** y que se mantiene la idea de que el trabajo del *productor* es generar datos y colocarlos en el búfer, al tiempo que el *consumidor* consume los datos de uno a la vez.

El problema es asegurarse de que el productor no trate de añadir datos en el búfer si está lleno y que el consumidor no trate de eliminar datos de un búfer vacío.



Supongamos que una cola consta de n búferes, siendo cada uno capaz de almacenar un elemento. El ***semáforo binario*** proporciona exclusión mutua para los accesos al conjunto de búferes y se inicializa con el valor 1.

Los semáforos ***empty*** y ***full*** (semáforos generales o con contador) cuentan el número de búferes vacíos y llenos.

El semáforo empty se inicializa con el valor n; el semáforo full se inicializa con el valor 0.

Estructura Proceso Productor	Estructura Proceso Consumidor

`do {`
`. . .`
`item = produceItem();`
`. . .`
`wait(empty);`
`wait(mutex);`
`. . .`
`putItemIntoBuffer(item);`
`. . .`
`signal(mutex);`
`signal(full);`
 	
`} while (true);`
	`do {`
	`. . .`
	`wait(full);`
	`wait(mutex);`
	`. . .`
	`item = removeItemFromBuffer();`
	`. . .`
	`signal(mutex);`
	`signal(empty);`
	`. . .`
	`consumeItem(item);`
	
	`} while (true);`

Observemos la simetría entre el productor y el consumidor: podemos interpretar este código como un productor que genera los búferes llenos para el consumidor o como un consumidor que genera búferes vacíos para el productor.

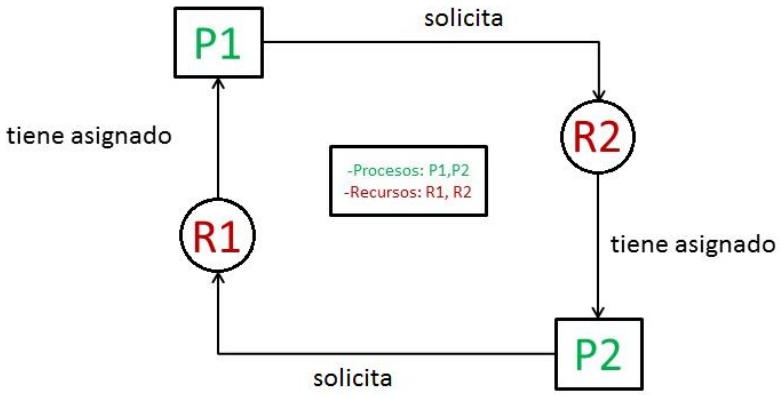
Interbloqueos e inanición.

La implementación de un semáforo con una cola de espera puede dar lugar a una situación en la que dos o más procesos estén esperando indefinidamente a que se produzca un suceso que sólo puede producirse como consecuencia de las operaciones efectuadas por otro de los procesos en espera. El suceso en cuestión es la ejecución de una operación signal.

Cuando se llega a un estado así, se dice que estos procesos se han ***interbloqueado***.

Un problema relacionado con los interbloqueos es el del bloqueo indefinido o muerte por ***inanición***, una situación en la que algunos procesos esperan de forma indefinida dentro del semáforo. El bloqueo indefinido puede producirse si añadimos y eliminamos los procesos a la lista asociada con el semáforo utilizando un esquema LIFO[23].

[23] El esquema LIFO (Last In, First Out) es un método de gestión de inventarios que consiste en priorizar la salida de los últimos artículos que llegan al almacén.



Referencias y bibliografía de la sección.

- Silberschatz 6-6.
- Tanenbaum 2-5.
- Stallings 5-3 pág. 215 / 5-6 pág. 241.

Planificación del uso de la CPU.

Conceptos básicos.

En un sistema de un único procesador, sólo puede ejecutarse un proceso cada vez; cualquier otro proceso tendrá que esperar hasta que la CPU quede libre y pueda volver a planificarse.

El objetivo de la multiprogramación es tener continuamente varios procesos en ejecución, con el fin de maximizar el uso de la CPU. La idea es bastante simple: un proceso se ejecuta hasta que tenga que esperar, normalmente porque es necesario completar alguna solicitud de E/S.

En un sistema informático simple (monoproceso), la CPU permanece entonces inactiva y todo el tiempo de espera se desperdicia; no se realiza ningún trabajo útil. Con la multiprogramación, se intenta usar ese tiempo de forma productiva.

En este último caso, se mantienen varios procesos en memoria a la vez. Cuando un proceso tiene que esperar, el sistema operativo retira el uso de la CPU a ese proceso y se lo cede a otro proceso. Este patrón se repite continuamente y cada vez que un proceso tiene que esperar, otro proceso puede hacer uso de la CPU.

Si sólo hay una CPU disponible, hay que decidir cuál proceso se va a ejecutar a continuación. La parte del sistema operativo que realiza esa decisión se conoce como ***planificador de procesos*** y el algoritmo que utiliza se conoce como ***algoritmo de planificación***.

Muchas de las mismas cuestiones que se aplican a la planificación de procesos también se aplican a la planificación de hilos, aunque algunas son distintas. Cuando el kernel administra hilos, por lo general la planificación se lleva a cabo por hilo, e importa muy poco (o nada) a cuál proceso pertenece ese hilo.

La planificación de la CPU es una función fundamental del sistema operativo; casi todos los recursos de la computadora se planifican antes de usarlos. Por supuesto, la CPU es uno de los principales recursos de la computadora, así que su correcta planificación resulta crucial en el diseño del sistema operativo.

Ciclo de ráfagas de CPU y de E/S.

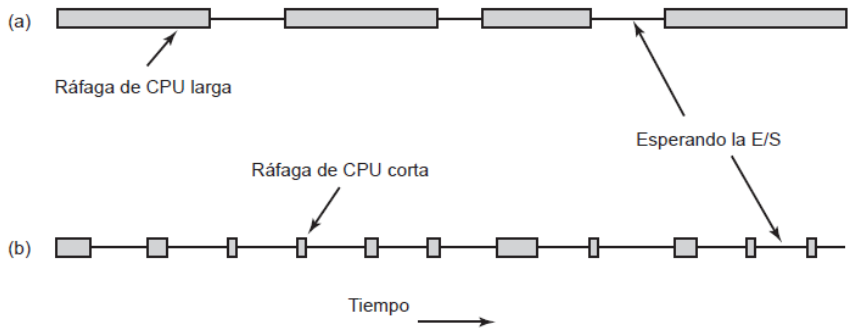
La adecuada planificación de la CPU depende de una propiedad observada de los procesos: ***la ejecución de un proceso consta de un ciclo de ejecución en la CPU***, seguido de una espera de E/S; los procesos alternan entre estos dos estados.

La ejecución del proceso comienza con una ráfaga de CPU[24]. Ésta va seguida de una ráfaga de E/S[25], a la cual sigue otra ráfaga de CPU, luego otra ráfaga de E/S, etc. Finalmente, la ráfaga definitiva de CPU concluye con una solicitud al sistema para terminar la ejecución.

Por lo general la CPU opera durante cierto tiempo sin detenerse, después se realiza una llamada al sistema para leer datos de un archivo o escribirlos en el mismo. Cuando se completa la llamada al sistema, la CPU realiza cálculos de nuevo hasta que necesita más datos o tiene que escribir más datos y así sucesivamente.

Hay que tener en cuenta que algunas actividades de E/S cuentan como cálculos. Por ejemplo, cuando la CPU copia bits a una RAM de video para actualizar la pantalla, está calculando y no realizando operaciones de E/S, ya que la CPU está en uso.

En este sentido, la E/S es cuando un proceso entra al estado bloqueado en espera de que un dispositivo externo complete su trabajo.



La duración de las ráfagas de CPU se ha medido exhaustivamente en la práctica. Aunque varían enormemente de un proceso a otro y de una computadora a otra, tienden a presentar una

[24] Una ráfaga de CPU es el tiempo que un proceso necesita usar el procesador. La duración de las ráfagas varía entre procesos y entre computadoras.
[25] Una ráfaga de E/S es un período de tiempo en el que un proceso utiliza o espera por la E/S.

curva es de tipo exponencial, con un gran número de ráfagas de CPU cortas y un número menor de ráfagas de CPU largas.

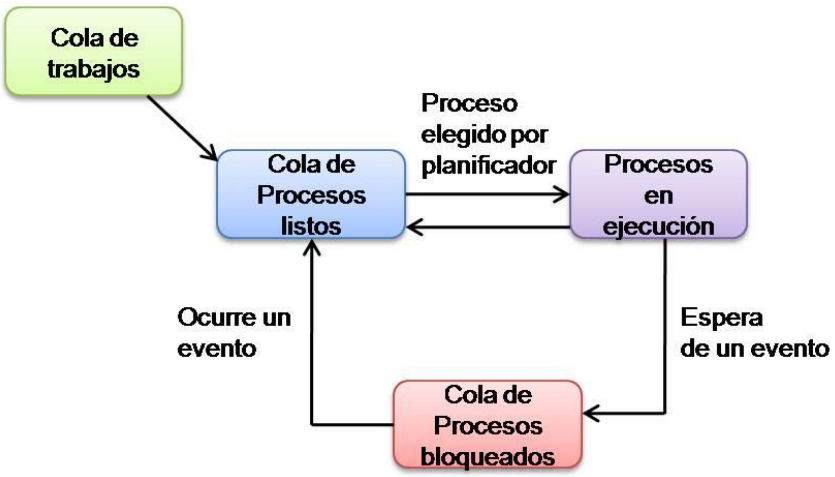


Normalmente, un programa limitado por E/S presenta muchas ráfagas de CPU cortas. Esta distribución puede ser importante en la selección de un algoritmo apropiado para la planificación de la CPU.

Planificador de la CPU.

Cuando la CPU queda inactiva, el sistema operativo debe seleccionar uno de los procesos que se encuentran en la cola de procesos preparados para ejecución. El planificador a corto plazo (o planificador de la CPU) lleva a cabo esa selección del proceso.

El planificador elige uno de los procesos que están en memoria preparados para ejecutarse y asigna la CPU a dicho proceso. La cola de procesos preparados no necesariamente tiene que ser una cola FIFO (first in, first out). Una cola de procesos preparados puede implementarse como una cola FIFO, una cola prioritaria, un árbol o simplemente una lista enlazada no ordenada.



Sin embargo, conceptualmente, todos los procesos que se encuentran en la cola de procesos preparados se ponen en fila esperando la oportunidad de ejecutarse en la CPU. Los registros que se almacenan en las colas son, generalmente, bloques de control de proceso (***PCB***) que describen los procesos en cuestión.

Planificación apropiativa y cooperativa.

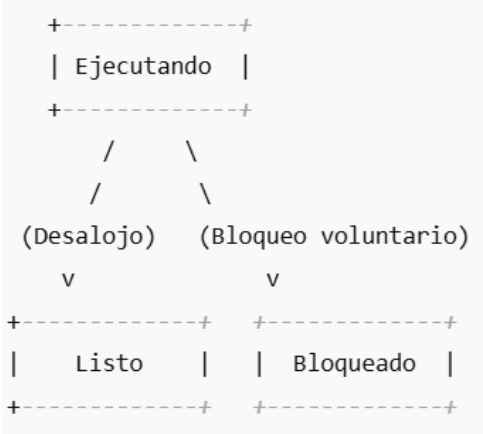
Puede ser necesario tomar decisiones sobre planificación de la CPU en las siguientes cuatro circunstancias:

1. Cuando un proceso cambia del estado de ejecución al estado de espera (por ejemplo, como resultado de una solicitud de E/S o de una invocación de ***wait*** para esperar a que termine uno de los procesos hijo).
2. Cuando un proceso cambia del estado de ejecución al estado preparado (por ejemplo, cuando se produce una interrupción).
3. Cuando un proceso cambia del estado de espera al estado preparado (por ejemplo, al completarse una operación de E/S).
4. Cuando un proceso termina.

En las situaciones 1 y 4, no hay más que una opción en términos de planificación: debe seleccionarse un nuevo proceso para su ejecución (si existe algún proceso en la cola de procesos preparados).

Sin embargo, en las situaciones 2 y 3 sí que existe la opción de planificar un nuevo proceso o no.

Cuando las decisiones de planificación sólo tienen lugar en las circunstancias 1 y 4, decimos que el esquema de planificación es sin desalojo o cooperativo; en caso contrario, se trata de un esquema ***apropiativo***.



En la planificación sin desalojo, una vez que se ha asignado la CPU a un proceso, el proceso se mantiene en la CPU hasta que ésta es liberada bien por la terminación del proceso o bien por

la conmutación al estado de espera. Este método de planificación era el utilizado por Microsoft Windows 3.x; Windows 95 introdujo la planificación apropiativa y todas las versiones siguientes de los sistemas operativos Windows han usado dicho tipo de planificación.

El sistema operativo Mac OS X para Macintosh utiliza la planificación apropiativa; las versiones anteriores del sistema operativo de Macintosh se basaban en la planificación cooperativa.

La planificación cooperativa es el único método que se puede emplear en determinadas plataformas hardware, dado que no requiere el hardware especial (por ejemplo, un temporizador) necesario para la planificación apropiativa.

Lamentablemente, la planificación apropiativa tiene un coste asociado con el acceso a los datos compartidos. Consideremos el caso de dos procesos que compartan datos y supongamos que, mientras que uno está actualizando los datos, resulta desalojado con el fin de que el segundo proceso pueda ejecutarse. El segundo proceso podría intentar entonces leer los datos, que se encuentran en un estado incoherente. En tales situaciones, son necesarios nuevos mecanismos para coordinar el acceso a los datos compartidos.

La técnica de desalojo también afecta al diseño del kernel del sistema operativo. Durante el procesamiento de una llamada al sistema, el kernel puede estar ocupado realizando una actividad en nombre de un proceso. Tales actividades pueden implicar cambios importantes en los datos del kernel (por ejemplo, en las colas de E/S). Hay que considerar qué ocurriría si el proceso se desaloja en mitad de estos cambios y el kernel (o el controlador del dispositivo) necesita leer o modificar la misma estructura. El resultado será un auténtico caos. Ciertos sistemas operativos, incluyendo la mayor parte de las versiones de UNIX, resuelven este problema esperando a que se complete la llamada al sistema o a que se transfiera un bloque de E/S antes de hacer un cambio de contexto[26].

Esta solución permite obtener una estructura del kernel simple, ya que el kernel no desalojará ningún proceso mientras que las estructuras de datos del kernel se encuentren en un estado incoherente.

Lamentablemente, este modelo de ejecución del kernel no resulta adecuado para permitir la realización de cálculos en tiempo real y el multiprocesamiento.

Dado que, por definición, las interrupciones pueden producirse en cualquier momento, y puesto que no siempre pueden ser ignoradas por el kernel, las secciones de código afectadas por las interrupciones deben ser resguardadas de un posible uso simultáneo. Sin embargo, el sistema operativo tiene que aceptar interrupciones casi continuamente, ya que de otra manera podrían perderse valores de entrada o sobrescribirse los valores de salida; por esto, para que no puedan acceder de forma concurrente varios procesos a estas secciones de código, lo que se hace es desactivar las interrupciones al principio de cada sección y volver a activarlas al final.

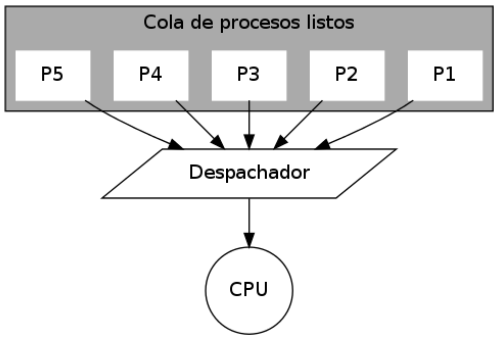
Es importante observar que no son muy numerosas las secciones de código que desactivan las interrupciones y que, normalmente, esas secciones contienen pocas instrucciones.

[26] Un cambio de contexto consiste en la ejecución de una rutina perteneciente al núcleo del sistema operativo multitarea de una computadora, cuyo propósito es parar la ejecución de un hilo o proceso para dar paso a la ejecución de otro distinto.

El despachador.

Un componente importante implicado en la función de planificación de la CPU es el despachador. Proporciona el control de la CPU a los procesos seleccionados por el planificador. Esta función implica lo siguiente:

- Cambio de contexto.
- Cambio al modo usuario.
- Salto a la posición correcta dentro del programa de usuario para reiniciar dicho programa.



El despachador debe ser lo más rápido posible, ya que se invoca en cada conmutación de proceso. El tiempo que tarda el despachador en detener un proceso e iniciar la ejecución de otro se conoce como latencia de despacho.

Criterios de planificación.

Los diferentes algoritmos de planificación de la CPU tienen distintas propiedades, y la elección de un algoritmo en particular puede favorecer una clase de procesos sobre otros.

A la hora de decidir qué algoritmo utilizar en una situación particular, debemos considerar las propiedades de los diversos algoritmos.

Se han sugerido muchos criterios para comparar los distintos algoritmos de planificación. Las características que se usen para realizar la comparación pueden afectar enormemente a la determinación de cuál es el mejor algoritmo. Los criterios son los siguientes:

- **Utilización de la CPU.** Deseamos mantener la CPU tan ocupada como sea posible. Conceptualmente, la utilización de la CPU se define en el rango comprendido entre el 0 y el 100 por cien. En un sistema real, debe variar entre el 40 por ciento (para un sistema ligeramente cargado) y el 90 por ciento (para un sistema intensamente utilizado).
- **Tasa de procesamiento.** Si la CPU está ocupada ejecutando procesos, entonces se estará llevando a cabo algún tipo de trabajo. Una medida de esa cantidad de trabajo es el número de procesos que se completan por unidad de tiempo, y dicha medida se denomina tasa de procesamiento. Para procesos de larga duración, este valor puede ser de un proceso por hora; para transacciones cortas, puede ser de 10 procesos por segundo.

- **Tiempo de ejecución.** Desde el punto de vista de un proceso individual, el criterio importante es cuánto tarda en ejecutarse dicho proceso. El intervalo que va desde el instante en que se ordena la ejecución de un proceso hasta el instante en que se completa es el tiempo de ejecución. Ese tiempo de ejecución es la suma de los períodos que el proceso invierte en esperar para cargarse en memoria, esperar en la cola de procesos preparados, ejecutarse en la CPU y realizar las operaciones de E/S.
- **Tiempo de espera.** El algoritmo de planificación de la CPU no afecta a la cantidad de tiempo durante la que un proceso se ejecuta o hace una operación de E/S; afecta sólo al período de tiempo que un proceso invierte en esperar en la cola de procesos preparados. El tiempo de espera es la suma de los períodos invertidos en esperar en la cola de procesos preparados.
- **Tiempo de respuesta.** En un sistema interactivo, el tiempo de ejecución puede no ser el mejor criterio. A menudo, un proceso puede generar parte de la salida con relativa rapidez y puede continuar calculando nuevos resultados mientras que los resultados previos se envían a la salida para ponerlos a disposición del usuario. Por tanto, otra medida es el tiempo transcurrido desde que se envía una solicitud hasta que se produce la primera respuesta. Esta medida, denominada tiempo de respuesta, es el tiempo que el proceso tarda en empezar a responder, no el tiempo que tarda en enviar a la salida toda la información de respuesta. Generalmente, el tiempo de respuesta está limitado por la velocidad del dispositivo de salida.

El objetivo consiste en maximizar la utilización de la CPU y la tasa de procesamiento, y minimizar el tiempo de ejecución, el tiempo de espera y el tiempo de respuesta. En la mayoría de los casos, lo que se hace es optimizar algún tipo de valor promedio. Sin embargo, en determinadas circunstancias, resulta deseable optimizar los valores máximo y mínimo en lugar del promedio. Por ejemplo, para garantizar que todos los usuarios tengan un buen servicio, podemos tratar de minimizar el tiempo de respuesta máximo.

Diversos investigadores han sugerido que, para los sistemas interactivos (como, por ejemplo, los sistemas de tiempo compartido), es más importante minimizar la varianza del tiempo de respuesta que minimizar el tiempo medio de respuesta. Un sistema con un tiempo de respuesta razonable y predecible puede considerarse más deseable que un sistema que sea más rápido por término medio, pero que resulte altamente variable.

Mas sobre planificación.

En los días de los sistemas de procesamiento por lotes, con entrada en forma de imágenes de tarjetas en una cinta magnética, el algoritmo de planificación era simple: bastaba con ejecutar el siguiente trabajo en la cinta. Con los sistemas de multiprogramación, el algoritmo de planificación se volvió más complejo debido a que comúnmente había varios usuarios esperando ser atendidos.

Debido a que el tiempo de la CPU es un recurso escaso en estas máquinas, un buen planificador puede hacer una gran diferencia en el rendimiento percibido y la satisfacción del usuario. En consecuencia, se ha puesto gran esfuerzo en idear algoritmos de planificación astutos y eficientes.

Con la llegada de las computadoras personales, la situación cambió de dos maneras. En primer lugar, la mayor parte del tiempo sólo hay un proceso activo. Es muy poco probable que un

usuario que entra a un documento en un procesador de palabras compile al mismo tiempo un programa en segundo plano. Cuando el usuario escribe un comando para el procesador de palabras, el planificador no tiene que esforzarse mucho en averiguar qué proceso ejecutar; el procesador de palabras es el único candidato.

En segundo lugar, las computadoras se han vuelto tan veloces con el paso de los años que la CPU es raras veces un recurso escaso. La mayoría de los programas para computadoras personales están limitados en cuanto a la velocidad a la que el usuario puede presentar los datos de entrada (al escribir o hacer clic), no por la velocidad con que la CPU puede procesarlos.

Incluso las compilaciones, que en el pasado consumían muchos ciclos de la CPU, requieren unos cuantos segundos en la mayoría de los casos hoy en día. Aún y cuando se estén ejecutando dos programas al mismo tiempo, como un procesador de palabras y una hoja de cálculo, raras veces es importante saber quién se ejecuta primero, ya que el usuario probablemente está esperando a que ambos terminen.

Además de elegir el proceso correcto que se va a ejecutar a continuación, el planificador también tiene que preocuparse por hacer un uso eficiente de la CPU, debido a que la conmutación de procesos es cara. Para empezar, debe haber un cambio del modo de usuario al modo kernel. Después se debe guardar el estado del proceso actual, incluyendo el almacenamiento de sus registros en la tabla de procesos para que puedan volver a cargarse más adelante.

Cuando planificar procesos.

Una cuestión clave relacionada con la planificación es saber cuándo tomar decisiones de planificación.

Resulta ser que hay una variedad de situaciones en las que se necesita la planificación. En primer lugar, cuando se crea un nuevo proceso se debe tomar una decisión en cuanto a si se debe ejecutar el proceso padre o el proceso hijo. Como ambos procesos se encuentran en el estado listo, es una decisión normal de programación y puede ejecutar cualquiera; es decir, el programador de procesos puede elegir ejecutar de manera legítima, ya sea el padre o el hijo.

En segundo lugar, se debe tomar una decisión de planificación cuando un proceso termina. Ese proceso ya no se puede ejecutar (debido a que ya no existe), por lo que se debe elegir algún otro proceso del conjunto de procesos listos. Si no hay un proceso listo, por lo general se ejecuta un proceso inactivo suministrado por el sistema.

En tercer lugar, cuando un proceso se bloquea por esperar una operación de E/S, un semáforo o por alguna otra razón, hay que elegir otro proceso para ejecutarlo. Algunas veces la razón del bloqueo puede jugar un papel en la elección. Por ejemplo, si ***A*** es un proceso importante y está esperando a que ***B*** salga de su región crítica, si dejamos que ***B*** se ejecute a continuación podrá salir de su región crítica y, por ende, dejar que ***A*** continúe. Sin embargo, el problema es que el planificador comúnmente no tiene la información necesaria para tomar en cuenta esta dependencia.

En cuarto lugar, cuando ocurre una interrupción de E/S tal vez haya que tomar una decisión de planificación. Si la interrupción proviene de un dispositivo de E/S que ha terminado su trabajo, tal vez ahora un proceso que haya estado bloqueado en espera de esa operación de E/S esté

listo para ejecutarse. Es responsabilidad del planificador decidir si debe ejecutar el proceso que acaba de entrar al estado listo, el proceso que se estaba ejecutando al momento de la interrupción, o algún otro.

Si un reloj de hardware proporciona interrupciones periódicas a 50 o 60 Hz o cualquier otra frecuencia, se puede tomar una decisión de planificación en cada interrupción de reloj o en cada *k*-ésima interrupción de reloj.

Categorías de los algoritmos de planificación.

No es sorprendente que distintos entornos requieran algoritmos de planificación diferentes. Esta situación se presenta debido a que las diferentes áreas de aplicación (y los distintos tipos de sistemas operativos) tienen diferentes objetivos. En otras palabras, lo que el planificador debe optimizar no es lo mismo en todos los sistemas.

Tres de los entornos que vale la pena mencionar son:

- Procesamiento por lotes.
- Interactivo.
- De tiempo real.

Los sistemas de procesamiento por lotes se utilizaron durante muchos años para realizar nóminas, inventarios, cuentas por cobrar, cuentas por pagar, cálculos de interés (en los bancos), procesamiento de reclamaciones (en las compañías de seguros) y otras tareas periódicas.

En los sistemas de ***procesamiento por lotes*** no hay usuarios que esperen impacientemente en sus terminales para obtener una respuesta rápida a una petición corta. En consecuencia, son aceptables los algoritmos no apropiativos (o apropiativos con largos periodos para cada proceso). Este método reduce la conmutación de procesos y, por ende, mejora el rendimiento.

En realidad, los algoritmos de procesamiento por lotes son bastante generales y a menudo se pueden aplicar a otras situaciones también, lo cual hace que valga la pena estudiarlos, incluso para las personas que no están involucradas en la computación con mainframes corporativas.

En un entorno con usuarios ***interactivos***, la apropiación es esencial para evitar que un proceso acapare la CPU y niegue el servicio a los demás. Aun si no hubiera un proceso que se ejecutara indefinidamente de manera intencional, podría haber un proceso que deshabilitara a los demás de manera indefinida, debido a un error en el programa. La apropiación es necesaria para evitar este comportamiento.

Los servidores también entran en esta categoría, ya que por lo general dan servicio a varios usuarios (remotos), todos los cuales siempre tienen mucha prisa.

En los sistemas con restricciones de ***tiempo real***, aunque parezca extraño, la apropiación a veces es no necesaria debido a que los procesos saben que no se pueden ejecutar durante periodos extensos, que por lo general realizan su trabajo y se bloquean con rapidez.

La diferencia con los sistemas interactivos es que los sistemas de tiempo real sólo ejecutan programas destinados para ampliar la aplicación en cuestión. Los sistemas interactivos son de

propósito general y pueden ejecutar programas arbitrarios que no sean cooperativos, o incluso malintencionados.

Planificación de hilos.

Cuando varios procesos tienen múltiples hilos cada uno, tenemos dos niveles de paralelismo presentes: procesos e hilos. La planificación en tales sistemas difiere en forma considerable, dependiendo de si hay soporte para hilos a nivel usuario o para hilos a nivel kernel (o ambos).

Consideremos primero los hilos a nivel usuario. Como el kernel no está consciente de la existencia de los hilos, opera en la misma forma de siempre: selecciona un proceso, por decir ***A***, y otorga a este proceso el control de su quántum[27].

El planificador de hilos dentro de ***A*** decide cuál hilo ejecutar, por decir ***A1***. Como no hay interrupciones de reloj para multiprogramar hilos, este hilo puede continuar ejecutándose todo el tiempo que quiera. Si utiliza todo el quántum del proceso, el kernel seleccionará otro proceso para ejecutarlo.

Cuando el proceso ***A*** por fin se ejecute de nuevo, el hilo ***A1*** continuará su ejecución. Seguirá consumiendo todo el tiempo de ***A*** hasta que termine. Sin embargo, su comportamiento antisocial no afectará a los demás procesos. Éstos recibirán lo que el planificador de procesos considere que es su parte apropiada, sin importar lo que esté ocurriendo dentro del proceso ***A***.

Ahora consideremos el caso en el que los hilos de ***A*** tienen relativamente poco trabajo que realizar por cada ráfaga de la CPU; por ejemplo, 5 mseg (milisegundos) de trabajo dentro de un quántum de 50 mseg. En consecuencia, cada uno se ejecuta por unos instantes y después entrega la CPU al planificador de hilos.

Esto podría producir la secuencia ***A1***, ***A2***, ***A3***, ***A1***, ***A2***, ***A3***, ***A1***, ***A2***, ***A3***, ***A1*** antes de que el kernel conmute al proceso ***B***.

Ahora consideremos la situación con hilos a nivel kernel. Acá el kernel selecciona un hilo específico para ejecutarlo. No tiene que tomar en cuenta a cuál proceso pertenece el hilo, pero puede hacerlo si lo desea. El hilo recibe un quántum y se suspende obligatoriamente si se excede de este quántum. Con un quántum de 50 mseg por hilos que se bloquean después de 5 mseg, el orden de los hilos para cierto periodo de 30 mseg podría ser ***A1***, ***B1***, ***A2***, ***B2***, ***A3***, ***B3***, algo que no sería posible con estos parámetros e hilos a nivel usuario.

Una diferencia importante entre los hilos a nivel usuario y los hilos a nivel kernel es el rendimiento. Para realizar una conmutación de hilos a nivel usuario se requiere de muchas instrucciones de máquina. Con hilos a nivel kernel se requiere una conmutación de contexto total, cambiar el mapa de memoria e invalidar la caché, lo cual es varias órdenes de magnitud más lento. Por otro lado, con los hilos a nivel kernel, cuando un hilo se bloquea en espera de E/S no se suspende todo el proceso, como con los hilos a nivel usuario.

[27] Intervalo de tiempo que se le asigna a un proceso durante el cual puede ejecutarse.

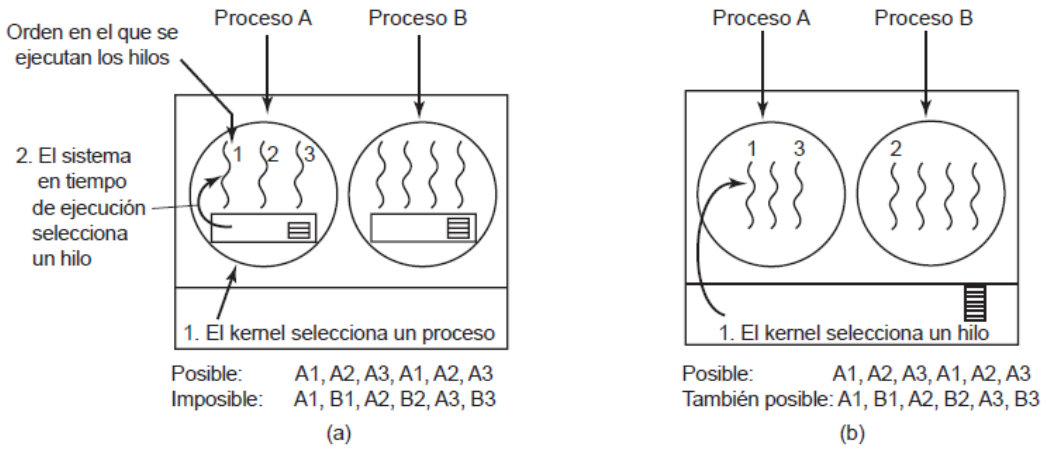


Ilustración 22 - (a) Posible planificación de hilos a nivel usuario con un quántum de 50 mseg para cada proceso e hilos que se ejecutan durante 5 mseg por cada ráfaga de la CPU. (b) Posible planificación de hilos a nivel kernel con las mismas características que (a).

Como el kernel sabe que es más costoso conmutar de un hilo en el proceso ***A*** a un hilo en el proceso ***B*** que ejecutar un segundo hilo en el proceso ***A*** (debido a que tiene que cambiar el mapa de memoria y se arruina la memoria caché), puede considerar esta información al tomar una decisión.

Por ejemplo, dados dos hilos que son de igual importancia, en donde uno de ellos pertenece al mismo proceso que un hilo que se acaba de bloquear y el otro pertenece a un proceso distinto, se podría dar preferencia al primero.

Otro factor importante es que los hilos a nivel usuario pueden emplear un planificador de hilos específico para la aplicación. Por ejemplo, consideremos el servidor Web; supongamos que un hilo trabajador acaba de bloquearse y que el hilo despachador, así como dos hilos trabajadores están listos. Sería importante determinar quién debe ejecutarse a continuación. Por esto, el sistema en tiempo de ejecución, que sabe lo que todos los hilos hacen, puede seleccionar fácilmente el despachador para que se ejecute a continuación, de manera que pueda iniciar la ejecución de otro hilo trabajador.

Esta estrategia maximiza la cantidad de paralelismo en un entorno en el que los trabajadores se bloquean con frecuencia en la E/S de disco. Con los hilos a nivel kernel, el kernel nunca sabría lo que hace cada hilo (aunque se les podría asignar distintas prioridades). Sin embargo, en general los planificadores de hilos específicos para la aplicación pueden optimizar mejor una aplicación de lo que el kernel puede.

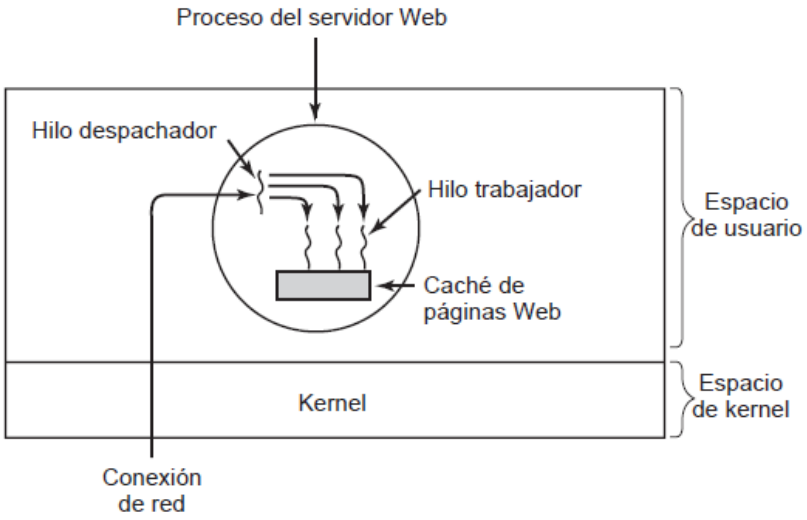


Ilustración 23 - Un servidor Web con múltiples hilos.

Niveles de planificación.

La planificación a ***corto plazo*** decide cómo compartir momento a momento la CPU entre todos los procesos que requieren de sus recursos. La planificación a corto plazo se lleva a cabo una gran cantidad de veces por segundo (razón por la cual el código debe ser eficiente y rápido). En este nivel de planificación se definen los procesos que están listos para ejecución. En algunos textos, al planificador a corto plazo se lo asocia directamente con el despachador (dispatcher).

La planificación a ***medio plazo*** decide que procesos es conveniente bloquear en determinado momento, sea por escasez/saturación de algún recurso o porque están realizando alguna solicitud que no puede satisfacerse momentáneamente. Este nivel de planificación se encarga de tomar decisiones respecto a los procesos conforme entran y salen del estado de bloqueado. En la planificación a medio plazo se suspenden o reintroducen los procesos que se encuentran en la memoria, controlando su traslado entre la memoria principal y el almacenamiento secundario (disco) para optimizar la utilización de recursos y el rendimiento general del sistema. Esta planificación ayuda a mantener un equilibrio, gestionando procesos bloqueados o en espera y decidiendo cuándo un proceso suspendido debe ser traído de vuelta a la memoria para su ejecución.

La planificación a **largo plazo** decide qué procesos serán los siguientes en ser iniciados. Este nivel se encarga de controlar qué procesos entran al sistema desde el almacenamiento secundario (disco) hacia la memoria principal. Decide cuántos y cuáles procesos se permiten ejecutar en el sistema en un momento dado para mantener un equilibrio entre la carga del sistema y la disponibilidad de recursos. En los sistemas de uso interactivo, es típicamente el usuario quien indica expresamente qué procesos iniciar.

1. **Largo plazo:** Decide qué procesos pueden pasar del disco a la memoria.
2. **Medio plazo:** Administra los procesos en memoria (puede suspender o reanudar) para optimizar el uso de recursos.
3. **Corto plazo:** Selecciona qué proceso activo en memoria se ejecuta en la CPU en cada instante.

Tipo de planificación	Objetivo principal	Frecuencia	Recursos involucrados
Largo plazo	Controlar entrada de procesos al sistema	Baja	Memoria principal y secundaria
Medio plazo	Controlar procesos activos y uso de memoria	Media	Memoria y CPU
Corto plazo	Seleccionar qué proceso ejecuta la CPU	Alta	CPU

Referencias y bibliografía de la sección.

- Silberschatz 5.
- Tanenbaum 2-4.

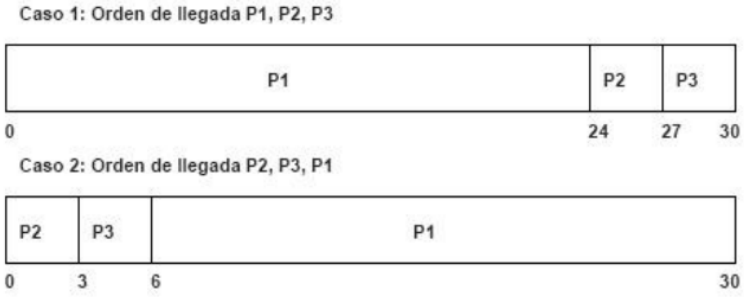
Algoritmos de planificación.

La cuestión de la planificación de la CPU aborda el problema de decidir a qué proceso de la cola de procesos preparados debe asignársele la CPU. Existen muchos algoritmos de planificación de la CPU; repasaremos algunos de ellos.

Planificación FCFS.

El algoritmo más simple de planificación de la CPU es el algoritmo ***FCFS*** (first come first served; primero en llegar, primero en ser servido). Con este esquema, se asigna primero la CPU al proceso que primero la solicite.

La implementación de la política FCFS se gestiona fácilmente con una cola ***FIFO***. Cuando un proceso entra en la cola de procesos preparados, su PCB se coloca al final de la cola FIFO.



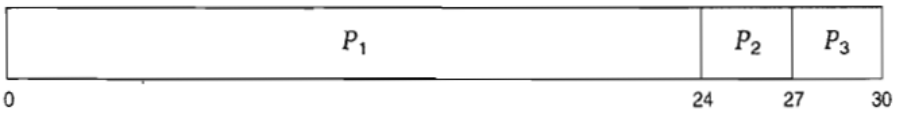
Cuando la CPU queda libre, se asigna al proceso que esté al principio de la cola, y ese proceso que pasa a ejecutarse se elimina de la cola.

El código del algoritmo de planificación FCFS es simple de escribir y fácil de comprender. Sin embargo, el tiempo medio de espera con el algoritmo FCFS es a menudo bastante largo.

Suponga que el siguiente conjunto de procesos llega en el instante 0, estando la duración de ráfaga de CPU especificada en milisegundos:

Proceso	Tiempo de ráfaga
P_1	24
P_2	3
P_3	3

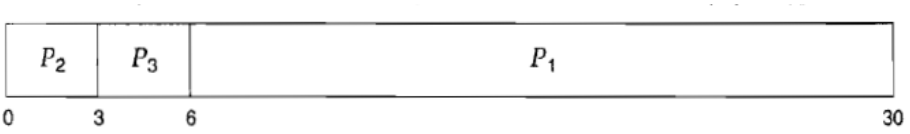
Si los procesos llegan en el orden P1, P2, P3, y se sirven según el orden FCFS, obtendremos resultado mostrado en el siguiente diagrama de Gantt[28]:



El tiempo de espera es de 10 milisegundos para el proceso P1, de 24 milisegundos para el proceso P2 y de 27 milisegundos para el proceso P3. Por tanto, el tiempo medio de espera es de (0 +24 +27)/3 = 17 milisegundos.

Sin embargo, si los procesos llegan en el orden P2, P3, P1, los resultados serán los mostrados en el siguiente diagrama de Gantt:

[28] Un diagrama de Gantt es una herramienta visual de gestión que utiliza barras horizontales para representar las tareas, sus fechas de inicio y finalización, y las dependencias entre ellas.



Ahora el tiempo de espera es de (6 + 0 + 3)/3 = 3 milisegundos. Esta reducción es sustancial. Por tanto, el tiempo medio de espera con una política FCFS no es, generalmente, mínimo y puede variar significativamente si la duración de las ráfagas de CPU de los procesos es muy variable.

El algoritmo de planificación FCFS es cooperativo[29]. Una vez que la CPU ha sido asignada a un proceso, dicho proceso conserva la CPU hasta que la libera, bien porque termina su ejecución o porque realiza una solicitud de E/S. El algoritmo FCFS resulta, por tanto, especialmente problemático en los sistemas de tiempo compartido, donde es importante que cada usuario obtenga una cuota de la CPU a intervalos regulares. Sería desastroso permitir que un proceso mantuviera la CPU durante un largo período de tiempo.

Planificación SJF.

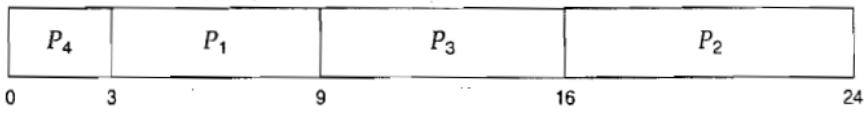
Otro método de planificación de la CPU es el algoritmo de planificación con selección del trabajo más corto (SJF, shortest job first). Este algoritmo asocia con cada proceso la duración de la siguiente ráfaga de CPU del proceso. Cuando la CPU está disponible, se asigna al proceso que tiene la siguiente ráfaga de CPU más corta. Si las siguientes ráfagas de CPU de dos procesos son iguales, se usa la planificación FCFS para romper el empate. Observemos que un término más apropiado para este método de planificación sería el de ***algoritmo de la siguiente ráfaga de CPU más corta***, ya que la planificación depende de la duración de la siguiente ráfaga de CPU de un proceso, en lugar de depender de su duración total.

Como ejemplo de planificación SJF, consideremos el siguiente conjunto de procesos, estando especificada la duración de la ráfaga de CPU en milisegundos:

Proceso	Tiempo de ráfaga
P_1	6
P_2	8
P_3	7
P_4	3

Usando SJF, planificaríamos estos procesos de acuerdo con el siguiente diagrama de Gantt:

[29] La planificación cooperativa permite que un nuevo proceso con menor prioridad se coloque al principio de la cola de procesos listos. En este caso, el proceso en ejecución permanece activo hasta que se complete o decida ceder el control.



El tiempo de espera es de 3 milisegundos para el proceso P1, de 16 milisegundos para el proceso P2, de 9 milisegundos para P3 y de 0 milisegundos para P4. Por tanto, el tiempo medio de espera es de (3 + 16 + 9 + 0)/4 = 7 milisegundos.

Por comparación, si estuviéramos usando el esquema de planificación FCFS, el tiempo medio de espera sería de 10,25 milisegundos.

El algoritmo de planificación SJF es probablemente óptimo, en el sentido de que proporciona el tiempo medio de espera mínimo para un conjunto de procesos dado. Anteponer un proceso corto a uno largo disminuye el tiempo de espera del proceso corto en mayor medida de lo que incrementa el tiempo de espera del proceso largo. Consecuentemente, el tiempo medio de espera disminuye.

La dificultad real del algoritmo SJF ***es conocer la duración de la siguiente solicitud de CPU***. En una planificación a largo plazo de trabajos en un sistema de procesamiento por lotes, podemos usar como duración el límite de tiempo del proceso que el usuario especifique en el momento de enviar el trabajo. Con este mecanismo, los usuarios están motivados para estimar el límite de tiempo del proceso de forma precisa, dado que un valor menor puede significar una respuesta más rápida (un valor demasiado bajo producirá un error de límite de tiempo excedido y será necesario reenviar el proceso).

La planificación SJF se usa frecuentemente como ***mecanismo de planificación a largo plazo***. Aunque el algoritmo SJF es óptimo, no se puede implementar en el nivel de la planificación la CPU a corto plazo, ya que no hay forma de conocer la duración de la siguiente ráfaga de CPU.

El algoritmo SJF puede ser cooperativo o apropiativo. La necesidad de elegir surge cuando un proceso llega a la cola de procesos preparados mientras que un proceso anterior está todavía en ejecución. La siguiente ráfaga de CPU del proceso que acaba de llegar puede ser más corta que lo que quede del proceso actualmente en ejecución.

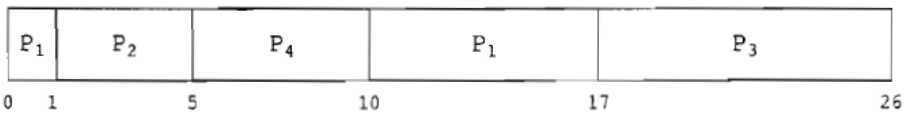
Un algoritmo SJF apropiativo detendrá el proceso actualmente en ejecución, mientras que un algoritmo sin desalojo permitirá que dicho proceso termine su ráfaga de CPU.

La planificación SJF apropiativa a veces se denomina planificación con selección del proceso con tiempo restante más corto.

Como ejemplo, considere los cuatro procesos siguientes, estando especificada la duración de la ráfaga de CPU en milisegundos:

Proceso	Tiempo de llegada	Tiempo de ráfaga
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

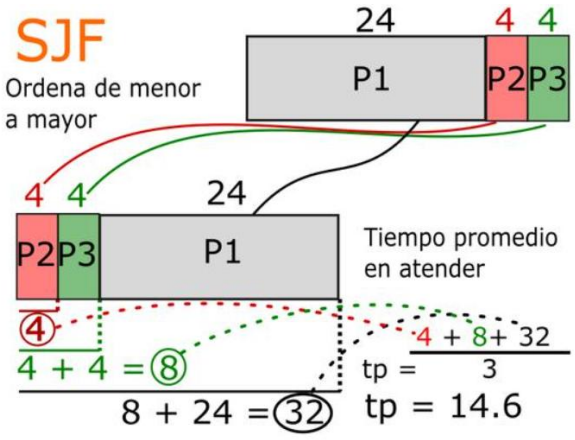
Si los procesos llegan a la cola de procesos preparados en los instantes que se muestran y necesitan los tiempos de ráfaga indicados, entonces la planificación SJF apropiativa es la que se muestra en el siguiente diagrama de Gantt:



El proceso P1 se inicia en el instante 0, dado que es el único proceso que hay en la cola. El proceso P2 llega en el instante 1. El tiempo que le queda al proceso P1 (7 milisegundos) es mayor que el tiempo requerido por el proceso P2 (4 milisegundos), por lo que el proceso P1 se desaloja y se planifica el proceso P2.

El tiempo medio de espera en este ejemplo es de ((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3) /4 = 26/4 = 6,5 milisegundos.

La planificación SJF cooperativa proporcionaría un tiempo medio de espera de 7,75 milisegundos.



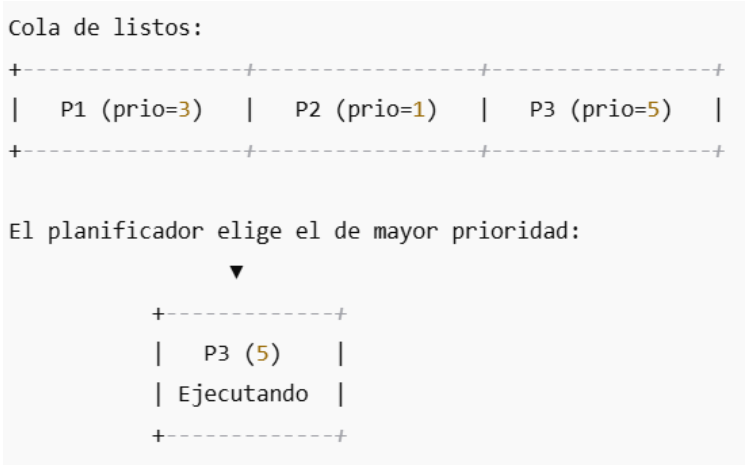
Planificación por prioridades.

El algoritmo SJF es un caso especial del algoritmo de planificación por prioridades general. A cada proceso se le asocia una prioridad y la CPU se asigna al proceso que tenga la prioridad más alta.

Los procesos con la misma prioridad se planifican en orden FCFS. Un algoritmo SJF es simplemente un algoritmo por prioridades donde la prioridad (p) es el inverso de la siguiente ráfaga de CPU (predicha).

Cuanto más larga sea la ráfaga de CPU, menor será la prioridad y viceversa.

Observemos que al hablar de planificación pensamos en términos de alta prioridad y baja prioridad. Generalmente, las prioridades se indican mediante un rango de números fijo, como por ejemplo de 0 a 7, o de 0 a 40%. Sin embargo, no existe un acuerdo general sobre si 0 es la prioridad más alta o la más baja.

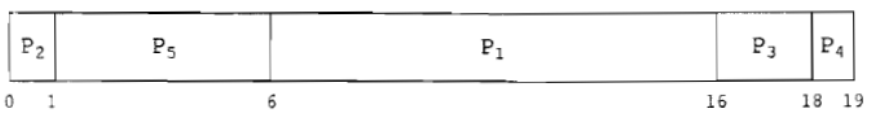


Algunos sistemas usan los números bajos para representar una prioridad baja; otros, emplean números bajos para especificar una prioridad alta; esta diferencia puede llevar a confusión. En nuestro caso, vamos a asumir que los números bajos representan una alta prioridad.

Por ejemplo, consideremos el siguiente conjunto de procesos y supongamos que llegan en el instante 0 en este orden: P1, P2, ..., P5, estando la duración de las ráfagas de CPU especificada en milisegundos:

Proceso	Tiempo de ráfaga	Prioridad
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Usando la planificación por prioridades, vamos a planificar estos procesos de acuerdo con el siguiente diagrama de Gantt:



El tiempo medio de espera es de 8,2 milisegundos.

Las prioridades pueden definirse interna o externamente. Las prioridades definidas internamente utilizan algún valor mensurable para calcular la prioridad de un proceso. Por ejemplo, para calcular las prioridades se han usado en diversos sistemas magnitudes tales como los límites de tiempo, los requisitos de memoria, el número de archivos abiertos y la relación entre la ráfaga de E/S promedio y la ráfaga de CPU promedio.

Las prioridades definidas externamente se establecen en función de criterios externos al sistema operativo, como por ejemplo la importancia del proceso, el coste monetario de uso de la computadora, el departamento que patrocina el trabajo y otros factores.

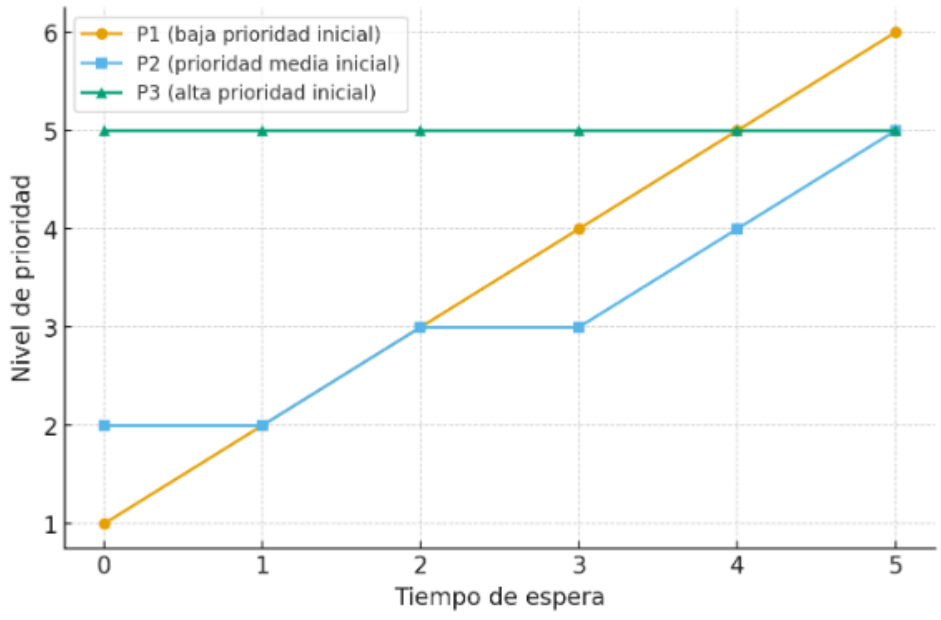
La planificación por prioridades puede ser apropiativa o cooperativa. Cuando un proceso llega a la cola de procesos preparados, su prioridad se compara con la prioridad del proceso actualmente en ejecución.

Un algoritmo de planificación por prioridades apropiativo, expulsará de la CPU al proceso actual si la prioridad del proceso que acaba de llegar es mayor. Un algoritmo de planificación por prioridades cooperativo simplemente pondrá el nuevo proceso al principio de la cola de procesos preparados.

Un problema importante de los algoritmos de planificación por prioridades es el bloqueo indefinido o la muerte por inanición. Un proceso que está preparado para ejecutarse, pero está esperando a acceder a la CPU puede considerarse bloqueado; un algoritmo de planificación por prioridades puede dejar a algunos procesos de baja prioridad esperando indefinidamente.

En un sistema informático con una carga de trabajo grande, un flujo estable de procesos de alta prioridad puede impedir que un proceso de baja prioridad llegue a la CPU. Generalmente, ocurrirá una de dos cosas: o bien el proceso se ejecutará finalmente (cuando el sistema finalmente tenga una menor carga de trabajo) o bien el sistema informático terminará fallando y se perderán todos los procesos con baja prioridad no terminados.

Una solución al problema del bloqueo indefinido de los procesos de baja prioridad consiste en aplicar mecanismos de envejecimiento.



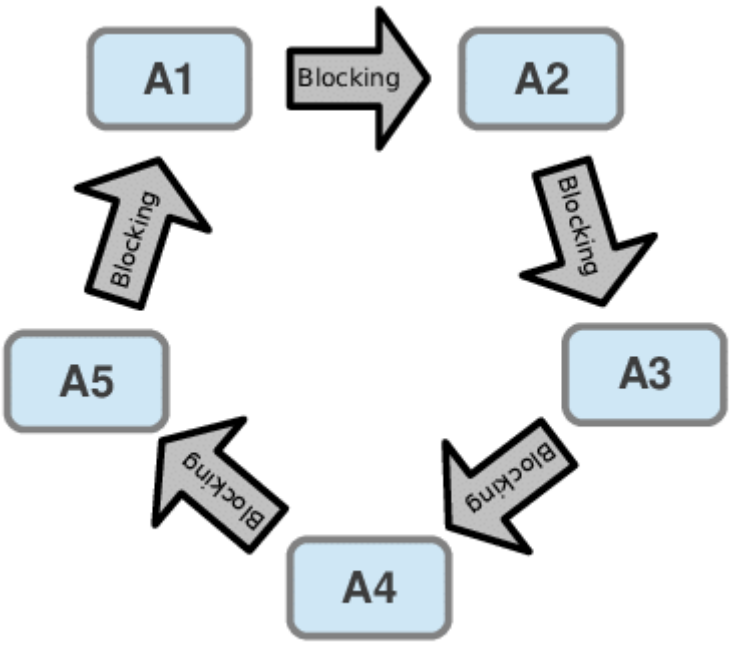
Esta técnica consiste en aumentar gradualmente la prioridad de los procesos que estén esperando en el sistema durante mucho tiempo. Por ejemplo, si el rango de prioridades va desde 127 (baja) a 0 (alta), podríamos restar 1 a la prioridad de un proceso en espera cada 15 minutos. Finalmente, incluso un proceso con una prioridad inicial de 127 llegaría a tener la prioridad más alta del sistema y podría ejecutarse. De hecho, un proceso con prioridad 127 no tardaría más de 32 horas en convertirse en un proceso con prioridad 0.

Planificación por turnos.

El algoritmo de planificación por turnos (RR, Round Robin) está diseñado especialmente para los sistemas de tiempo compartido. Es similar a la planificación FCFS, pero se añade la técnica de desalojo para conmutar entre procesos. En este tipo de sistema se define una pequeña unidad de tiempo, denominada cuanto de tiempo, franja temporal o ***quantum***. Generalmente, el quantum se encuentra en el rango comprendido entre 10 y 100 milisegundos.

La cola de procesos preparados se trata como una cola circular. El planificador de la CPU recorre la cola de procesos preparados, asignando la CPU a cada proceso durante un intervalo de tiempo de hasta 1 quantum.

Para implementar la planificación por turnos, mantenemos la cola de procesos preparados como una cola FIFO de procesos. Los procesos nuevos se añaden al final de la cola de procesos preparados.



El planificador de la CPU toma el primer proceso de la cola de procesos preparados, configura un temporizador para que interrumpa pasado 1 quantum y despacha el proceso.

Puede ocurrir una de dos cosas. El proceso puede tener una ráfaga de CPU cuya duración sea menor que 1 quantum; en este caso, el propio proceso liberará voluntariamente la CPU.

El planificador continuará entonces con el siguiente proceso de la cola de procesos preparados. En caso contrario, si la ráfaga de CPU del proceso actualmente en ejecución tiene una duración mayor que 1 quantum, se producirá un fin de cuenta del temporizador y éste enviará una interrupción al sistema operativo; entonces se ejecutará un cambio de contexto y el proceso se colocará al final de la cola de procesos preparados.

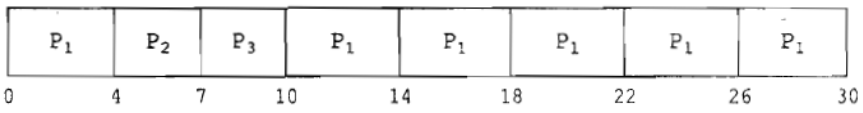
El planificador de la CPU seleccionará a continuación el siguiente proceso de la cola de procesos preparados.

El tiempo medio de espera en los sistemas por turnos es, con frecuencia, largo. Consideremos el siguiente conjunto de procesos que llegan en el instante 0, estando especificada la duración de las ráfagas de CPU en milisegundos:

Proceso	Tiempo de ráfaga
P_1	24
P_2	3
P_3	3

Si usamos un quantum de 4 milisegundos, entonces el proceso P1 obtiene los 4 primeros milisegundos. Dado que necesita otros 20 milisegundos, es desalojado después del primer quantum, y la CPU se concede al siguiente proceso de la cola, el proceso P2. Puesto que el proceso P2 no necesita 4 milisegundos, termina antes de que caduque su cuanto de tiempo. Entonces la CPU se proporciona al siguiente proceso, el proceso P3. Una vez que cada proceso ha recibido 1 quantum, la CPU es devuelta al proceso P1 para quantum adicional.

La planificación por tumos resultante es:

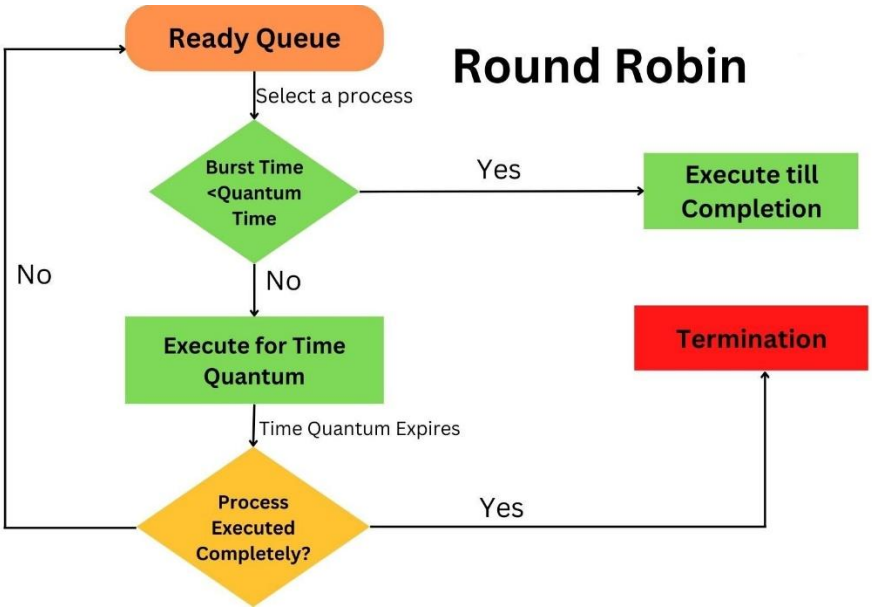


El tiempo medio de espera es de 17/3 = 5,66 milisegundos.

En el algoritmo de planificación por turnos, a ningún proceso se le asigna la CPU por más de 1 quantum en cada turno (a menos que sea el único proceso ejecutable). Si una ráfaga de CPU de un proceso excede 1 quantum, dicho proceso se desaloja y se coloca de nuevo en la cola de procesos preparados. El algoritmo de planificación por turnos incluye, por tanto, un mecanismo de desalojo.

El rendimiento del algoritmo de planificación por turnos depende enormemente del tamaño del quantum. Por un lado, si el quantum es extremadamente largo, la planificación por turnos

es igual que la FCFS. Si el cuanto de tiempo es muy pequeño (por ejemplo, 1 milisegundo), el método por turnos se denomina compartición del procesador y (en teoría) crea la apariencia de que cada uno de los n procesos tiene su propio procesador ejecutándose a 1/1 de la velocidad del procesador real.



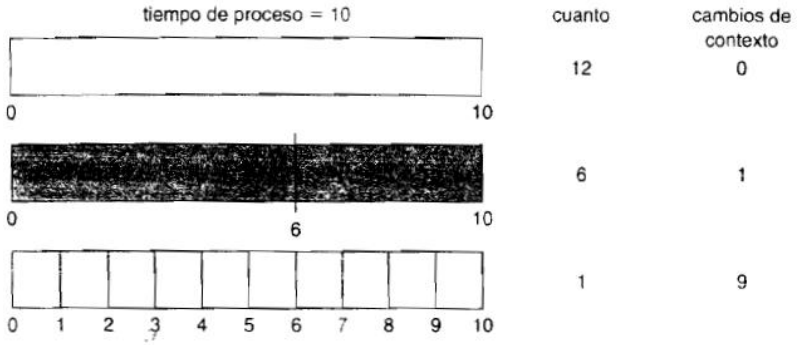
En software, también necesitamos considerar el efecto del cambio de contexto en el rendimiento de la planificación por turnos. Supongamos que sólo tenemos un proceso de 10 unidades de tiempo. Si el quantum tiene 12 unidades de tiempo, el proceso termina en menos de 1 quantum sin requerir ninguna carga de trabajo adicional de cambio de contexto. Sin embargo, si el quantum dura 6 unidades, el proceso requerirá 2 quantum, requiriendo un cambio de contexto, si el quantum es de 1 unidad, entonces se producirán nueve cambios de contexto, ralentizando correspondientemente la ejecución del proceso.

Planificación mediante colas multinivel.

Otra clase de algoritmos de planificación es la que se ha desarrollado para aquellas situaciones en las que los procesos pueden clasificarse fácilmente en grupos diferentes. Por ejemplo, una clasificación habitual consiste en diferenciar entre procesos de primer plano (interactivos) y procesos de segundo plano (por lotes).

Estos dos tipos de procesos tienen requisitos diferentes de tiempo de respuesta y, por tanto, pueden tener distintas necesidades de planificación. Además, los procesos de primer plano pueden tener prioridad (definida externamente) sobre los procesos de segundo plano.

Un algoritmo de planificación mediante colas multinivel divide la cola de procesos preparados en varias colas distintas. Los procesos se asignan permanentemente a una cola, generalmente en función de alguna propiedad del proceso, como por ejemplo el tamaño de memoria, la prioridad del proceso o el tipo de proceso. Cada cola tiene su propio algoritmo de planificación.



Por ejemplo, pueden emplearse colas distintas para los procesos de primer plano y de segundo plano. La cola de primer plano puede planificarse mediante un algoritmo por turnos, mientras que para la cola de segundo piano puede emplearse un algoritmo FCFS.

Además, debe definirse una planificación entre las colas, la cual suele implementarse como una planificación apropiativa y prioridad fija. Por ejemplo, la cola de procesos de primer plano puede tener prioridad absoluta sobre la cola de procesos de segundo plano.

Veamos un ejemplo de algoritmo de planificación mediante colas multinivel con las cinco colas que se enumeran a continuación, según su orden de prioridad:

1. Procesos del sistema.
2. Procesos interactivos.
3. Procesos de edición interactivos.
4. Procesos por lotes.
5. Procesos de estudiantes.

Cada cola tiene prioridad absoluta sobre las colas de prioridad más baja. Por ejemplo, ningún proceso de la cola por lotes podrá ejecutarse hasta que se hayan vaciado completamente las colas de los procesos del sistema, los procesos interactivos y los procesos de edición interactivos. Si un proceso de edición interactivo llega a la cola de procesos preparados mientras se está ejecutando un proceso por lotes, el proceso por lotes será desalojado.

Otra posibilidad consiste en repartir el tiempo entre las colas. En este caso, cada cola obtiene una cierta porción del tiempo de CPU, con la que puede entonces planificar sus distintos procesos.

Por ejemplo, en el caso de las colas de procesos de primer plano y segundo plano, la cola de primer plano puede disponer del 80 por ciento del tiempo de CPU para planificar por turnos sus procesos, mientras que la cola de procesos de segundo plano recibe el 20 por ciento del tiempo de CPU para gestionar sus procesos mediante el método FCFS.

Referencias y bibliografía de la sección.

- Silberschatz 5-3.
- Stallings 9-2 pág. 406.

Planificación de sistemas multiprocesador.

Hasta el momento, nos hemos centrado en los problemas de planificación de la CPU en un sistema con un solo procesador. Si hay disponibles múltiples CPU, se puede compartir la carga; sin embargo, ***el problema de la planificación se hace más complejo***.

Vamos a concentrarnos en los sistemas en los que los procesadores son idénticos, homogéneos en cuanto a su funcionalidad; en este tipo de sistemas, podemos usar cualquiera de los procesadores disponibles para ejecutar cualquier proceso de la cola.

Vale aclarar que incluso con múltiples procesadores homogéneos, en ocasiones hay limitaciones que afectan a la planificación; consideremos un sistema con un dispositivo de E/S conectado a un bus privado de un procesador. Los procesos que deseen emplear ese dispositivo deberán planificarse para ser ejecutados en dicho procesador.

Métodos de planificación en los sistemas multiprocesador.

Uno de los métodos para planificar las CPU en un sistema multiprocesador consiste en que todas las decisiones sobre la planificación, el procesamiento de E/S y otras actividades del sistema sean gestionadas por un mismo procesador, el servidor maestro. Los demás procesadores sólo ejecutan código de usuario. Este ***multiprocesamiento asimétrico*** resulta simple, porque sólo hay un procesador que accede a las estructuras de datos del sistema, reduciendo la necesidad de compartir datos.

Un segundo método utiliza el ***multiprocesamiento simétrico*** (SMP), en el que cada uno de los

procesadores se autoplanifica. Todos los procesos pueden estar en una cola común de procesos preparados, o cada procesador puede tener su propia cola privada de procesos preparado. Independientemente de esto, la planificación se lleva a cabo haciendo que el planificador de cada procesador examine la cola de procesos preparados y seleccione un proceso para ejecutarlo.

Si tenemos varios procesadores intentando acceder a una estructura de datos común para actualizarla, el planificador tiene que programarse cuidadosamente: tendremos que asegurar que dos procesadores no elegirán el mismo proceso y que no se perderán procesos de la cola.

Prácticamente todos los sistemas operativos modernos soportan el multiprocesamiento simétrico, incluyendo Windows, Solaris, Linux y Mac OS X.

Referencias y bibliografía de la sección.

- Silberschatz 5-4.

Hardware de multiprocesador.

Un **multiprocesador con memoria compartida** (o sólo multiprocesador) es un sistema de cómputo en el que dos o más CPUs comparten todo el acceso a una RAM común. Un programa que se ejecuta en cualquiera de las CPUs ve un espacio normal de direcciones virtuales (por lo general paginadas).

La única propiedad inusual que tiene este sistema es que la CPU puede escribir cierto valor en una palabra de memoria y después puede volver a leer esa palabra y obtener un valor distinto (tal vez porque otra CPU lo cambió). Si se organiza en forma correcta, esta propiedad forma la base de la comunicación entre procesadores: una CPU escribe ciertos datos en la memoria y otra lee esos datos.

En su mayor parte, los sistemas operativos multiprocesadores son sólo sistemas operativos regulares. Manejan las llamadas al sistema, administran la memoria, proveen un sistema de archivos y administran los dispositivos de E/S. Sin embargo, hay ciertas áreas en las que tienen características únicas. Estas áreas son la sincronización de procesos, la administración de recursos y la programación de tareas.

Aunque todos los multiprocesadores tienen la propiedad de que cada CPU puede direccionar toda la memoria, algunos tienen la característica adicional de que cada palabra de memoria se puede leer con la misma velocidad que cualquier otra palabra de memoria. Estas máquinas se conocen como multiprocesadores **UMA** (*Uniform Memory Access,* Acceso uniforme a la memoria).

Por el contrario, los multiprocesadores **NUMA** (*Non-uniform Memory Access,* Acceso no uniforme a la memoria) no tienen esta propiedad.

Multiprocesadores UMA con arquitecturas basadas en bus.

Los multiprocesadores más simples se basan en un solo bus. Dos o más CPUs y uno o más módulos de memoria utilizan el mismo bus para comunicarse. Cuando una CPU desea leer una palabra de memoria, primero comprueba si el bus está ocupado. Si está inactivo, la CPU coloca la dirección de la palabra que desea en el bus, declara unas cuantas señales de control y espera hasta que la memoria coloque la palabra deseada en el bus.

Si el bus está ocupado cuando una CPU desea leer o escribir en la memoria, la CPU espera sólo hasta que el bus esté inactivo. Aquí es donde se encuentra el problema con este diseño. Con dos o tres CPUs, la contención por el bus será manejable; con 32 o 64 será imposible. El sistema estará totalmente limitado por el ancho de banda del bus, y la mayoría de las CPUs estarán inactivas la mayor parte del tiempo.

La solución a este problema es agregar una caché a cada CPU. La caché puede estar dentro del chip de la CPU, a un lado de ésta, en el tablero del procesador o puede ser alguna combinación de las tres opciones anteriores. Como esto permite satisfacer muchas operaciones de lectura desde la caché local, habrá mucho menos tráfico en el bus y el sistema podrá soportar más CPUs.

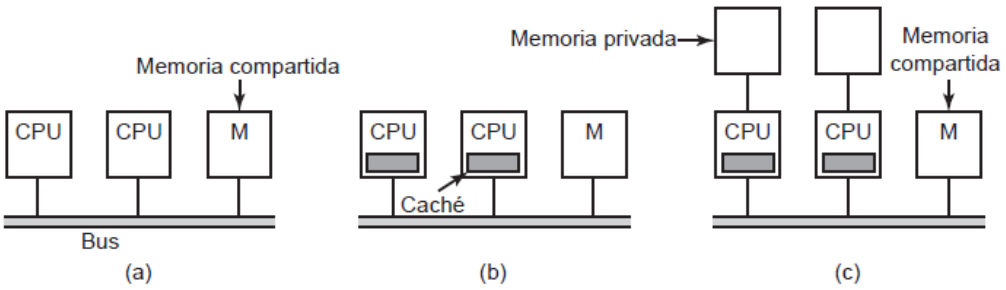


Ilustración 24 - Tres multiprocesadores basados en bus. (a) Sin caché. (b) Con caché. (c) Con caché y memorias privadas.

Multiprocesadores NUMA.

Los multiprocesadores UMA de un solo bus por lo general están limitados a no más de unas cuantas docenas de CPUs. Para poder tener más de 100 CPUs, hay que ceder algo. Por lo general, lo que cede es la idea de que todos los módulos de memoria tienen el mismo tiempo de acceso.

Esta concesión nos lleva a la idea de los multiprocesadores NUMA. Al igual que sus parientes UMA, proveen un solo espacio de direcciones a través de todas las CPUs, pero a diferencia de las máquinas UMA, el acceso a los módulos de memoria locales es más rápido que el acceso a los remotos. Por ende, todos los programas UMA se ejecutarán sin cambios en las máquinas NUMA, pero el rendimiento será peor que en una máquina UMA a la misma velocidad de reloj.

Las máquinas NUMA poseen tres características clave que, en conjunto, las distinguen de otros

multiprocesadores:

1. Hay un solo espacio de direcciones visible para todas las CPUs.
2. El acceso a la memoria remota es mediante instrucciones LOAD y STORE.
3. El acceso a la memoria remota es más lento que el acceso a la memoria local.

Chips multinúcleo.

A medida que mejora la tecnología de fabricación de chips, los transistores se están haciendo cada vez más pequeños y es posible colocar cada vez más de ellos en un chip.

Para aprovechar esta situación, una opción es agregar megabytes de caché al chip. Pero en cierto punto, incrementar el tamaño de la caché tal vez no mejore mucho el rendimiento de las aplicaciones.

La otra opción es colocar dos o más CPUs completas, a las que generalmente se les denomina ***núcleos***, en el mismo chip.

Aunque las CPUs pueden o no compartir cachés, siempre comparten la memoria principal, y ésta es consistente en el sentido de que siempre hay un valor único para cada palabra de memoria.

El resultado de este diseño es que los chips multinúcleo son sólo pequeños multiprocesadores. De hecho, a los chips multinúcleo se les conoce algunas veces como ***CMPs (Multiprocesadores a nivel de chip)***.

Desde la perspectiva del software, los CMPs no son en realidad tan distintos de los multiprocesadores basados en bus, o de los multiprocesadores que utilizan redes de conmutación.

Sin embargo, hay algunas diferencias. Para empezar, en un multiprocesador basado en bus cada CPU tiene su propia caché.

Otra área en la que los CMPs difieren de sus primos más grandes es la tolerancia a fallas. Como las CPUs tienen una conexión tan estrecha, las fallas en los componentes compartidos pueden afectar a varias CPUs a la vez, algo que es menos probable en los multiprocesadores tradicionales.

Además de los chips multinúcleo simétricos, en donde todos los núcleos son idénticos, hay otra categoría de chip multinúcleo conocida como ***sistema en un chip***. Estos chips tienen una o más CPUs principales, pero también tienen núcleos de propósito especial, como decodificadores de video y audio, criptoprocesadores, interfaces de red y demás, con lo cual se obtiene un sistema de cómputo completo en un chip.

Como ha ocurrido con frecuencia en el pasado, el hardware tiende a estar muy adelantado al software. Los lenguajes de programación deben adaptarse constantemente para escribir programas altamente paralelos.

La sincronización, la eliminación de las condiciones de carrera y la prevención del interbloqueo pueden ser una pesadilla para los programadores.

Referencias y bibliografía de la sección.

- Tanenbaum 8-1-1.